

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN ĐIỆN TỬ

BỘ MÔN ĐIỆN TỬ - THÔNG TIN



HCMUTE

ĐỒ ÁN TỐT NGHIỆP

**HỆ THỐNG NHẬN BIẾT PHƯƠNG TIỆN ĐI
SAI LÀN ĐƯỜNG DỰA TRÊN PHÂN TÍCH
VIDEO TỪ CAMERA GIAO THÔNG**

NGÀNH HỆ THỐNG NHÚNG VÀ IOT

GVHD : TS. Huỳnh Thế Thiện

SVTH : Huỳnh Ngọc Nhi

MSSV : 22139049

Khóa : 2022

TP. Hồ Chí Minh – 06/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN ĐIỆN TỬ - THÔNG TIN



HCMUTE

ĐỒ ÁN TỐT NGHIỆP

**HỆ THỐNG NHẬN BIẾT PHƯƠNG TIỆN ĐI
SAI LÀN ĐƯỜNG DỰA TRÊN PHÂN TÍCH
VIDEO TỪ CAMERA GIAO THÔNG**

NGÀNH HỆ THỐNG NHÚNG VÀ IOT

GVHD : TS. Huỳnh Thế Thiện

SVTH : Huỳnh Ngọc Nhi

MSSV : 22139049

Khóa : 2022

TP. Hồ Chí Minh – 06/2026

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: HỆ THỐNG NHẬN BIẾT PHƯƠNG TIỆN ĐI SAI LÀN ĐƯỜNG DỰA TRÊN PHÂN TÍCH VIDEO TỪ CAMERA GIAO THÔNG

Sinh viên: Huỳnh Ngọc Nhi

MSSV: 22139049

Hướng dẫn: TS. Huỳnh Thế Thiện

Nhận xét bao gồm các nội dung sau đây:

- Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:** Đề tài giải quyết một bài toán cấp thiết tại Việt Nam: giám sát giao thông hỗn hợp với mật độ xe máy cao. Việc kết hợp phân tích đối tượng và phân tích hình học mặt đường từ camera cố định là hướng tiếp cận có ý nghĩa khoa học và mang tính ứng dụng thực tiễn lớn đối với hệ thống giao thông thông minh.
- Phương pháp thực hiện/ phân tích/ thiết kế:** Sinh viên có tư duy thiết kế hệ thống tốt. Phương pháp kết hợp mạng học sâu (YOLOv11n) với theo dõi đối tượng (ByteTrack) và biến đổi không gian Bird's-Eye View (BEV) để mô hình hóa làn đường bằng đa thức bậc hai là hoàn toàn hợp lý, khoa học và sáng tạo.
- Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:** Hệ thống vận hành tốt, đáp ứng mục tiêu đề tài. Sinh viên đã giải quyết thành công các thách thức về che khuất, ổn định Track ID, và gán làn chính xác. Việc thực nghiệm, cấu hình lượng tử hóa INT8 và phân tích chi tiết hiệu năng trên thiết bị nhúng Raspberry Pi 5 thể hiện sự kiểm định chặt chẽ.
- Kết luận và đề xuất:** Phần kết luận đánh giá khách quan ưu/nhược điểm của hệ thống. Những đề xuất khắc phục (như bổ sung ground truth end-to-end, tối ưu nhận diện ban đêm) đều rất xác đáng và mở ra hướng phát triển khả thi.
- Hình thức trình bày và bố cục báo cáo:** Báo cáo có cấu trúc rất logic, rõ ràng theo đúng chuẩn mực khoa học. Sinh viên đã hiệu chỉnh cẩn thận các lỗi hình thức, công thức toán học và biểu đồ, mang lại một văn bản trau chuốt, mạch lạc và dễ theo dõi.
- Kỹ năng chuyên nghiệp và tính sáng tạo:** Sinh viên thể hiện khả năng làm việc độc lập tốt, kỹ năng lập trình vững vàng ở cả mảng huấn luyện AI lẫn tối ưu hóa phần cứng nhúng (Edge AI). Quá trình tổ chức mã nguồn và chia module pipeline chuyên nghiệp.
- Tài liệu trích dẫn:** Hệ thống tài liệu tham khảo phong phú, trích dẫn đúng quy định APA/IEEE. Các công trình học thuật nền tảng được chọn lọc kỹ lưỡng, phục vụ tốt cho lập luận trong bài.
- Đánh giá về sự trùng lặp của đề tài:** Đề tài mang tính độc lập cao, không trùng lặp với các công trình đã công bố. Bộ dữ liệu gán nhãn, phương pháp ổn định làn đường (Temporal Consensus) là nỗ lực cá nhân đáng ghi nhận.
- Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa:** Sinh viên đã khắc phục triệt để các sai sót về lỗi gõ công thức (Homography, Temporal Smoothing) và chuẩn hóa định dạng văn bản. Hiện tại, báo cáo đã hoàn thiện, không còn thiếu sót lớn trước khi bảo vệ.
- Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên:** Thái độ làm việc nghiêm túc, cầu thị. Sinh viên nỗ lực trong việc tự học hỏi các kiến thức mới về tối ưu mô hình, đáp ứng đầy đủ và vượt tiến độ các yêu cầu do GVHD đặt ra.

Đề nghị của giảng viên hướng dẫn:

Báo cáo hoàn toàn đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép bảo vệ khóa luận tốt nghiệp trước Hội đồng.

Tp. HCM, ngày 23 tháng 06 năm 2026

Người nhận xét
(Ký và ghi rõ họ tên)

TS. Huỳnh Thế Thiện

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ

THUẬT THÀNH PHỐ HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ

NGÀNH HỆ THỐNG NHÚNG VÀ IOT (ĐT)

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM

Độc lập – Tự do – Hạnh phúc

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Họ và tên sinh viên: Huỳnh Ngọc Nhi

MSSV: 22139049

1. Tên đề tài: Hệ thống nhận biết phương tiện đi sai làn đường dựa trên phân tích video từ camera giao thông.

2. Các số liệu, tài liệu ban đầu:

- Tài liệu về xử lý ảnh, YOLO11n, ByteTrack và nhận diện làn đường.
- Dữ liệu hình ảnh, video giao thông từ camera cố định.
- Mô hình YOLO11n đã được huấn luyện để phát hiện và phân loại phương tiện giao thông.
- Raspberry Pi 5 và các công cụ OpenVINO, ONNX.

3. Nội dung thực hiện đề tài:

- Huấn luyện mô hình phát hiện và phân loại phương tiện.
- Theo dõi phương tiện bằng ByteTrack.
- Xây dựng thuật toán nhận diện làn và gán phương tiện vào làn.
- Phát hiện phương tiện đi sai làn và đi ngược chiều.
- Đánh giá và triển khai thử nghiệm trên Raspberry Pi 5.

4. Sản phẩm:

- Phần mềm phân tích video và phát hiện vi phạm.
- Mô hình YOLO11n và mã nguồn chương trình.
- Video, số liệu thực nghiệm và báo cáo đồ án tốt nghiệp.

TRƯỞNG NGÀNH

GIẢNG VIÊN HƯỚNG DẪN

LỜI CAM ĐOAN

Em tên là: **Huỳnh Ngọc Nhi** – MSSV: **22139049**.

Em xin cam đoan đồ án tốt nghiệp với đề tài “Hệ thống nhận biết phương tiện đi sai làn đường dựa trên phân tích video giao thông từ camera giám sát” là công trình do cá nhân em nghiên cứu và thực hiện dưới sự hướng dẫn của TS. Huỳnh Thế Thiện.

Các nội dung, phương pháp, mã nguồn và kết quả thực nghiệm được trình bày trong báo cáo là trung thực. Những mô hình, thuật toán, thư viện, hình ảnh, số liệu và tài liệu được kế thừa từ các nguồn khác đều được trích dẫn rõ ràng theo quy định.

Em xin hoàn toàn chịu trách nhiệm trước Hội đồng bảo vệ đồ án và Nhà trường nếu có bất kỳ hành vi gian lận, sao chép hoặc sai phạm nào liên quan đến nội dung của đồ án.

TP. Hồ Chí Minh, ngày 22 tháng 6 năm 2026

Sinh viên thực hiện

Huỳnh Ngọc Nhi

LỜI CẢM ƠN

Em xin chân thành cảm ơn quý thầy cô Trường Đại học Sư phạm Kỹ thuật Thành phố Hồ Chí Minh, đặc biệt là quý thầy cô Khoa Điện – Điện tử, đã tận tình giảng dạy, truyền đạt kiến thức và tạo điều kiện thuận lợi cho em trong suốt quá trình học tập và thực hiện đồ án tốt nghiệp.

Em xin bày tỏ lòng biết ơn sâu sắc đến TS. Huỳnh Thế Thiện, giảng viên hướng dẫn, đã tận tình định hướng, góp ý chuyên môn và hỗ trợ em trong quá trình xây dựng hệ thống, thực hiện các thí nghiệm và hoàn thiện báo cáo. Những nhận xét và hướng dẫn quý báu của thầy đã giúp em nâng cao kiến thức, kỹ năng nghiên cứu và khả năng giải quyết vấn đề.

Em cũng xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn động viên, chia sẻ và đồng hành cùng em trong suốt thời gian học tập cũng như quá trình thực hiện đồ án.

Mặc dù đã cố gắng hoàn thành đề tài, báo cáo khó tránh khỏi những thiếu sót do hạn chế về kinh nghiệm và điều kiện thực nghiệm. Em kính mong nhận được những ý kiến đóng góp của quý thầy cô để đề tài được tiếp tục hoàn thiện.

Một lần nữa, em xin chân thành cảm ơn!

TÓM TẮT

Trong bối cảnh số lượng phương tiện giao thông ngày càng gia tăng, việc tự động phân tích video từ camera giám sát có ý nghĩa quan trọng đối với công tác quản lý và bảo đảm an toàn giao thông. Tuy nhiên, đặc điểm giao thông hỗn hợp tại Việt Nam, mật độ xe máy cao, tình trạng che khuất giữa các phương tiện và chất lượng vạch làn không đồng đều làm cho bài toán nhận biết phương tiện đi sai làn và đi ngược chiều trở nên phức tạp.

Đề án này nghiên cứu và xây dựng hệ thống nhận biết phương tiện đi sai làn đường dựa trên phân tích video giao thông từ camera giám sát cố định. Hệ thống được tổ chức thành một pipeline gồm phát hiện phương tiện, theo dõi đa đối tượng, nhận diện cấu trúc làn đường, gán phương tiện vào làn, xác định hướng chuyển động và xác nhận trạng thái vi phạm qua nhiều khung hình.

Mô hình YOLO11n được sử dụng để phát hiện bốn lớp phương tiện gồm xe máy, ô tô, xe tải và xe buýt. ByteTrack được tích hợp để duy trì Track ID và lưu lịch sử chuyển động của từng phương tiện. Đối với nhận diện làn đường, vùng mặt đường được chuyển sang không gian Bird's-Eye View nhằm giảm ảnh hưởng của phối cảnh. Hệ thống kết hợp thông tin màu sắc, độ sáng, tương phản và gradient để tạo ảnh nhị phân, sau đó phát hiện peak theo nhiều dải chiều sâu, liên kết các peak thành candidate boundary và mô hình hóa bằng đa thức bậc hai.

Các candidate được tinh chỉnh dựa trên pixel vạch sơn nằm trong hành lang tìm kiếm quanh đường cong ban đầu. Bộ lựa chọn mô hình đánh giá đồng thời các tổ hợp boundary dựa trên mức hỗ trợ của vạch sơn, chất lượng hình học, chiều rộng làn, thứ tự trái-phải và các ràng buộc topology. Hệ thống đồng thời hỗ trợ suy luận hai boundary ngoài khi mép đường không có vạch sơn rõ ràng. Temporal consensus và cơ chế lane locking được sử dụng để duy trì cấu trúc làn ổn định qua nhiều khung hình.

MỤC LỤC

LỜI CAM ĐOAN	i
LỜI CẢM ƠN	ii
TÓM TẮT	iii
DANH MỤC HÌNH	ix
DANH MỤC BẢNG	xii
DANH MỤC TỪ VIẾT TẮT	xiii
1 TỔNG QUAN	1
1.1 GIỚI THIỆU	1
1.2 MỤC TIÊU	3
1.3 PHƯƠNG PHÁP NGHIÊN CỨU	4
1.4 GIỚI HẠN NGHIÊN CỨU	5
1.5 BỐ CỤC	7
2 CƠ SỞ LÝ THUYẾT	9
2.1 Tổng quan về thị giác máy tính trong giám sát giao thông ..	9
2.1.1 Khái niệm thị giác máy tính	9
2.1.2 Đặc điểm dữ liệu video giao thông	10
2.1.3 Các bài toán chính trong giám sát giao thông bằng camera	12

2.2	Cơ sở phát hiện phương tiện giao thông	13
2.2.1	Khái niệm phát hiện đối tượng	13
2.2.2	Bounding box và điểm đại diện phương tiện	15
2.2.3	Intersection over Union, confidence và Non-Maximum Suppression	17
2.2.4	Mô hình YOLO trong phát hiện phương tiện	19
2.2.5	Tổng quan YOLO11 và biến thể YOLO11n	23
2.3	Cơ sở theo dõi đa đối tượng	25
2.3.1	Khái niệm theo dõi đa đối tượng	25
2.3.2	Tracking-by-detection, Kalman Filter và bài toán gán	26
2.3.3	Thuật toán ByteTrack	28
2.3.4	Vai trò của tracking trong phân tích vi phạm	31
2.4	Hình học ảnh và biểu diễn Bird's-Eye View	32
2.4.1	Vùng quan tâm ROI	32
2.4.2	Homography và Perspective Transform	33
2.4.3	Bird's-Eye View	34
2.5	Tiền xử lý ảnh phục vụ nhận diện làn	34
2.5.1	Mục đích và nguyên tắc kết hợp đặc trưng	34
2.5.2	Canny và Sobel Gradient	35
2.5.3	Không gian màu, CLAHE và Adaptive Threshold ..	36
2.5.4	Morphology và Connected Component Analysis ...	36
2.6	Trích xuất, mô hình hóa và ổn định đường biên làn	37
2.6.1	Histogram Projection và phân tích theo chiều sâu ..	37
2.6.2	Sliding Window và liên kết peak	38
2.6.3	Polynomial Fitting và tinh chỉnh đường cong	39

2.6.4	Đánh giá chất lượng boundary và topology	39
2.6.5	Temporal Smoothing và Persistence Map	41
2.6.6	Lane Boundary, Lane Polygon và biên ngoài	42
2.7	Gán phương tiện vào làn và phát hiện vi phạm	43
2.7.1	Biến đổi vị trí phương tiện và gán làn	43
2.7.2	Phát hiện phương tiện đi sai làn	44
2.7.3	Phát hiện phương tiện đi ngược chiều	45
2.7.4	Xác nhận vi phạm theo thời gian	47
2.8	Tối ưu hệ thống trên thiết bị nhúng	48
2.8.1	Các ràng buộc của thiết bị nhúng	48
2.8.2	OpenVINO và tối ưu suy luận	49
2.8.3	Lượng tử hóa INT8	50
2.8.4	Tối ưu ở cấp pipeline	52
2.9	Tổng kết chương	53
3	THIẾT KẾ HỆ THỐNG	55
3.1	Yêu cầu của hệ thống	55
3.1.1	Yêu cầu chức năng	55
3.1.2	Yêu cầu về độ chính xác và độ ổn định	56
3.1.3	Yêu cầu về hiệu năng và khả năng triển khai	58
3.2	Tổng quan kiến trúc hệ thống	59
3.2.1	Các khối chức năng	60
3.2.2	Tổ chức module phần mềm	61
3.2.3	Luồng xử lý một khung hình	63
3.3	Phương pháp đề xuất	66
3.3.1	Phát hiện, theo dõi và xác định điểm đại diện	66

3.3.2	Xác định ROI và xây dựng Bird's-Eye View	67
3.3.3	Trích xuất bằng chứng và candidate vạch làn	68
3.3.4	Mô hình hóa và lựa chọn cấu trúc làn	69
3.3.5	Ổn định và khóa mô hình làn	70
3.3.6	Gán làn và phát hiện vi phạm	71
3.3.7	Tối ưu lịch xử lý và profiling	72
3.4	Cài đặt thực nghiệm	73
3.4.1	Nguồn dữ liệu và bộ dữ liệu sử dụng	73
3.4.2	Quy trình xây dựng và gán nhãn dữ liệu	74
3.4.3	Cấu hình huấn luyện mô hình phát hiện	76
3.4.4	Cấu hình hệ thống và môi trường triển khai	78
3.4.5	Giao thức đánh giá	80
3.5	Tổng kết chương	82
4	KẾT QUẢ VÀ THẢO LUẬN	85
4.1	KẾT QUẢ HOẠT ĐỘNG CỦA HỆ THỐNG ĐỀ XUẤT ...	85
4.1.1	Kết quả huấn luyện mô hình YOLO11n	85
4.1.2	Kết quả phát hiện phương tiện	88
4.1.3	Kết quả theo dõi đa đối tượng	90
4.1.4	Kết quả nhận diện làn đường	92
4.1.4.1	Kết quả tiền xử lý	92
4.1.4.2	Kết quả trích xuất candidate và hiệu chỉnh đa thức	93
4.1.4.3	Kết quả lựa chọn và khóa mô hình làn	95
4.1.5	Kết quả gán phương tiện vào làn	98
4.1.6	Kết quả phát hiện vi phạm	99

4.2	ĐÁNH GIÁ HIỆU NĂNG VÀ KHẢ NĂNG TRIỂN KHAI	100
4.2.1	Hiệu năng của mô hình phát hiện trên GPU	100
4.2.2	So sánh hiệu năng trên máy tính phát triển và Raspberry Pi 5	101
4.3	THẢO LUẬN	106
4.4	TỔNG KẾT CHƯƠNG	112
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	115
5.1	KẾT LUẬN	115
5.2	HƯỚNG PHÁT TRIỂN	122
	TÀI LIỆU THAM KHẢO	126

Danh sách hình vẽ

2.1.1 Khung hình giao thông minh họa ảnh hưởng của phối cảnh, sự thay đổi kích thước phương tiện theo chiều sâu và hiện tượng che khuất trong giao thông hỗn hợp.	11
2.1.2 Pipeline tổng quan của hệ thống giám sát giao thông bằng camera, từ video đầu vào đến phát hiện, theo dõi, phân tích làn đường và xác định vi phạm.	13
2.2.1 Minh họa kết quả phát hiện nhiều phương tiện trong một ảnh, trong đó mỗi đối tượng được biểu diễn bằng bounding box và nhãn lớp.	15
2.2.2 Bounding box và điểm bottom-center được sử dụng làm điểm đại diện gần vị trí tiếp xúc giữa phương tiện và mặt đường.	17
2.2.3 Minh họa cách tính IoU giữa hai bounding box và nguyên lý hoạt động của Non-Maximum Suppression trong việc loại bỏ các dự đoán trùng lặp.	19
2.2.4 Sơ đồ khái quát kiến trúc YOLO gồm Backbone, Neck và Detection Head, với các feature map đa tỉ lệ phục vụ phát hiện đối tượng có kích thước khác nhau.	22

2.2.5 Sơ đồ khái quát kiến trúc YOLO11 gồm Backbone, Neck và Head, với quá trình trích xuất và hợp nhất đặc trưng ở nhiều tỉ lệ.	24
2.3.1 Quy trình tracking-by-detection gồm dự đoán vị trí track, xây dựng ma trận chi phí và gán detection cho track. Chi phí càng nhỏ biểu thị mức độ phù hợp càng cao giữa một track và một detection.	28
2.3.2 Minh họa ý tưởng của ByteTrack: chỉ sử dụng detection score cao có thể làm quỹ đạo bị đứt, trong khi việc tiếp tục liên kết với detection score thấp giúp duy trì track tốt hơn. ...	30
2.8.1 Quy trình dự kiến chuyển đổi, lượng tử hóa và đánh giá mô hình YOLO11n bằng OpenVINO trên thiết bị đích.....	52
3.2.1 Kiến trúc tổng thể của hệ thống, gồm nhánh phát hiện–theo dõi phương tiện, nhánh mô hình hóa làn và khối phân tích vi phạm.	60
3.2.2 Quan hệ gọi và trao đổi dữ liệu giữa các module phần mềm trong hệ thống.	62
3.2.3 Luồng xử lý một khung hình trong hệ thống, từ phát hiện và tracking đến cập nhật mô hình làn, gán Lane ID và xác nhận vi phạm.	65
3.3.1 Quá trình tích lũy candidate model, xây dựng temporal consensus và chuyển từ giai đoạn calibration sang trạng thái khóa.....	71
3.4.1 Một số điều kiện xuất hiện trong dữ liệu kiểm thử, gồm mật độ giao thông cao, vạch làn rõ, góc quan sát khác nhau và phương tiện có kích thước lớn.	74

3.4.2 Quy trình kiểm tra video, trích xuất khung hình, gán nhãn, kiểm tra nhãn và phân chia dữ liệu.	75
3.4.3 Các nhóm chỉ số dùng để đánh giá detection, tracking, lane detection, nhận biết vi phạm và hiệu năng hệ thống.	82
4.1.1 Sự thay đổi của các thành phần loss trong quá trình huấn luyện YOLO11n.	87
4.1.2 Các bước tiền xử lý ảnh phục vụ nhận diện làn đường.	93
4.1.3 Ví dụ hệ thống phát hiện và cảnh báo phương tiện đi sai làn dựa trên lớp phương tiện và Lane ID.	100
4.2.1 So sánh tốc độ xử lý của các cấu hình YOLO11n trên Raspberry Pi 5.	104
4.2.2 So sánh thời gian suy luận của các cấu hình YOLO11n trên Raspberry Pi 5.....	105

Danh sách bảng

3.4.1 Phân chia bộ dữ liệu phát hiện phương tiện.	76
3.4.2 Phân bố số đối tượng theo lớp và tập dữ liệu.	76
3.4.3 Cấu hình huấn luyện mô hình YOLO11n.	77
3.4.4 Các tham số chính của lane detector.	79
4.1.1 Thông tin cấu trúc và cấu hình huấn luyện YOLO11n.	86
4.1.2 Kết quả phát hiện phương tiện của YOLO11n trên validation.	89
4.1.3 Kết quả đánh giá thủ công khả năng duy trì Track ID.	91
4.1.4 Ảnh hưởng của candidate refinement đến sai số boundary. ...	95
4.1.5 Kết quả xác định cấu trúc làn đường.	96
4.1.6 Kết quả ổn định và khóa mô hình làn.	97
4.1.7 Kết quả gán phương tiện vào làn.	98
4.2.1 Thời gian xử lý của YOLO11n trên GPU NVIDIA Tesla T4.	101
4.2.2 Kích thước các định dạng mô hình sau khi export.	101
4.2.3 So sánh hiệu năng module phát hiện trên máy tính phát triển và Raspberry Pi 5.	103

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tên tiếng Anh	Ý nghĩa tiếng Việt
AP	Average Precision	Độ chính xác trung bình của một lớp đối tượng
BEV	Bird's-Eye View	Góc nhìn từ trên xuống
C2PSA	C2 with Position-Sensitive Attention	Khối C2 tích hợp cơ chế chú ý nhạy vị trí
CCTV	Closed-Circuit Television	Hệ thống camera giám sát
CLAHE	Contrast Limited Adaptive Histogram Equalization	Phương pháp cân bằng lược đồ xám thích nghi có giới hạn tương phản
CPU	Central Processing Unit	Bộ xử lý trung tâm
CUDA	Compute Unified Device Architecture	Kiến trúc tính toán song song của NVIDIA
FN	False Negative	Trường hợp có đối tượng hoặc vi phạm nhưng hệ thống không phát hiện được

Từ viết tắt	Tên tiếng Anh	Ý nghĩa tiếng Việt
FP	False Positive	Trường hợp hệ thống phát hiện nhầm đối tượng hoặc cảnh báo sai vi phạm
FP16	16-bit Floating-Point	Kiểu dữ liệu số thực dấu phẩy động 16 bit
FP32	32-bit Floating-Point	Kiểu dữ liệu số thực dấu phẩy động 32 bit
FPS	Frames Per Second	Số khung hình được xử lý trong một giây
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
HLS	Hue, Lightness, Saturation	Không gian màu gồm sắc độ, độ sáng và độ bão hòa
HOTA	Higher Order Tracking Accuracy	Chỉ số đánh giá tổng hợp chất lượng theo dõi đa đối tượng
HSV	Hue, Saturation, Value	Không gian màu gồm sắc độ, độ bão hòa và giá trị sáng
ID	Identifier	Mã định danh của đối tượng được theo dõi

Từ viết tắt	Tên tiếng Anh	Ý nghĩa tiếng Việt
IDF1	Identification F1-Score	Chỉ số F1 đánh giá khả năng duy trì chính xác mã định danh đối tượng
INT8	8-bit Integer	Kiểu dữ liệu số nguyên 8 bit
IoU	Intersection over Union	Tỷ lệ giữa diện tích phần giao và diện tích phần hợp của hai vùng
IR	Intermediate Representation	Định dạng biểu diễn trung gian của mô hình trong OpenVINO
LAB	CIELAB Color Space	Không gian màu CIELAB
mAP	Mean Average Precision	Giá trị trung bình của độ chính xác trung bình trên các lớp đối tượng
MOT	Multi-Object Tracking	Theo dõi đồng thời nhiều đối tượng
MOTA	Multiple Object Tracking Accuracy	Chỉ số đánh giá độ chính xác của quá trình theo dõi đa đối tượng

Từ viết tắt	Tên tiếng Anh	Ý nghĩa tiếng Việt
NMS	Non-Maximum Suppression	Phương pháp loại bỏ các khung bao phát hiện trùng lặp
NNCF	Neural Network Compression Framework	Bộ công cụ nén và lượng tử hóa mạng nơ-ron
ONNX	Open Neural Network Exchange	Định dạng trao đổi mô hình mạng nơ-ron
OpenVINO	Open Visual Inference and Neural Network Optimization	Bộ công cụ tối ưu hóa và triển khai mô hình suy luận của Intel
P50	50th Percentile	Phân vị thứ 50, tương ứng với giá trị trung vị
P95	95th Percentile	Phân vị thứ 95 của thời gian xử lý
PTQ	Post-Training Quantization	Phương pháp lượng tử hóa mô hình sau quá trình huấn luyện
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
ROI	Region of Interest	Vùng ảnh được lựa chọn để xử lý

Từ viết tắt	Tên tiếng Anh	Ý nghĩa tiếng Việt
SPPF	Spatial Pyramid Pooling – Fast	Khối gộp kim tự tháp không gian phiên bản nhanh
TP	True Positive	Trường hợp đối tượng hoặc vi phạm được hệ thống phát hiện chính xác
WL	Wrong Lane	Hành vi phương tiện đi sai làn đường
WW	Wrong Way	Hành vi phương tiện đi ngược chiều quy định
YOLO	You Only Look Once	Họ mô hình phát hiện đối tượng một giai đoạn

CHƯƠNG 1

TỔNG QUAN

1.1 GIỚI THIỆU

Sự phát triển nhanh chóng của đô thị hóa và nhu cầu đi lại đã làm gia tăng đáng kể số lượng phương tiện tham gia giao thông. Tại các tuyến đường đô thị ở Việt Nam, nhiều loại phương tiện như xe máy, ô tô, xe tải và xe buýt thường lưu thông đồng thời. Đặc điểm giao thông hỗn hợp làm cho quá trình giám sát và phát hiện vi phạm trở nên phức tạp, đặc biệt đối với các hành vi đi sai làn hoặc đi ngược chiều.

Phương tiện lưu thông không đúng làn quy định có thể gây xung đột giữa các nhóm phương tiện có kích thước và tốc độ khác nhau, làm gián đoạn dòng giao thông và tăng nguy cơ xảy ra tai nạn. Hiện nay, nhiều tuyến đường đã được trang bị camera giám sát, tuy nhiên phần lớn hệ thống mới chủ yếu thực hiện chức năng ghi hình và lưu trữ video. Việc phát hiện vi phạm vẫn phụ thuộc đáng kể vào quá trình quan sát trực tiếp hoặc xem lại dữ liệu bởi con người.

Khi số lượng camera và lượng video cần xử lý ngày càng lớn, phương pháp giám sát thủ công bộc lộ nhiều hạn chế. Người vận hành khó có thể theo dõi liên tục nhiều luồng camera trong thời gian dài. Trong điều kiện giao thông đông đúc, phương tiện có thể che khuất lẫn nhau, di chuyển nhanh hoặc thay đổi làn trong thời gian ngắn, làm tăng khả năng bỏ sót vi phạm. Vì vậy, việc xây dựng hệ thống có khả năng tự động phân tích video giao thông là cần thiết nhằm nâng cao hiệu quả khai thác hạ tầng camera

hiện có.

Sự phát triển của thị giác máy tính và học sâu đã tạo điều kiện cho việc tự động phát hiện, phân loại và theo dõi phương tiện trong video. Tuy nhiên, để xác định một phương tiện có đi sai làn hay không, hệ thống không chỉ cần nhận biết phương tiện mà còn phải xác định được cấu trúc làn đường, vị trí của phương tiện so với các đường biên làn và trạng thái của đối tượng qua nhiều khung hình liên tiếp.

Bài toán này gặp nhiều khó khăn do ảnh hưởng của phối cảnh, sự thay đổi kích thước phương tiện theo khoảng cách, tình trạng che khuất và sự không đồng đều của điều kiện chiếu sáng. Các vạch làn có thể bị mờ, đứt đoạn hoặc bị phương tiện che khuất. Bên cạnh đó, mép đường, vỉa hè, bóng cây và các vùng phản xạ cũng có thể tạo ra những đặc trưng tương tự vạch làn, làm tăng nguy cơ nhận diện sai cấu trúc mặt đường.

Từ những vấn đề trên, đề tài tập trung xây dựng một hệ thống nhận biết phương tiện đi sai làn dựa trên video giao thông từ camera cố định. Hệ thống kết hợp mô hình phát hiện phương tiện, thuật toán theo dõi đa đối tượng, phương pháp mô hình hóa làn đường và cơ chế xác nhận vi phạm theo thời gian. Bốn lớp phương tiện được xem xét gồm xe máy, ô tô, xe tải và xe buýt.

Trọng tâm kỹ thuật của đề tài là xây dựng mô hình làn có khả năng hoạt động ổn định trong điều kiện camera cố định. Thay vì đưa ra quyết định từ một khung hình riêng lẻ, hệ thống tích lũy thông tin qua nhiều khung hình để giảm ảnh hưởng của nhiễu, che khuất và biến động ánh sáng. Kết quả mô hình làn được kết hợp với lịch sử di chuyển của từng phương tiện để thực hiện gán làn và xác nhận trạng thái vi phạm.

Bên cạnh yêu cầu về độ chính xác, đề tài cũng quan tâm đến hiệu năng tính toán và khả năng triển khai trên thiết bị nhúng. Pipeline được thiết kế theo kiến trúc module, cho phép đo thời gian của từng thành phần, giảm các phép tính không cần thiết và tạo cơ sở cho việc tối ưu trên Raspberry Pi 5.

1.2 MỤC TIÊU

Mục tiêu tổng quát của đề tài là nghiên cứu, thiết kế và đánh giá một hệ thống tự động nhận biết phương tiện đi sai làn từ video giao thông của camera cố định. Hệ thống cần có khả năng phát hiện và theo dõi phương tiện, xác định cấu trúc làn đường, gán phương tiện vào làn, phân tích trạng thái vi phạm theo thời gian và có khả năng tiếp tục tối ưu để triển khai trên thiết bị nhúng.

Các mục tiêu cụ thể của đề tài gồm:

- Xây dựng mô hình phát hiện và phân loại bốn lớp phương tiện gồm xe máy, ô tô, xe tải và xe buýt trong video giao thông.
- Tích hợp thuật toán theo dõi đa đối tượng nhằm duy trì track ID và xây dựng lịch sử chuyển động của từng phương tiện.
- Xây dựng phương pháp tự động mô hình hóa các đường biên làn bên trong vùng quan tâm của camera cố định.
- Xác định lane index của từng phương tiện và ổn định kết quả gán làn qua nhiều khung hình.
- Phát hiện phương tiện đi sai làn theo loại phương tiện và hỗ trợ xác

định phương tiện di chuyển ngược chiều.

- Đánh giá độ chính xác của các thành phần phát hiện, tracking, nhận diện làn và phân tích vi phạm.
- Đo lường hiệu năng của toàn bộ pipeline và khảo sát khả năng triển khai trên Raspberry Pi 5.

Thông qua các mục tiêu trên, đề tài hướng tới xây dựng một hệ thống có khả năng hoạt động tự động, có thể giải thích được các giai đoạn xử lý và phù hợp với đặc điểm của video giao thông tại Việt Nam.

1.3 PHƯƠNG PHÁP NGHIÊN CỨU

Nghiên cứu được thực hiện bằng cách kết hợp giữa khảo sát lý thuyết, xây dựng dữ liệu, phát triển hệ thống phần mềm, thực nghiệm trên video giao thông và phân tích kết quả.

Nghiên cứu lý thuyết: Các tài liệu liên quan đến phát hiện đối tượng, theo dõi đa đối tượng, xử lý ảnh làn đường, biến đổi phối cảnh, mô hình hóa đường cong và tối ưu mô hình trên thiết bị nhúng được khảo sát. Kết quả tổng quan được sử dụng để lựa chọn các phương pháp phù hợp với bài toán và điều kiện triển khai của đề tài.

Xây dựng dữ liệu và huấn luyện mô hình: Dữ liệu ảnh và video giao thông được thu thập, kiểm tra và gán nhãn theo bốn lớp phương tiện. Mô hình phát hiện được huấn luyện, kiểm chứng và đánh giá trên các tập dữ liệu độc lập. Việc phân chia dữ liệu được thực hiện theo video hoặc đoạn video để hạn chế rò rỉ các khung hình gần giống nhau giữa các tập dữ liệu.

Thiết kế hệ thống: Hệ thống được xây dựng theo kiến trúc module,

gồm các thành phần thu nhận video, phát hiện phương tiện, tracking, lựa chọn vùng quan tâm, nhận diện làn, gán làn, phân tích vi phạm, hiển thị kết quả và đo hiệu năng. Các module trao đổi dữ liệu thông qua một pipeline thống nhất.

Thực nghiệm và cải tiến: Các phiên bản thuật toán được chạy trên nhiều video giao thông có điều kiện khác nhau. Các kết quả trung gian được sử dụng để xác định nguyên nhân của các trường hợp phát hiện thiếu làn, lựa chọn sai đường biên, dao động lane index hoặc mất track ID. Thuật toán được điều chỉnh tuần tự và được so sánh trên cùng dữ liệu đầu vào.

Đánh giá hệ thống: Mô hình phát hiện được đánh giá bằng Precision, Recall và mAP. Khả năng tracking được xem xét thông qua độ ổn định của track ID. Nhận diện làn được đánh giá dựa trên số lượng đường biên, vị trí đường biên, số làn và độ ổn định theo thời gian. Kết quả phát hiện vi phạm được đánh giá ở cấp phương tiện bằng các chỉ số TP, FP, FN, Precision, Recall và F1-score.

Đánh giá hiệu năng: Thời gian xử lý của từng module, tốc độ khung hình, mức sử dụng CPU và RAM được ghi nhận. Kết quả này được sử dụng để xác định các nút thắt tính toán và đề xuất phương án tối ưu trước khi triển khai trên Raspberry Pi 5.

1.4 GIỚI HẠN NGHIÊN CỨU

Đề tài được thực hiện trong một phạm vi xác định nhằm bảo đảm tính tập trung và phù hợp với thời gian nghiên cứu.

Nguồn dữ liệu đầu vào: Hệ thống sử dụng video giao thông từ camera cố định. Camera được giả định không thay đổi vị trí, góc quay hoặc

tiêu cự trong quá trình xử lý. Camera gắn trên phương tiện hoặc camera bị rung mạnh không thuộc phạm vi nghiên cứu chính.

Vùng quan tâm: ROI được thiết lập cho từng vị trí camera nhằm giới hạn vùng mặt đường cần xử lý. Đề tài không tập trung giải quyết bài toán tự động xác định ROI trong mọi bối cảnh. Sau khi ROI được cung cấp, các đường biên làn bên trong vùng này được hệ thống tự động nhận diện.

Loại phương tiện: Hệ thống tập trung vào bốn nhóm phương tiện gồm xe máy, ô tô, xe tải và xe buýt. Các đối tượng như người đi bộ, xe đạp và phương tiện chuyên dụng chưa được xử lý đầy đủ.

Loại vi phạm: Đề tài tập trung vào hành vi đi sai làn theo loại phương tiện và hỗ trợ phát hiện phương tiện đi ngược chiều. Các vi phạm khác như vượt đèn đỏ, chạy quá tốc độ, dừng đỗ sai quy định hoặc không đội mũ bảo hiểm không thuộc phạm vi nghiên cứu.

Điều kiện môi trường: Hệ thống được đánh giá chủ yếu trong điều kiện ban ngày và ban đêm có chiều sáng đường phố. Các trường hợp mưa lớn, sương mù dày, ngập nước, ánh sáng quá yếu hoặc phản xạ mặt đường nghiêm trọng chưa được xử lý đầy đủ.

Cấu trúc làn đường: Phương pháp phù hợp với các đoạn đường thẳng hoặc cong nhẹ có cấu trúc làn tương đối ổn định. Hệ thống có thể gặp khó khăn khi vạch làn bị xóa hoàn toàn, bị che khuất trong thời gian dài hoặc tại các khu vực nhập làn, tách làn và giao cắt phức tạp.

Quy tắc làn đường: Tập phương tiện được phép lưu thông trên từng làn được cấu hình theo bối cảnh thử nghiệm. Khi áp dụng trong thực tế, các quy tắc này cần được xác định dựa trên biển báo và quy định cụ thể của từng tuyến đường.

Kết luận vi phạm: Kết quả của hệ thống được xem là thông tin hỗ trợ giám sát. Đề tài không thực hiện nhận dạng biển số, xác định danh tính người điều khiển hoặc tự động đưa ra quyết định xử phạt.

Nền tảng triển khai: Raspberry Pi 5 được lựa chọn làm thiết bị mục tiêu. Hiệu năng thực tế phụ thuộc vào mô hình suy luận, độ phân giải video, số lượng phương tiện, chế độ hiển thị và điều kiện tản nhiệt. Đề tài không khẳng định một mức FPS cụ thể khi chưa có kết quả thực nghiệm tương ứng.

1.5 BỐ CỤC

Luận văn được tổ chức thành năm chương:

- **Chương 1 – Tổng quan:** Trình bày bối cảnh, tính cấp thiết, mục tiêu, phương pháp nghiên cứu, giới hạn và bố cục của đề tài.
- **Chương 2 – Cơ sở lý thuyết:** Trình bày các kiến thức nền tảng về phát hiện đối tượng, theo dõi đa đối tượng, hình học ảnh, nhận diện làn đường, phân tích vi phạm và tối ưu trên thiết bị nhúng.
- **Chương 3 – Thiết kế hệ thống:** Trình bày kiến trúc tổng thể, tổ chức phần mềm, phương pháp nhận diện làn, gán phương tiện vào làn và xác nhận trạng thái vi phạm.
- **Chương 4 – Kết quả và thảo luận:** Trình bày dữ liệu thực nghiệm, cấu hình huấn luyện, kết quả phát hiện phương tiện, nhận diện làn, phân tích vi phạm và đánh giá hiệu năng.
- **Chương 5 – Kết luận và hướng phát triển:** Tổng kết các kết quả đạt

được, phân tích hạn chế và đề xuất hướng cải tiến trong tương lai.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về thị giác máy tính trong giám sát giao thông

2.1.1 Khái niệm thị giác máy tính

Thị giác máy tính là lĩnh vực nghiên cứu các phương pháp giúp máy tính thu nhận, xử lý, phân tích và diễn giải thông tin từ hình ảnh hoặc video[1]. Trong hệ thống giám sát giao thông, thị giác máy tính cho phép camera không chỉ ghi lại hiện trường mà còn trích xuất các thông tin có ý nghĩa như vị trí phương tiện, lớp phương tiện, làn đường, quỹ đạo chuyển động và trạng thái vi phạm.

Đối với con người, việc quan sát một đoạn video và nhận biết phương tiện nào đang đi đúng làn hoặc sai làn là một quá trình trực quan. Đối với máy tính, mỗi khung hình chỉ là một ma trận điểm ảnh. Ý nghĩa về phương tiện, mặt đường và hành vi phải được hình thành thông qua nhiều lớp xử lý, từ tăng cường ảnh, phát hiện đối tượng, theo dõi chuyển động đến suy luận hình học và phân tích theo thời gian.

Bài toán nhận biết phương tiện đi sai làn không thể được giải quyết bằng một thuật toán đơn lẻ. Hệ thống phải phát hiện phương tiện trong từng khung hình, duy trì định danh qua nhiều khung hình, nhận diện cấu trúc làn, xác định quan hệ giữa phương tiện và làn, sau đó kiểm tra trạng thái này theo thời gian. Vì vậy, bài toán mang đồng thời đặc trưng của nhận thức thị giác, hình học ảnh và phân tích chuỗi thời gian.

2.1.2 Đặc điểm dữ liệu video giao thông

Video giao thông là chuỗi các khung hình được ghi liên tiếp theo thời gian. Mỗi khung hình cung cấp thông tin không gian về mặt đường, vạch làn, phương tiện và môi trường quan sát. Trong khi đó, sự thay đổi giữa các khung hình liên tiếp cung cấp thông tin thời gian về vị trí, vận tốc tương đối, hướng chuyển động và trạng thái của từng đối tượng. Nhờ kết hợp hai nguồn thông tin này, hệ thống có thể xây dựng quỹ đạo, duy trì nhận dạng phương tiện và đánh giá các hành vi kéo dài theo thời gian.

Dữ liệu từ camera giám sát giao thông thường được thu nhận ở góc nhìn từ trên cao và nhìn xiên xuống mặt đường. Góc quan sát này làm xuất hiện biến dạng phối cảnh, trong đó các đường song song trong thực tế có xu hướng hội tụ trong ảnh, khoảng cách biểu kiến giữa các làn thay đổi theo chiều sâu và kích thước phương tiện giảm dần khi phương tiện ở xa camera. Vì vậy, khoảng cách và tọa độ đo trực tiếp trong ảnh không phản ánh đầy đủ khoảng cách hình học trên mặt đường thực tế nếu chưa có bước hiệu chỉnh phối cảnh.

Hình 2.1.1 minh họa rõ ảnh hưởng của phối cảnh trong một cảnh giao thông hỗn hợp. Các phương tiện ở gần camera có kích thước lớn hơn đáng kể so với phương tiện ở xa, trong khi các vạch làn có xu hướng hội tụ về phía đường chân trời. Mật độ xe máy cao cũng làm xuất hiện nhiều trường hợp phương tiện nằm sát nhau hoặc che khuất lẫn nhau.



Hình 2.1.1: Khung hình giao thông minh họa ảnh hưởng của phối cảnh, sự thay đổi kích thước phương tiện theo chiều sâu và hiện tượng che khuất trong giao thông hỗn hợp.

Ngoài biến dạng phối cảnh, video giao thông còn chịu ảnh hưởng của nhiều yếu tố bất lợi. Che khuất xảy ra khi một phương tiện bị phương tiện khác che một phần hoặc toàn bộ, khiến detector có thể bỏ sót đối tượng hoặc tracker bị gián đoạn ID. Nhòe chuyển động xuất hiện khi phương tiện di chuyển nhanh hoặc thời gian phơi sáng của camera không phù hợp, làm giảm độ rõ của biên và đặc trưng hình dạng.

Điều kiện ánh sáng cũng có thể thay đổi theo thời gian trong ngày, thời tiết và môi trường xung quanh. Bóng đổ, phản xạ mặt đường, ánh sáng đèn xe và sự chênh lệch độ sáng giữa các vùng ảnh có thể làm giảm độ tương phản của vạch làn hoặc tạo ra các vùng sáng bị nhận nhầm là vạch sơn. Bên cạnh đó, vạch làn có thể bị mờ, đứt đoạn, che khuất hoặc suy giảm chất lượng do quá trình sử dụng mặt đường.

Trong cảnh giao thông hỗn hợp, nhiều loại phương tiện như xe máy,

ô tô, xe tải và xe buýt cùng xuất hiện trong một vùng quan sát. Xe máy thường có kích thước nhỏ, mật độ cao và khoảng cách giữa các phương tiện hẹp, làm tăng khả năng chồng lấn bounding box, bỏ sót detection và thay đổi Track ID. Các phương tiện lớn có thể che khuất xe máy hoặc che khuất một phần vạch làn trong nhiều khung hình liên tiếp.

Vì vậy, hệ thống giám sát giao thông không nên chỉ dựa vào kết quả của một khung hình độc lập. Thông tin không gian được sử dụng để xác định phương tiện, vạch làn và quan hệ giữa phương tiện với mặt đường tại từng thời điểm. Thông tin thời gian được sử dụng để duy trì Track ID, xây dựng lịch sử chuyển động, ổn định kết quả nhận diện làn và tránh kết luận vi phạm từ một quan sát nhiều đơn lẻ.

Những đặc điểm trên là cơ sở để hệ thống sử dụng vùng quan tâm ROI, phép biến đổi phối cảnh, thuật toán theo dõi đa đối tượng và cơ chế tích lũy thông tin qua nhiều khung hình.

2.1.3 Các bài toán chính trong giám sát giao thông bằng camera

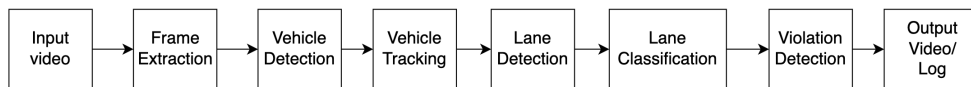
Một hệ thống giám sát giao thông bằng camera thường bao gồm các bài toán chính: trích xuất khung hình, phát hiện phương tiện, theo dõi đa đối tượng, nhận diện làn đường, gán phương tiện vào làn, ước lượng hướng chuyển động và phân tích vi phạm. Các bài toán này tạo thành một chuỗi xử lý có quan hệ phụ thuộc, trong đó kết quả của bước trước được sử dụng làm đầu vào cho bước tiếp theo.

Đầu tiên, video đầu vào được tách thành các khung hình liên tiếp để phục vụ quá trình xử lý. Khối phát hiện phương tiện xác định vị trí, lớp đối tượng và độ tin cậy của từng phương tiện trong mỗi khung hình. Sau đó,

thuật toán theo dõi đa đối tượng liên kết các kết quả phát hiện qua thời gian và duy trì một Track ID ổn định cho từng phương tiện.

Song song với quá trình phát hiện và theo dõi phương tiện, khối nhận diện làn đường xác định các đường biên và vùng làn trong khu vực mặt đường. Kết quả này được sử dụng để gán từng phương tiện vào làn tương ứng dựa trên vị trí của phương tiện so với các boundary đã phát hiện.

Lịch sử vị trí của từng Track ID được sử dụng để ước lượng hướng chuyển động. Cuối cùng, lớp phương tiện, Lane ID, hướng di chuyển và luật giao thông được kết hợp để xác định các hành vi vi phạm như đi sai làn hoặc đi ngược chiều. Kết quả được hiển thị trên video đầu ra và có thể được lưu dưới dạng log để phục vụ thống kê hoặc kiểm tra lại.



Hình 2.1.2: Pipeline tổng quan của hệ thống giám sát giao thông bằng camera, từ video đầu vào đến phát hiện, theo dõi, phân tích làn đường và xác định vi phạm.

2.2 Cơ sở phát hiện phương tiện giao thông

2.2.1 Khái niệm phát hiện đối tượng

Phát hiện đối tượng là bài toán xác định đồng thời vị trí và lớp của các đối tượng xuất hiện trong ảnh[2]. Khác với phân loại ảnh, trong đó toàn bộ ảnh thường được gán một hoặc một số nhãn, object detection trả về một tập các dự đoán riêng biệt. Mỗi dự đoán thường gồm bounding box, nhãn lớp và confidence score.

Bounding box là hình chữ nhật bao quanh đối tượng và có thể được biểu diễn bằng bốn tọa độ $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$, hoặc bằng tọa độ tâm cùng chiều rộng và chiều cao. Nhãn lớp cho biết loại đối tượng được phát hiện, trong khi confidence score biểu diễn mức độ tin cậy của mô hình đối với dự đoán đó. Các detection có confidence thấp thường được loại bỏ bằng một ngưỡng xác định trước.

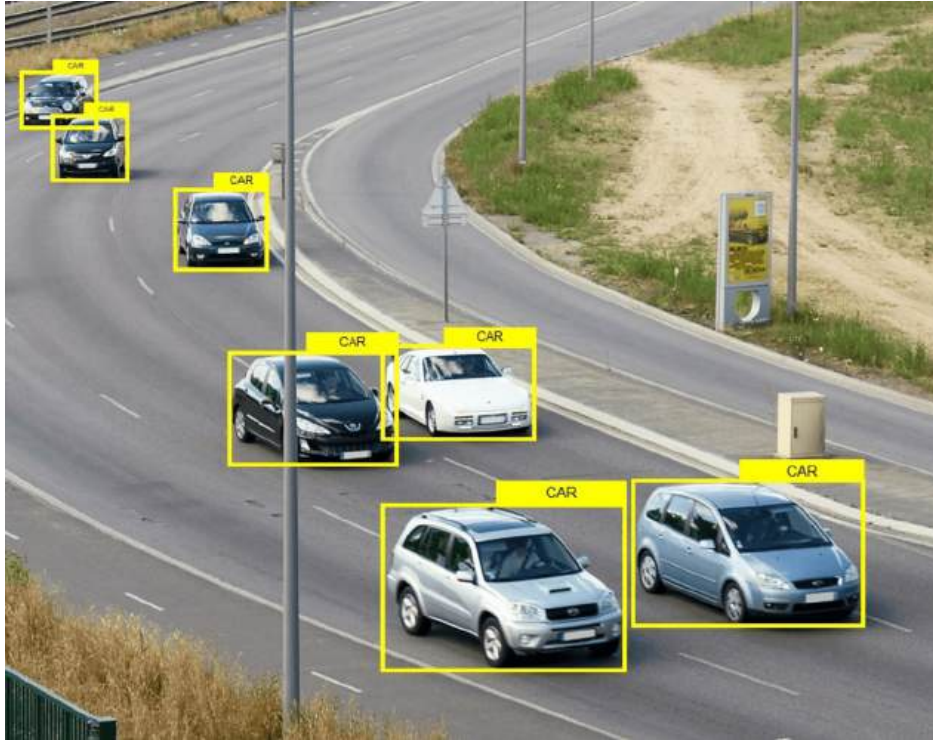
Trong bài toán giao thông, đối tượng có thể là xe máy, ô tô, xe tải, xe buýt hoặc người đi bộ. Phạm vi đề tài tập trung vào bốn lớp phương tiện gồm *motorbike*, *car*, *truck* và *bus*. Kết quả phát hiện được sử dụng làm đầu vào cho thuật toán tracking, đồng thời cung cấp nhãn lớp và vị trí của phương tiện cho các bước gán làn và phân tích vi phạm.

Đối với mỗi bounding box, hệ thống sử dụng điểm giữa cạnh dưới làm điểm đại diện của phương tiện. Điểm này thường gần vị trí tiếp xúc giữa phương tiện và mặt đường hơn tâm bounding box, do đó phù hợp hơn cho việc xác định phương tiện đang nằm trong làn nào.

Chất lượng phát hiện đối tượng được đánh giá không chỉ dựa trên khả năng phân loại đúng mà còn dựa trên độ chính xác định vị. Một chỉ số thường được sử dụng là Intersection over Union (IoU), được tính bằng tỷ lệ giữa diện tích phần giao và diện tích phần hợp của bounding box dự đoán với bounding box tham chiếu. Giá trị IoU càng cao cho thấy bounding box dự đoán càng gần với vị trí thực tế của đối tượng [3].

Sai số của khối detection có thể lan truyền đến các giai đoạn xử lý sau. Nếu phương tiện bị bỏ sót, tracker không thể khởi tạo hoặc duy trì quỹ đạo, từ đó hệ thống không thể đánh giá hành vi vi phạm. Nếu bounding box bị lệch, điểm đại diện có thể bị gán sang làn bên cạnh. Nếu nhãn lớp bị

nhận sai, hệ thống có thể áp dụng sai quy tắc phương tiện được phép trong từng làn. Vì vậy, độ chính xác phân loại và độ chính xác định vị đều có ảnh hưởng trực tiếp đến hiệu quả của toàn bộ pipeline.



Hình 2.2.1: Minh họa kết quả phát hiện nhiều phương tiện trong một ảnh, trong đó mỗi đối tượng được biểu diễn bằng bounding box và nhãn lớp.

2.2.2 Bounding box và điểm đại diện phương tiện

Bounding box thường được biểu diễn bằng tọa độ của hai góc đối diện. Với hệ tọa độ ảnh có gốc tại góc trên bên trái, một bounding box được viết dưới dạng:

$$B = (x_1, y_1, x_2, y_2), \quad (2.2.1)$$

trong đó (x_1, y_1) là tọa độ góc trên bên trái và (x_2, y_2) là tọa độ góc dưới bên phải. Chiều rộng và chiều cao của bounding box được xác định theo:

$$w = x_2 - x_1, \quad h = y_2 - y_1. \quad (2.2.2)$$

Tâm hình học của bounding box được xác định bởi:

$$P_c = \left(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2} \right). \quad (2.2.3)$$

Tuy nhiên, tâm hình học không phải lúc nào cũng phù hợp để xác định vị trí của phương tiện trên mặt đường. Trong ảnh giao thông được ghi từ camera nhìn xiên trên cao, tâm bounding box thường nằm tại phần thân phương tiện, trong khi quá trình gán làn cần một điểm gần vị trí tiếp xúc giữa phương tiện và mặt đường.

Do đó, hệ thống sử dụng điểm giữa cạnh dưới của bounding box làm điểm đại diện:

$$P_v = \left(\frac{x_1 + x_2}{2}, y_2 \right). \quad (2.2.4)$$

Điểm P_v , còn được gọi là bottom-center, gần với vị trí bánh xe tiếp xúc với mặt đường hơn so với tâm hình học của bounding box. Vì vậy, điểm này phù hợp hơn cho việc xác định phương tiện đang nằm trong vùng làn nào.

Sau khi được xác định trong hệ tọa độ ảnh gốc, điểm đại diện được biến đổi sang không gian Bird's-Eye View bằng ma trận homography. Trong cùng hệ tọa độ BEV, hệ thống so sánh vị trí của điểm với các lane boundary

để xác định phương tiện thuộc làn tương ứng.

Cần lưu ý rằng điểm bottom-center chỉ là một giá trị xấp xỉ vị trí tiếp xúc với mặt đường. Sai số của bounding box, bóng phương tiện, che khuất hoặc trường hợp bounding box chạm mép ảnh có thể làm điểm này bị lệch. Vì vậy, kết quả gán làn được tiếp tục ổn định bằng lịch sử làn và cơ chế voting qua nhiều khung hình.



Hình 2.2.2: Bounding box và điểm bottom-center được sử dụng làm điểm đại diện gần vị trí tiếp xúc giữa phương tiện và mặt đường.

2.2.3 Intersection over Union, confidence và Non-Maximum Suppression

Intersection over Union, viết tắt là IoU, là chỉ số đo mức độ chồng lấn giữa hai bounding box. Với hai bounding box A và B , IoU được xác định theo công thức:

$$\text{IoU}(A, B) = \frac{\text{Area}(A \cap B)}{\text{Area}(A \cup B)}. \quad (2.2.5)$$

Trong đó, $\text{Area}(A \cap B)$ là diện tích phần giao giữa hai bounding box, còn $\text{Area}(A \cup B)$ là diện tích phần hợp của chúng. Giá trị IoU nằm trong khoảng từ 0 đến 1. IoU bằng 1 khi hai bounding box trùng khớp hoàn toàn và bằng 0 khi chúng không giao nhau.

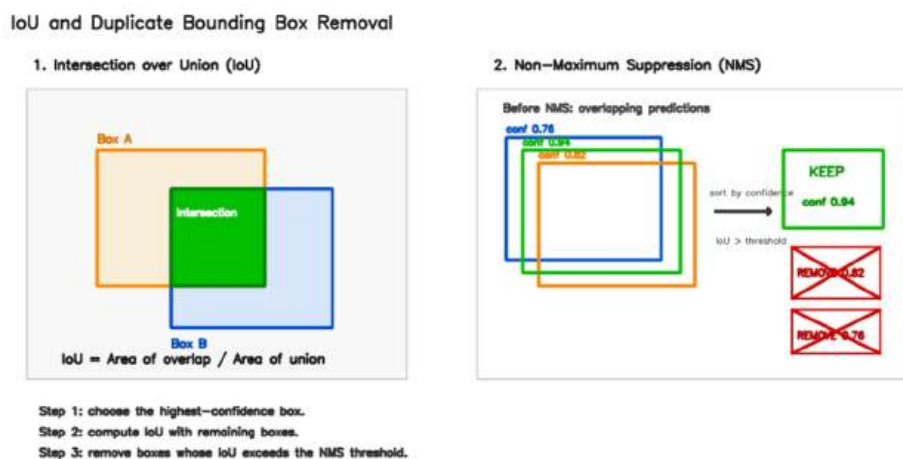
Trong bài toán phát hiện đối tượng, IoU được sử dụng trong nhiều ngữ cảnh khác nhau. Thứ nhất, IoU được dùng để đánh giá chất lượng định vị của bounding box dự đoán so với bounding box tham chiếu. Thứ hai, IoU được dùng trong Non-Maximum Suppression để xác định mức độ trùng lặp giữa các dự đoán. Thứ ba, một số thuật toán tracking-by-detection cũng sử dụng IoU để đo mức tương đồng vị trí giữa detection mới và track dự đoán.

Confidence score biểu thị mức độ tin cậy của mô hình đối với một dự đoán. Ngưỡng confidence thấp giúp giữ lại nhiều detection hơn, đặc biệt với các đối tượng nhỏ hoặc bị che khuất, nhưng đồng thời có thể làm tăng số lượng false positive. Ngược lại, ngưỡng confidence cao giúp giảm nhiễu nhưng dễ làm tăng false negative, đặc biệt với xe máy ở xa camera hoặc trong điều kiện chiếu sáng kém. Vì vậy, ngưỡng confidence cần được lựa chọn phù hợp với dữ liệu mục tiêu và nên được đánh giá thông qua Precision, Recall hoặc đường Precision–Recall thay vì chỉ dựa trên quan sát trực quan.

Non-Maximum Suppression, viết tắt là NMS, là bước hậu xử lý dùng để loại bỏ các dự đoán trùng lặp cho cùng một đối tượng. Nguyên lý cơ bản của NMS là giữ lại bounding box có confidence cao nhất, sau đó loại bỏ các bounding box còn lại có IoU lớn hơn một ngưỡng xác định so với bounding

box đã chọn. Quá trình này được lặp lại cho đến khi không còn box nào cần xét.

NMS giúp tránh hiện tượng một phương tiện bị phát hiện nhiều lần trong cùng một khung hình. Tuy nhiên, nếu ngưỡng IoU của NMS được chọn không phù hợp, một số detection hợp lệ cũng có thể bị loại bỏ, đặc biệt trong các cảnh giao thông hỗn hợp có nhiều xe máy di chuyển sát nhau và có bounding box chồng lấn một phần. Do đó, việc lựa chọn ngưỡng confidence và ngưỡng IoU của NMS có ảnh hưởng trực tiếp đến chất lượng phát hiện phương tiện.



Hình 2.2.3: Minh họa cách tính IoU giữa hai bounding box và nguyên lý hoạt động của Non-Maximum Suppression trong việc loại bỏ các dự đoán trùng lặp.

2.2.4 Mô hình YOLO trong phát hiện phương tiện

YOLO là họ mô hình phát hiện đối tượng một giai đoạn[4]. Khác với các mô hình hai giai đoạn, trong đó hệ thống trước tiên tạo các vùng đề xuất rồi mới phân loại từng vùng, YOLO dự đoán trực tiếp vị trí bounding box và

nhân lớp từ các feature map trong một quy trình suy luận thống nhất. Nhờ đó, mô hình có tốc độ xử lý cao và phù hợp với các ứng dụng video thời gian gần thực.

Một kiến trúc YOLO hiện đại thường được mô tả qua ba thành phần chính gồm Backbone, Neck và Detection Head. Backbone tiếp nhận ảnh đầu vào và trích xuất các đặc trưng từ mức thấp đến mức cao. Các đặc trưng mức thấp thường biểu diễn cạnh, texture và chi tiết cục bộ, trong khi các đặc trưng mức cao thể hiện hình dạng và ngữ nghĩa của đối tượng.

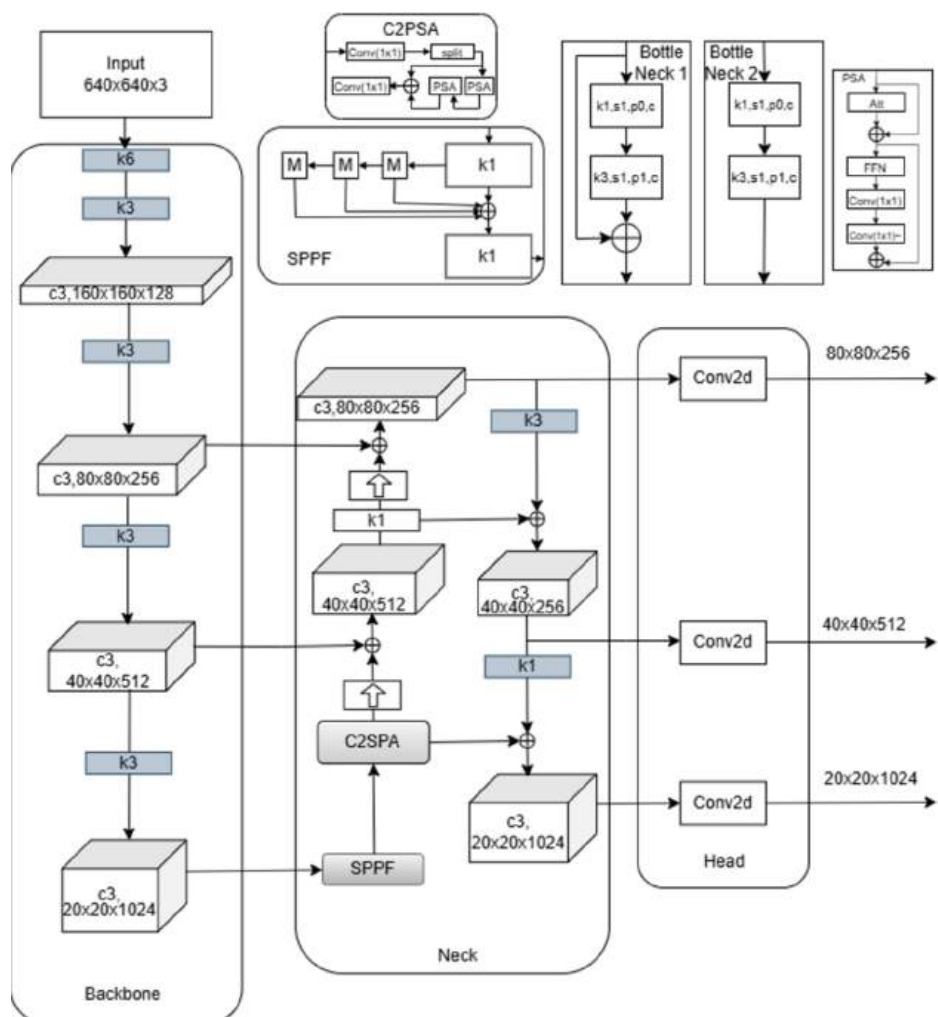
Neck tiếp nhận các feature map từ nhiều tầng của Backbone và thực hiện hợp nhất đặc trưng đa tỉ lệ. Feature map có độ phân giải cao giữ được nhiều thông tin không gian, phù hợp với các phương tiện nhỏ hoặc ở xa camera. Ngược lại, feature map có độ phân giải thấp chứa đặc trưng ngữ nghĩa mạnh hơn, phù hợp với các phương tiện có kích thước lớn. Việc kết hợp nhiều mức đặc trưng đặc biệt quan trọng trong video giao thông, nơi kích thước phương tiện thay đổi đáng kể theo khoảng cách đến camera.

Detection Head sử dụng các feature map đã được hợp nhất để tạo ra các dự đoán cuối cùng, bao gồm vị trí bounding box, lớp đối tượng và độ tin cậy. Các dự đoán được tạo ở nhiều mức tỉ lệ nhằm tăng khả năng phát hiện đồng thời phương tiện nhỏ, trung bình và lớn.

Trong các kiến trúc YOLO mới, các khối như SPPF và C2PSA có thể được sử dụng để tăng khả năng biểu diễn đặc trưng. SPPF giúp mở rộng vùng tiếp nhận của mạng và tổng hợp thông tin ở nhiều phạm vi không gian với chi phí tính toán tương đối thấp. C2PSA tích hợp cơ chế attention nhằm tăng trọng số cho các vùng đặc trưng quan trọng và giảm ảnh hưởng của thông tin không liên quan.

Ưu điểm của YOLO là tốc độ suy luận cao, khả năng phát hiện nhiều đối tượng trong cùng một khung hình và quy trình huấn luyện tương đối thuận tiện. Tuy nhiên, chất lượng mô hình vẫn phụ thuộc vào kích thước ảnh đầu vào, mức độ cân bằng dữ liệu, điều kiện chiếu sáng và sự tương đồng giữa dữ liệu huấn luyện với môi trường triển khai thực tế.

Trong đề tài, biến thể YOLO11n được lựa chọn để phát hiện bốn lớp phương tiện gồm *motorbike*, *car*, *truck* và *bus*. Ký tự “n” biểu thị phiên bản nano, có số lượng tham số và chi phí tính toán thấp hơn các biến thể lớn. Đặc điểm này phù hợp với mục tiêu tối ưu và triển khai hệ thống trên Raspberry Pi 5.



Hình 2.2.4: Sơ đồ khái quát kiến trúc YOLO gồm Backbone, Neck và Detection Head, với các feature map đa tỉ lệ phục vụ phát hiện đối tượng có kích thước khác nhau.

2.2.5 Tổng quan YOLO11 và biến thể YOLO11n

YOLO11 là thể hệ mô hình mới trong họ YOLO do Ultralytics phát triển, hỗ trợ nhiều tác vụ thị giác máy tính như phát hiện đối tượng, phân đoạn ảnh, ước lượng pose, phân loại và theo dõi đối tượng. Trong phạm vi đề tài này, YOLO11 được sử dụng ở cấu hình object detection để phát hiện các phương tiện giao thông trong từng khung hình.

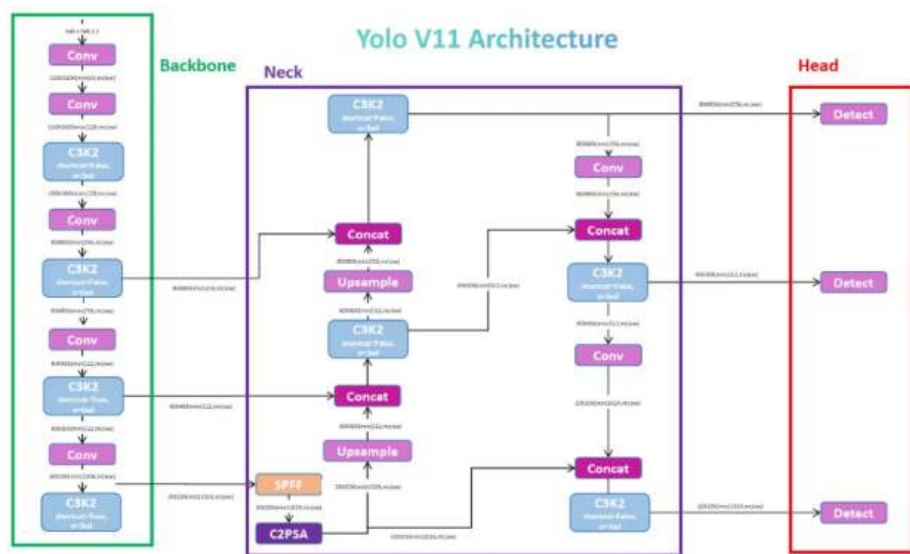
Trong cấu hình detection, YOLO11 tiếp tục duy trì kiến trúc tổng quát gồm Backbone, Neck và Head. Backbone có nhiệm vụ trích xuất đặc trưng từ ảnh đầu vào. Neck thực hiện hợp nhất các feature map ở nhiều mức tỉ lệ để hỗ trợ phát hiện đồng thời các đối tượng nhỏ, trung bình và lớn. Detection Head tạo ra các dự đoán cuối cùng, bao gồm vị trí bounding box, nhãn lớp và độ tin cậy của từng đối tượng.

Trong tài liệu của Ultralytics, một số khối như C3k2, SPPF và C2PSA được sử dụng trong cấu hình YOLO11 nhằm tăng khả năng biểu diễn đặc trưng và tổng hợp ngữ cảnh. C3k2 hỗ trợ trích xuất đặc trưng hiệu quả hơn thông qua các kết nối tắt, SPPF giúp mở rộng vùng tiếp nhận với chi phí tính toán tương đối thấp, còn C2PSA tích hợp cơ chế attention để tăng cường các vùng đặc trưng quan trọng[5]. Tuy nhiên, bố cục chi tiết có thể thay đổi giữa các biến thể, vì vậy luận văn chỉ trình bày vai trò tổng quát của các thành phần thay vì khẳng định mọi biến thể có cấu trúc hoàn toàn giống nhau.

Biến thể YOLO11n là phiên bản có kích thước nhỏ trong họ YOLO11. So với các biến thể lớn hơn, YOLO11n có số lượng tham số, dung lượng mô hình và chi phí tính toán thấp hơn, nhờ đó phù hợp hơn với các hệ thống cần tối ưu tài nguyên. Đổi lại, khả năng phát hiện các đối tượng rất nhỏ, ở xa hoặc bị che khuất có thể giảm khi sử dụng kích thước ảnh đầu vào thấp. Với

mục tiêu hướng tới triển khai trên Raspberry Pi 5, YOLO11n là lựa chọn phù hợp để khảo sát sự cân bằng giữa tốc độ xử lý và độ chính xác phát hiện.

Trong hệ thống nhận biết phương tiện đi sai làn, YOLO11n chỉ đảm nhiệm khối phát hiện phương tiện. Mô hình không trực tiếp xác định làn đường hoặc hành vi vi phạm. Tuy nhiên, chất lượng của toàn bộ pipeline phụ thuộc mạnh vào độ chính xác của kết quả detection. Nếu detector bỏ sót phương tiện, tracker không thể khởi tạo hoặc duy trì Track ID. Nếu bounding box bị lệch hoặc nhãn lớp bị nhận sai, quá trình gán làn và xác định vi phạm cũng có thể bị ảnh hưởng.



Hình 2.2.5: Sơ đồ khái quát kiến trúc YOLO11 gồm Backbone, Neck và Head, với quá trình trích xuất và hợp nhất đặc trưng ở nhiều tỉ lệ.

2.3 Cơ sở theo dõi đa đối tượng

2.3.1 Khái niệm theo dõi đa đối tượng

Theo dõi đa đối tượng, hay Multi-Object Tracking, là bài toán duy trì vị trí và định danh của nhiều đối tượng qua các khung hình liên tiếp. Nếu khối detection xác định những đối tượng xuất hiện trong khung hình hiện tại, thì khối tracking có nhiệm vụ xác định detection hiện tại có tương ứng với đối tượng nào đã xuất hiện trong các khung hình trước đó hay không.

Đầu ra quan trọng của tracking là Track ID và quỹ đạo chuyển động. Track ID là mã định danh được gán cho từng đối tượng và được duy trì trong suốt thời gian đối tượng xuất hiện trong vùng quan sát. Nhờ Track ID, hệ thống có thể tích lũy lịch sử vị trí, Lane ID, hướng di chuyển và trạng thái vi phạm cho từng phương tiện.

Nếu Track ID bị thay đổi trong quá trình theo dõi, lịch sử của phương tiện có thể bị gián đoạn. Hiện tượng một phương tiện đang có một ID nhưng sau đó bị gán sang một ID khác được gọi là ID switch. ID switch có thể làm mất lịch sử chuyển động, làm gián đoạn bộ đếm vi phạm hoặc khiến cùng một phương tiện bị thống kê nhiều lần.

Môi trường giao thông gây nhiều khó khăn cho tracking do hiện tượng che khuất, chuyển động nhanh, thay đổi kích thước theo phối cảnh, sự tương đồng giữa các phương tiện và kết quả detection không ổn định. Trong cảnh có mật độ xe máy cao, nhiều bounding box có kích thước, hình dạng và vị trí gần nhau, làm tăng nguy cơ ghép sai detection và xảy ra ID switch.

2.3.2 Tracking-by-detection, Kalman Filter và bài toán gán

Tracking-by-detection là phương pháp sử dụng các detection được tạo ra tại từng khung hình làm đầu vào cho bộ theo dõi. Tại mỗi thời điểm, tracker thực hiện ba bước chính: dự đoán trạng thái của các track đã tồn tại, xây dựng ma trận chi phí giữa các track và detection mới, sau đó giải bài toán gán để xác định các cặp tương ứng.

Kalman Filter thường được sử dụng để dự đoán trạng thái tiếp theo của mỗi track dựa trên trạng thái chuyển động trong các khung hình trước đó [6]. Trong một mô hình tuyến tính đơn giản, vector trạng thái có thể chứa vị trí, kích thước bounding box và vận tốc của đối tượng. Quá trình xử lý của Kalman Filter gồm hai giai đoạn:

- **Prediction:** dự đoán trạng thái và vị trí của track tại khung hình hiện tại;
- **Update:** hiệu chỉnh trạng thái dự đoán dựa trên detection mới được gán cho track.

Khi detection bị nhiễu hoặc mất trong một số khung hình ngắn, Kalman Filter vẫn có thể cung cấp vị trí dự đoán tạm thời. Tuy nhiên, nếu đối tượng không được phát hiện trong thời gian dài, sai số dự đoán sẽ tăng và track có thể bị xóa.

Sau bước dự đoán, tracker xây dựng ma trận chi phí giữa các track và detection. Trong ma trận này, mỗi hàng tương ứng với một track dự đoán, mỗi cột tương ứng với một detection mới và mỗi phần tử biểu diễn mức độ không phù hợp giữa một cặp track–detection.

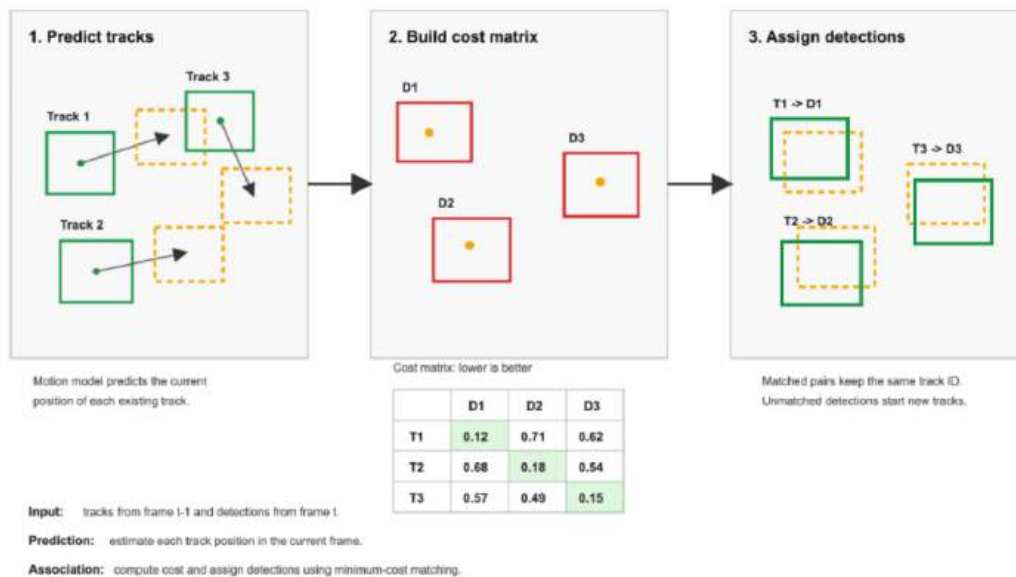
Nếu sử dụng IoU làm tiêu chí, chi phí có thể được xác định theo:

$$C_{ij} = 1 - \text{IoU}(T_i, D_j), \quad (2.3.1)$$

trong đó T_i là bounding box dự đoán của track thứ i , còn D_j là bounding box của detection thứ j . IoU càng lớn thì chi phí càng nhỏ, cho thấy detection có khả năng thuộc về track tương ứng cao hơn.

Bài toán gán được giải bằng một thuật toán linear assignment, chẳng hạn Hungarian Algorithm. Mục tiêu là tìm tập cặp track–detection có tổng chi phí nhỏ nhất. Tuy nhiên, không phải mọi cặp đều được chấp nhận. Các cặp có IoU quá thấp, khoảng cách quá lớn hoặc chi phí vượt ngưỡng được loại bỏ bằng cơ chế gating[7].

Sau quá trình gán, các cặp track–detection hợp lệ được sử dụng để cập nhật trạng thái track. Detection không được gán có thể được sử dụng để khởi tạo track mới. Track không nhận được detection phù hợp có thể được giữ lại trong một số khung hình dựa trên trạng thái dự đoán; nếu vượt quá số khung hình cho phép mà vẫn không được cập nhật, track sẽ bị xóa.



Hình 2.3.1: Quy trình tracking-by-detection gồm dự đoán vị trí track, xây dựng ma trận chi phí và gán detection cho track. Chi phí càng nhỏ biểu thị mức độ phù hợp càng cao giữa một track và một detection.

Trong đề tài, ByteTrack được sử dụng làm bộ theo dõi đa đối tượng. ByteTrack cũng hoạt động theo nguyên lý tracking-by-detection nhưng thực hiện association theo hai giai đoạn. Trước tiên, các track được ghép với những detection có confidence cao. Sau đó, các track chưa được ghép tiếp tục được so sánh với một phần detection có confidence thấp. Cơ chế này giúp hạn chế mất track khi confidence của detector suy giảm tạm thời do che khuất, ánh sáng kém hoặc đối tượng có kích thước nhỏ[8].

2.3.3 Thuật toán ByteTrack

ByteTrack là một thuật toán theo dõi đa đối tượng thuộc nhóm tracking-by-detection. Điểm nổi bật của ByteTrack là không chỉ sử dụng các detection có confidence cao, mà còn khai thác thêm một phần detection có confidence

thấp trong quá trình liên kết track. Ý tưởng này xuất phát từ nhận xét rằng nhiều detection score thấp vẫn có thể tương ứng với đối tượng thật, đặc biệt khi đối tượng bị che khuất một phần, ở xa camera hoặc xuất hiện trong điều kiện chiếu sáng bất lợi.

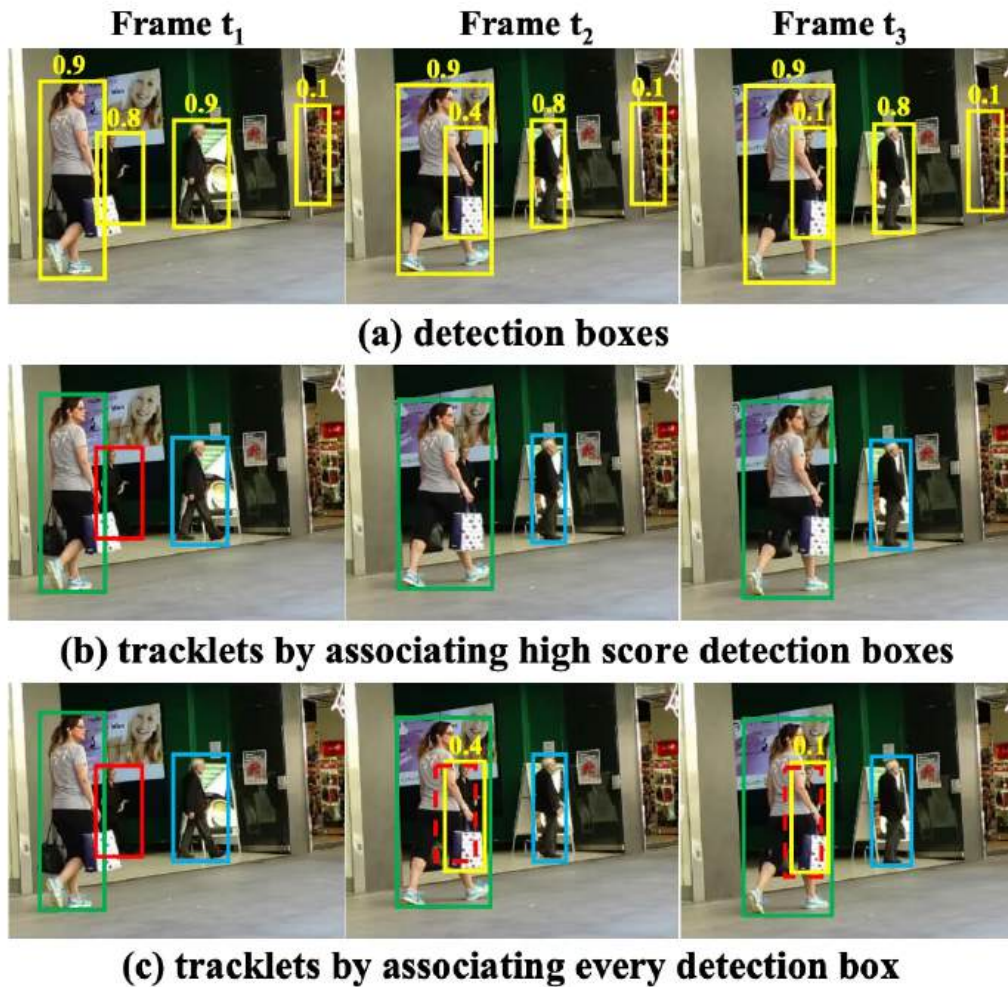
Cơ chế liên kết của ByteTrack được thực hiện theo hai giai đoạn. Trong giai đoạn thứ nhất, các track đang hoạt động được ghép với nhóm detection có confidence cao. Đây là bước association chính nhằm cập nhật những track có độ tin cậy tốt. Sau đó, các track chưa được ghép tiếp tục được so sánh với nhóm detection có confidence thấp ở giai đoạn thứ hai. Những detection score thấp không khớp với track nào sẽ không được dùng để khởi tạo track mới và thường được xem như nền hoặc nhiễu.

Chiến lược hai giai đoạn này giúp giảm hiện tượng track bị đứt do loại bỏ quá sớm các detection yếu. Trong bối cảnh giao thông, điều này đặc biệt hữu ích khi phương tiện bị che khuất, có kích thước nhỏ hoặc ở xa camera, khiến detector tạm thời giảm confidence. Nếu chỉ giữ các detection score cao, quỹ đạo của phương tiện có thể bị gián đoạn và Track ID dễ bị thay đổi. Khi đó, lịch sử vị trí, Lane ID và trạng thái vi phạm của phương tiện cũng có thể bị mất tính liên tục.

Tuy nhiên, ByteTrack vẫn phụ thuộc mạnh vào chất lượng của detector. Nếu detection bị bỏ sót trong nhiều khung hình liên tiếp, nếu các đối tượng di chuyển quá gần nhau hoặc nếu bounding box chồng lấn mạnh, quá trình association vẫn có thể xảy ra lỗi. Trong những trường hợp đó, ID switch hoặc fragmented track vẫn có thể xuất hiện.

Trong đề tài này, ByteTrack được sử dụng để duy trì Track ID cho các phương tiện giao thông sau bước phát hiện bởi YOLO11n. Track ID ổn

định là điều kiện quan trọng để tích lũy lịch sử làn, lịch sử hướng di chuyển và bộ đếm vi phạm, từ đó hỗ trợ quá trình phát hiện đi sai làn và đi ngược chiều.



Hình 2.3.2: Minh họa ý tưởng của ByteTrack: chỉ sử dụng detection score cao có thể làm quỹ đạo bị đứt, trong khi việc tiếp tục liên kết với detection score thấp giúp duy trì track tốt hơn.

2.3.4 Vai trò của tracking trong phân tích vi phạm

Tracking cung cấp thông tin thời gian cần thiết cho quá trình phân tích hành vi của phương tiện. Trong khi khối detection chỉ xác định vị trí và lớp của phương tiện tại từng khung hình độc lập, tracking liên kết các detection qua thời gian và duy trì một Track ID cho cùng một phương tiện. Nhờ đó, hệ thống có thể xây dựng lịch sử vị trí, lịch sử làn và trạng thái vi phạm riêng cho từng đối tượng.

Đối với bài toán phát hiện sai làn, kết quả gán làn tại một khung hình có thể bị sai do bounding box dao động, lane boundary dịch chuyển hoặc phương tiện đang nằm gần vạch phân làn. Nếu hệ thống kết luận chỉ dựa trên một khung hình, số lượng cảnh báo sai có thể tăng cao. Khi có Track ID ổn định, hệ thống có thể lưu lại Lane ID qua nhiều khung hình, áp dụng cơ chế voting và yêu cầu trạng thái sai làn tồn tại liên tiếp trong một số khung hình tối thiểu trước khi xác nhận vi phạm.

Đối với bài toán phát hiện đi ngược chiều, tracking đóng vai trò đặc biệt quan trọng vì hướng chuyển động được xác định từ sự thay đổi vị trí của cùng một phương tiện theo thời gian. Lịch sử các điểm đại diện của Track ID được sử dụng để tính độ dịch chuyển và ước lượng hướng di chuyển. Những thay đổi vị trí nhỏ do bounding box dao động có thể được loại bỏ bằng ngưỡng dịch chuyển tối thiểu, trong khi kết quả hướng được ổn định bằng cách tổng hợp nhiều quan sát liên tiếp.

Tracking còn giúp hệ thống tránh đếm trùng vi phạm. Sau khi một Track ID đã được xác nhận và ghi nhận vi phạm, hệ thống có thể lưu trạng thái tương ứng để không tăng bộ đếm ở các khung hình tiếp theo. Nhờ đó, kết quả được thống kê ở cấp phương tiện thay vì cấp khung hình; mỗi

phương tiện chỉ được tính một lần cho mỗi loại vi phạm trong thời gian tồn tại của track.

Tuy nhiên, chất lượng phân tích vi phạm phụ thuộc trực tiếp vào độ ổn định của tracking. Nếu xảy ra ID switch, lịch sử làn và lịch sử chuyển động có thể bị chuyển sang một ID khác hoặc bị khởi tạo lại. Nếu track bị mất rồi được tạo lại, cùng một phương tiện có nguy cơ bị thống kê nhiều lần. Vì vậy, khả năng duy trì Track ID là một điều kiện quan trọng để bảo đảm độ tin cậy của quá trình nhận biết sai làn và đi ngược chiều.

2.4 Hình học ảnh và biểu diễn Bird's-Eye View

2.4.1 Vùng quan tâm ROI

Region of Interest, viết tắt là ROI, là vùng ảnh được lựa chọn để tập trung quá trình xử lý. Trong bài toán nhận diện làn đường, ROI thường bao phủ phần mặt đường cần giám sát và loại bỏ các vùng không liên quan như bầu trời, cây cối, vỉa hè hoặc khu vực ngoài luồng giao thông. Với camera cố định, ROI có thể được thiết lập một lần và tái sử dụng cho toàn bộ video nếu vị trí và góc quan sát không thay đổi.

ROI quá hẹp có thể cắt mất vạch hoặc đường biên của làn ngoài cùng. Ngược lại, ROI quá rộng có thể đưa thêm vỉa hè, đường phụ, bóng đổ và các cấu trúc không thuộc mặt đường vào quá trình trích xuất candidate. Vì vậy, ROI cần bao phủ đầy đủ vùng lưu thông nhưng không nên được xem trực tiếp là đường biên làn thực tế. Mép ROI chỉ đóng vai trò giới hạn vùng xử lý; nếu được sử dụng để suy luận boundary ngoài thì cần có thêm các điều kiện hình học và độ tin cậy phù hợp.

2.4.2 Homography và Perspective Transform

Homography là phép biến đổi xạ ảnh mô tả quan hệ ánh xạ giữa hai mặt phẳng. Khi giả định mặt đường gần phẳng, một điểm trong ảnh gốc có thể được ánh xạ sang mặt phẳng đích bằng tọa độ đồng nhất:

$$s \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \mathbf{H} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad (2.4.1)$$

trong đó $\mathbf{H}$ là ma trận Homography kích thước 3×3 , còn s là hệ số tỉ lệ. Sau phép nhân ma trận, tọa độ được chuẩn hóa bằng cách chia cho thành phần đồng nhất thứ ba[9].

Do ma trận Homography được xác định đến một hệ số tỉ lệ, phép biến đổi có tám bậc tự do và có thể được ước lượng từ tối thiểu bốn cặp điểm tương ứng trong cấu hình không suy biến. Trong thực tế, bốn điểm nguồn thường được lựa chọn trên vùng mặt đường trong ảnh gốc và ánh xạ tới bốn điểm đích tạo thành một hình chữ nhật trong không gian Bird's-Eye View.

Perspective Transform là thao tác áp dụng ma trận Homography lên toàn bộ ảnh hoặc một tập điểm. Chất lượng phép biến đổi phụ thuộc trực tiếp vào vị trí các điểm nguồn và điểm đích. Nếu các điểm này không phù hợp, ảnh sau biến đổi có thể bị kéo giãn, nén hoặc lệch không đều, làm sai vị trí đường biên và chiều rộng làn.

Homography chỉ mô tả tốt các điểm nằm trên hoặc gần cùng một mặt phẳng. Vì vậy, mặt đường có thể được biến đổi tương đối ổn định, trong khi các phương tiện có chiều cao sẽ bị biến dạng trong ảnh BEV. Do đó, BEV

trong đề tài chủ yếu được sử dụng để phân tích hình học mặt đường và gán điểm đại diện phương tiện, không nhằm khôi phục hình dạng ba chiều chính xác của phương tiện.

2.4.3 Bird's-Eye View

Bird's-Eye View, viết tắt là BEV, là biểu diễn mặt đường theo góc nhìn gần từ trên xuống. Phép biến đổi này làm giảm hiện tượng hội tụ của các vạch làn trong ảnh phối cảnh và giúp quan hệ trái-phải giữa các đường biên làn trở nên rõ ràng hơn.

Trong không gian BEV, khoảng cách giữa các boundary có xu hướng ổn định hơn theo chiều sâu. Nhờ đó, các kỹ thuật Histogram Projection, peak detection, liên kết candidate và Polynomial Fitting dễ áp dụng hơn so với ảnh gốc. Điểm đại diện của phương tiện cũng được biến đổi sang cùng hệ tọa độ BEV để xác định nó nằm giữa hai boundary nào.

Tuy nhiên, BEV không loại bỏ hoàn toàn sai số phối cảnh. Mặt đường không phẳng, góc nhìn quá xiên hoặc ROI không chính xác có thể làm vạch làn bị méo. Với đoạn đường cong mạnh, khu vực nhập làn hoặc tách làn, một ma trận Homography duy nhất có thể không mô tả chính xác toàn bộ vùng mặt đường.

2.5 Tiền xử lý ảnh phục vụ nhận diện làn

2.5.1 Mục đích và nguyên tắc kết hợp đặc trưng

Mục đích của tiền xử lý là làm tăng độ phân biệt giữa vạch sơn làn đường và nền đường, đồng thời giảm nhiễu trước khi trích xuất candidate. Không có một đặc trưng đơn lẻ hoạt động ổn định trong mọi điều kiện, vì

vậy hệ thống thường kết hợp thông tin độ sáng, màu sắc, gradient và hình dạng.

Việc kết hợp các đặc trưng cần được kiểm soát. Nếu thực hiện phép OR trực tiếp giữa mọi mask, cạnh phương tiện, vết nứt và bóng đổ có thể tạo ra nhiều candidate giả. Ngược lại, điều kiện quá chặt có thể làm mất vạch mờ, vạch đứt hoặc vạch bị che khuất một phần. Vì vậy, các mask cần được kết hợp cùng những điều kiện hình học và bộ lọc nhiễu phù hợp.

2.5.2 Canny và Sobel Gradient

Canny Edge Detection phát hiện các vị trí có thay đổi cường độ mạnh thông qua các bước làm mờ ảnh, tính gradient, non-maximum suppression và ngưỡng kép. Thuật toán tạo ra biên tương đối mảnh, nhưng đồng thời cũng có thể phát hiện cạnh phương tiện, vỉa hè, biển báo và texture mặt đường[10].

Sobel Gradient tính đạo hàm cường độ ảnh theo phương ngang hoặc phương dọc. Trong ảnh BEV, các vạch làn thường gần phương dọc nên gradient theo trục x có thể làm nổi bật hai cạnh của vạch sơn. Tuy nhiên, một vạch sơn có thể sinh ra hai biên song song, từ đó tạo hai peak gần nhau nếu không có bước hợp nhất hoặc lọc phù hợp.

Vì vậy, Canny và Sobel được sử dụng như các nguồn bằng chứng hỗ trợ, kết hợp với mask màu, phép hình thái học và các tiêu chí hình học thay vì đóng vai trò quyết định duy nhất.

2.5.3 Không gian màu, CLAHE và Adaptive Threshold

Các không gian màu HSV, HLS và LAB tách thông tin màu sắc khỏi độ sáng ở những mức độ khác nhau. HSV và HLS thuận lợi khi cần phân biệt màu của vạch sơn với nền đường, trong khi LAB cung cấp kênh L biểu diễn độ sáng tương đối độc lập với hai kênh màu còn lại. Việc lựa chọn kênh phù hợp giúp tăng khả năng phát hiện vạch trắng hoặc vàng trong điều kiện chiếu sáng thay đổi.

CLAHE cải thiện tương phản cục bộ bằng cách chia ảnh thành các ô nhỏ, cân bằng histogram trong từng ô và giới hạn mức khuếch đại. Kỹ thuật này có thể làm rõ vạch trong vùng tối nhưng đồng thời cũng tăng cường texture mặt đường và vết nứt. Vì vậy, CLAHE chỉ được sử dụng như một bước tăng cường ảnh đầu vào, không phải bằng chứng đủ để kết luận pixel thuộc vạch làn.

Adaptive Threshold tính ngưỡng dựa trên vùng lân cận của từng pixel và phù hợp khi ánh sáng không đồng đều. Tuy nhiên, kỹ thuật này có thể tạo nhiều nhiễu trên mặt đường có texture mạnh hoặc vùng phản xạ. Kích thước cửa sổ và hằng số hiệu chỉnh cần được lựa chọn theo độ phân giải ảnh và độ rộng biểu kiến của vạch sơn trong BEV.

2.5.4 Morphology và Connected Component Analysis

Các phép hình thái học gồm erosion, dilation, opening và closing được sử dụng để điều chỉnh cấu trúc ảnh nhị phân. Kernel, hay phần tử cấu trúc, quy định hình dạng và phạm vi tác động của mỗi phép toán. Opening có thể loại bỏ các vùng nhiễu nhỏ, trong khi closing giúp nối các đoạn vạch bị đứt gãy.

Hiệu quả của morphology phụ thuộc mạnh vào kích thước và hướng của kernel. Kernel quá dài có thể nối hai vạch khác nhau hoặc nối vạch với vùng nhiễu. Ngược lại, opening quá mạnh có thể xóa các đoạn vạch đứt có diện tích nhỏ. Vì vậy, kernel cần được lựa chọn phù hợp với hướng và kích thước của lane marking trong ảnh BEV.

Connected Component Analysis nhóm các pixel trắng liên thông thành từng component và mô tả chúng thông qua diện tích, chiều dài, chiều rộng, tỷ lệ hình dạng và hướng. Các component quá nhỏ, quá rộng hoặc có hình dạng không phù hợp có thể được loại bỏ. Tuy nhiên, vạch bị che khuất hoặc vạch đứt cũng có thể tạo ra component ngắn, vì vậy không nên lọc chỉ dựa trên diện tích.

2.6 Trích xuất, mô hình hóa và ổn định đường biên làn

2.6.1 Histogram Projection và phân tích theo chiều sâu

Histogram Projection được xây dựng bằng cách cộng số pixel candidate theo từng cột của ảnh nhị phân:

$$H(x) = \sum_y B(x, y), \quad (2.6.1)$$

trong đó $B(x, y) \in \{0, 1\}$ là giá trị của ảnh nhị phân đã chuẩn hóa tại tọa độ (x, y) . Các cột có mật độ pixel lớn tạo thành peak và được xem là vị trí ứng viên của đường biên làn.

Histogram toàn cục hoạt động tốt khi vạch liên tục và xuất hiện trên phần lớn chiều cao ảnh. Tuy nhiên, phương pháp này dễ bỏ sót vạch chỉ xuất hiện tại một phần chiều sâu hoặc bị che khuất cục bộ. Vì vậy, ảnh BEV được

chia thành nhiều dải theo trục y, sau đó histogram được tính riêng tại từng dải.

Phân tích theo nhiều dải cho phép theo dõi sự thay đổi vị trí của vạch cong và duy trì khả năng liên kết candidate khi vạch bị đứt, bị che hoặc có độ tương phản thấp tại một số vùng. Các peak ở từng dải được dùng làm điểm quan sát cho quá trình liên kết candidate.

2.6.2 Sliding Window và liên kết peak

Sliding Window là phương pháp cơ sở bắt đầu từ các peak ở phần dưới ảnh và dịch chuyển các cửa sổ tìm kiếm theo chiều dọc. Các pixel trắng trong mỗi cửa sổ được sử dụng để cập nhật tâm của cửa sổ tiếp theo. Phương pháp này phù hợp khi vạch rõ, tương đối liên tục và điểm khởi tạo chính xác.

Hạn chế của Sliding Window là phụ thuộc mạnh vào peak ban đầu. Khi peak khởi tạo sai hoặc vạch bị che trong một đoạn dài, cửa sổ có thể bám theo nhiễu hoặc chuyển sang vạch bên cạnh. Để tăng tính linh hoạt, hệ thống sử dụng liên kết peak theo nhiều dải chiều sâu. Các peak được ghép dựa trên khoảng cách ngang, xu hướng chuyển động, độ liên tục và độ phủ theo chiều sâu để hình thành candidate boundary.

Một candidate có thể cho phép thiếu peak tại một số dải khi vạch bị đứt hoặc bị che khuất, nhưng phải đạt số điểm hợp lệ và độ phủ tối thiểu. Khoảng cách liên kết cần đủ lớn để theo được đường cong nhẹ nhưng không được quá lớn đến mức ghép các peak thuộc hai vạch khác nhau.

2.6.3 Polynomial Fitting và tính chỉnh đường cong

Do các đường biên làn trong BEV thường gần phương dọc, tập điểm candidate được mô hình hóa dưới dạng x theo y bằng đa thức bậc hai:

$$x(y) = ay^2 + by + c, \quad (2.6.2)$$

trong đó a , b và c là các hệ số của đường cong. Hệ số a phản ánh mức độ cong, b biểu diễn xu hướng nghiêng và c xác định vị trí ngang cơ bản của boundary[11].

Đa thức bậc hai đủ khả năng biểu diễn đường thẳng và đường cong nhẹ với chi phí tính toán thấp. Nếu sử dụng bậc cao hơn, mô hình có thể linh hoạt hơn nhưng dễ overfitting khi dữ liệu có nhiễu, bị che khuất hoặc chỉ có ít điểm quan sát.

Quá trình fitting có thể được thực hiện lặp để loại bỏ outlier. Sau lần fit đầu tiên, residual giữa tọa độ điểm quan sát và vị trí được dự đoán bởi đường cong được tính. Những điểm có residual lớn hơn ngưỡng được xem là outlier và bị loại trước khi fit lại.

Ngoài ra, một hành lang tìm kiếm hẹp quanh candidate ban đầu có thể được sử dụng để thu thập lại các pixel có khả năng thuộc vạch sơn. Tập điểm mới sau đó được dùng để fit lại đa thức, giúp đường cong tiến gần hơn về tâm lane marking và giảm ảnh hưởng của các peak bị lệch.

2.6.4 Đánh giá chất lượng boundary và topology

Trước khi được sử dụng để xây dựng mô hình làn, mỗi candidate boundary cần được đánh giá dựa trên mức độ phù hợp với dữ liệu quan

sát. Các tiêu chí thường gồm marking support, coverage, continuity, fitting residual, orientation và độ cong.

Marking support biểu diễn lượng pixel có đặc trưng giống vạch sơn nằm gần candidate. Coverage là tỷ lệ chiều sâu mà candidate có điểm quan sát hợp lệ, còn continuity phản ánh mức liên tục của candidate giữa các dải chiều sâu liên kề. Fitting residual là sai số giữa các điểm quan sát và đường cong được fit. Candidate có nhiều điểm hỗ trợ, xuất hiện trên phần lớn chiều sâu, có residual nhỏ và hình dạng ổn định được đánh giá cao hơn candidate ngắn, gián đoạn hoặc dao động mạnh.

Tuy nhiên, mô hình làn không nên được tạo bằng cách chọn độc lập các candidate có điểm cao nhất. Một candidate có chất lượng riêng lẻ tốt vẫn có thể không phù hợp khi kết hợp với các boundary khác. Vì vậy, toàn bộ topology cần được đánh giá để bảo đảm các boundary có thứ tự trái-phải nhất quán, khoảng cách dương và không giao cắt trong vùng chiều sâu được xét.

Chiều rộng giữa hai boundary liên kề cũng cần nằm trong miền hợp lý và không thay đổi đột ngột theo chiều sâu. Nếu hai boundary quá gần, chúng có thể là hai cạnh của cùng một vạch sơn. Nếu khoảng cách quá lớn, mô hình có thể đang thiếu một boundary ở giữa.

Ngoài điểm chất lượng của các boundary được chọn, quá trình đánh giá topology còn sử dụng các thành phần phạt. Crossing penalty làm giảm điểm của mô hình có các boundary giao nhau. Unused candidate penalty được áp dụng khi một candidate có bằng chứng mạnh nhưng không được sử dụng. Virtual boundary penalty làm giảm độ tin cậy của các boundary được suy luận từ mép ROI hoặc chiều rộng làn thay vì được phát hiện trực tiếp từ

vạch sơn.

Việc đánh giá đồng thời chất lượng candidate và tính hợp lý của topology giúp hạn chế hai trường hợp đối lập: lựa chọn thêm boundary nhiều làm xuất hiện lane ảo, hoặc loại bỏ candidate thật nhưng yếu làm thiếu lane.

2.6.5 Temporal Smoothing và Persistence Map

Do camera được giữ cố định, vị trí của các lane boundary không thay đổi lớn giữa các khung hình liên tiếp. Temporal Smoothing khai thác đặc điểm này để giảm dao động của kết quả phát hiện theo thời gian.

Giả sử trạng thái của một boundary được biểu diễn bằng vector hệ số đa thức:

$$\mathbf{L}_t = \begin{bmatrix} a_t & b_t & c_t \end{bmatrix}^T, \quad (2.6.3)$$

trong đó a_t , b_t và c_t là các hệ số của đường cong tại thời điểm t . Trạng thái được làm mượt bằng trung bình mũ:

$$\mathbf{L}_t = \alpha \mathbf{L}_{t-1} + (1 - \alpha) \hat{\mathbf{L}}_t, \quad (2.6.4)$$

trong đó $\hat{\mathbf{L}}_t$ là kết quả quan sát tại khung hình hiện tại và α là hệ số làm mượt. Giá trị α lớn làm tăng ảnh hưởng của lịch sử, giúp boundary ổn định hơn nhưng phản ứng chậm với thay đổi. Ngược lại, giá trị α nhỏ giúp mô hình phản ứng nhanh hơn nhưng dễ dao động do nhiễu.

Bên cạnh việc làm mượt hệ số đường cong, Persistence Map tích lũy tần suất một pixel hoặc vùng ảnh được xem là lane candidate qua nhiều

khung hình. Bản đồ có thể được cập nhật theo:

$$P_t(x, y) = \beta P_{t-1}(x, y) + (1 - \beta) B_t(x, y), \quad (2.6.5)$$

trong đó $B_t(x, y) \in \{0, 1\}$ là mask lane candidate tại thời điểm t , còn β là hệ số duy trì. Với cách khởi tạo phù hợp, $P_t(x, y)$ nằm trong khoảng từ 0 đến 1. Giá trị lớn cho biết pixel thường xuyên xuất hiện như một phần của cấu trúc lane, trong khi cạnh phương tiện, bóng đổ hoặc nhiễu chuyển động thường chỉ tồn tại trong thời gian ngắn.

Temporal Smoothing và Persistence Map giúp giảm hiện tượng lane flicker khi vạch bị che khuất hoặc mất tín hiệu trong một số khung hình. Tuy nhiên, các cơ chế này cũng có thể duy trì sai số nếu một mô hình không chính xác được lựa chọn liên tục. Vì vậy, kết quả hiện tại chỉ nên được cập nhật vào lịch sử khi model score, chất lượng boundary và topology đạt các điều kiện tin cậy.

2.6.6 Lane Boundary, Lane Polygon và biên ngoài

Lane boundary là đường phân chia giữa hai vùng lưu thông hoặc giữa một làn với biên ngoài của phần đường. Lane marking là vạch sơn quan sát được trên mặt đường, trong khi lane boundary là biểu diễn hình học được hệ thống sử dụng để xây dựng vùng làn. Trong nhiều trường hợp, một boundary có thể được suy luận ngay cả khi không tồn tại vạch sơn rõ ràng.

Khi có n boundary được sắp xếp đúng theo thứ tự trái-phải và bao phủ đầy đủ vùng đường cần xét, hệ thống có thể tạo tối đa $n - 1$ lane polygon bằng cách ghép từng cặp boundary liền kề. Mỗi polygon biểu diễn một vùng làn thay vì bản thân đường vạch.

Biên ngoài của phần đường có thể không được đánh dấu bằng vạch sơn. Sử dụng trực tiếp mép ROI làm outer boundary là phương án đơn giản nhưng có thể gây sai lệch vì ROI chỉ là giới hạn xử lý. Một phương án khác là suy luận boundary ngoài từ mép đường quan sát được hoặc từ chiều rộng làn trung vị của các lane đã phát hiện. Boundary được suy luận cần có confidence thấp hơn boundary được hỗ trợ trực tiếp bởi pixel vạch và phải chịu virtual boundary penalty trong quá trình lựa chọn mô hình.

Lane polygon có thể được sử dụng để hiển thị vùng làn hoặc kiểm tra point-in-polygon. Trong không gian BEV, một phương án khác là tính vị trí x của các boundary tại cùng tọa độ y với điểm phương tiện, sau đó so sánh quan hệ trái-phải. Phương pháp này tận dụng trực tiếp mô hình đa thức và tránh phải tạo polygon mới ở mỗi khung hình.

2.7 Gán phương tiện vào làn và phát hiện vi phạm

2.7.1 Biến đổi vị trí phương tiện và gán làn

Điểm đại diện của phương tiện trong ảnh gốc được biến đổi sang không gian BEV bằng ma trận Homography. Việc biến đổi này bảo đảm điểm phương tiện và các lane boundary được biểu diễn trong cùng một hệ tọa độ trước khi so sánh.

Giả sử điểm phương tiện trong BEV là $P_v = (x_v, y_v)$. Tại tọa độ y_v , vị trí ngang của từng boundary được tính từ hàm đa thức tương ứng. Nếu điểm nằm giữa hai boundary liên kề, phương tiện được gán vào làn i :

$$x_i(y_v) < x_v < x_{i+1}(y_v). \quad (2.7.1)$$

Trong thực tế, một vùng biên có thể được thiết lập xung quanh mỗi boundary. Nếu điểm phương tiện nằm quá gần boundary, lane index được xem là chưa chắc chắn hoặc được giữ theo trạng thái ổn định trước đó. Cách xử lý này giúp giảm hiện tượng lane ID dao động khi phương tiện nằm trên vạch hoặc đang chuyển làn.

Nếu điểm nằm ngoài topology đã khóa, boundary chưa đủ tin cậy hoặc không tồn tại cặp boundary phù hợp, hệ thống nên trả về trạng thái *unknown* thay vì tự động kết luận phương tiện thuộc một làn hoặc đang vi phạm.

Một phương án khác là kiểm tra điểm có nằm trong lane polygon hay không. Point-in-polygon phù hợp khi polygon được xây dựng chính xác trong ảnh gốc hoặc BEV. Cả phương pháp so sánh boundary và phương pháp point-in-polygon đều phụ thuộc vào chất lượng của điểm đại diện, ma trận Homography và mô hình lane boundary.

2.7.2 Phát hiện phương tiện đi sai làn

Phát hiện sai làn yêu cầu ánh xạ từ Lane ID đến tập lớp phương tiện được phép. Với phương tiện thuộc lớp c và được gán vào làn l , trạng thái sai làn tại một khung hình được xác định bởi:

$$\text{WrongLane}(c, l) = \begin{cases} 1, & c \notin \text{Allowed}(l), \\ 0, & c \in \text{Allowed}(l). \end{cases} \quad (2.7.2)$$

Trong đó, $\text{Allowed}(l)$ là tập các lớp phương tiện được phép lưu thông trong làn l . Quy tắc này có thể được cấu hình dựa trên biển báo, tổ chức giao thông và hướng lưu thông tại vị trí camera.

Quyết định vi phạm không nên chỉ dựa trên kết quả của một khung hình, vì bounding box, điểm đại diện hoặc Lane ID có thể dao động. Với mỗi Track ID, hệ thống lưu lịch sử lần và số khung hình liên tiếp thỏa mãn điều kiện sai lần. Bộ đếm sai lần có thể được cập nhật theo:

$$S_t^{\text{WL}} = \begin{cases} S_{t-1}^{\text{WL}} + 1, & \text{WrongLane}(c, l) = 1, \\ 0, & \text{WrongLane}(c, l) = 0. \end{cases} \quad (2.7.3)$$

Phương tiện chỉ được xác nhận vi phạm khi S_t^{WL} đạt ngưỡng số khung hình tối thiểu. Cơ chế này giúp giảm cảnh báo sai khi phương tiện tạm thời nằm gần boundary, đang chuyển làn hoặc khi kết quả lane assignment bị nhiễu trong một số khung hình.

Khi Lane ID ở trạng thái *unknown* hoặc topology chưa ổn định, hệ thống không tăng bộ đếm vi phạm. Sau khi một Track ID đã được xác nhận sai lần, trạng thái này được lưu để tránh đếm cùng một phương tiện nhiều lần trong các khung hình tiếp theo.

2.7.3 Phát hiện phương tiện đi ngược chiều

Hướng chuyển động của phương tiện được ước lượng từ lịch sử vị trí của cùng một Track ID. Với hai thời điểm cách nhau k khung hình, độ dịch chuyển theo trục dọc được xác định từ điểm đại diện của phương tiện:

$$\Delta y_t = y_t - y_{t-k}, \quad (2.7.4)$$

trong đó y_t và y_{t-k} lần lượt là tọa độ theo phương dọc của điểm đại diện tại thời điểm hiện tại và thời điểm trước đó k khung hình. Các tọa độ

có thể được xét trong không gian BEV để giảm ảnh hưởng của phối cảnh.

Trong hệ tọa độ ảnh thông thường, trục y tăng theo hướng từ trên xuống dưới. Vì vậy, với camera nhìn dọc theo mặt đường, $\Delta y_t > 0$ thường biểu thị phương tiện di chuyển từ xa đến gần camera, còn $\Delta y_t < 0$ thường biểu thị phương tiện di chuyển từ gần ra xa. Tuy nhiên, ý nghĩa của dấu dịch chuyển phụ thuộc vào vị trí và hướng lắp đặt camera, do đó quy ước hướng cần được xác định riêng cho từng video hoặc từng vị trí giám sát.

Những thay đổi nhỏ của tọa độ có thể xuất phát từ dao động bounding box thay vì chuyển động thật. Vì vậy, hệ thống chỉ ghi nhận một quan sát hướng khi độ dịch chuyển đạt ngưỡng tối thiểu:

$$|\Delta y_t| \geq d_{\min}, \quad (2.7.5)$$

trong đó $d_{\min}$ là ngưỡng dịch chuyển tối thiểu. Nếu $|\Delta y_t| < d_{\min}$, hướng của quan sát được xem là *unknown* và không được sử dụng để kết luận ngược chiều.

Để giảm ảnh hưởng của nhiễu tracking, hướng cuối cùng của một track không được xác định từ một cặp vị trí duy nhất. Hệ thống lưu nhiều quan sát hướng trong một cửa sổ thời gian và sử dụng cơ chế bỏ phiếu. Track chỉ được xem là có hướng ổn định khi có đủ số phiếu hợp lệ và một hướng chiếm ưu thế rõ ràng.

Mỗi làn l được gán một hướng lưu thông hợp lệ $D_{\text{expected}}(l)$. Hướng quan sát của phương tiện được ký hiệu là D_{track} . Trạng thái ngược chiều tại một thời điểm được xác định bởi:

$$\text{WrongWay}(D_{\text{track}}, l) = \begin{cases} 1, & D_{\text{track}} \neq D_{\text{expected}}(l), \\ 0, & D_{\text{track}} = D_{\text{expected}}(l). \end{cases} \quad (2.7.6)$$

Điều kiện trên chỉ được đánh giá khi Lane ID và hướng của track đều ổn định. Các track có quỹ đạo quá ngắn, độ dịch chuyển chưa đủ lớn, Lane ID chưa xác định hoặc hướng đang ở trạng thái *unknown* không được sử dụng để kết luận vi phạm.

Hướng hợp lệ của từng làn có thể được cấu hình dựa trên tổ chức giao thông tại vị trí camera. Trong trường hợp chưa có cấu hình, hướng chủ đạo có thể được ước lượng bằng cách bỏ phiếu từ nhiều track có quỹ đạo dài và ổn định. Tuy nhiên, phương pháp suy luận này cần loại bỏ các track nghi ngờ vi phạm để tránh làm sai hướng tham chiếu.

2.7.4 Xác nhận vi phạm theo thời gian

Kết quả gán làn, phân loại phương tiện và ước lượng hướng có thể dao động giữa các khung hình. Vì vậy, hệ thống không xác nhận vi phạm từ một quan sát đơn lẻ mà duy trì một bộ đếm liên tiếp, gọi là violation streak, cho từng Track ID và từng loại vi phạm.

Với một điều kiện vi phạm V_t tại thời điểm t , streak được cập nhật theo:

$$S_t = \begin{cases} S_{t-1} + 1, & V_t = 1, \\ 0, & V_t = 0. \end{cases} \quad (2.7.7)$$

Trong đó, $V_t = 1$ cho biết phương tiện thỏa mãn điều kiện vi phạm tại khung hình hiện tại, còn $V_t = 0$ cho biết điều kiện không còn thỏa mãn. Đối với các trạng thái chưa chắc chắn như Lane ID không xác định, hướng *unknown* hoặc track quá mới, hệ thống không nên tăng streak.

Vi phạm chỉ được xác nhận khi streak đạt ngưỡng tối thiểu:

$$S_t \geq S_{\min}, \quad (2.7.8)$$

trong đó $S_{\min}$ là số khung hình liên tiếp cần thiết để xác nhận. Có thể sử dụng các ngưỡng khác nhau cho sai lầm và ngược chiều vì độ ổn định của hai loại điều kiện không giống nhau.

Cơ chế streak giúp kết hợp thông tin không gian và thời gian, đồng thời giảm cảnh báo sai do bounding box dao động, lane boundary dịch chuyển hoặc hướng chuyển động chưa ổn định. Sau khi một Track ID được xác nhận vi phạm, ID và loại vi phạm tương ứng được lưu lại để tránh tăng bộ đếm nhiều lần cho cùng một phương tiện trong các khung hình tiếp theo.

Nếu tracker làm mất phương tiện rồi tạo lại một Track ID mới, cơ chế chống đếm trùng theo ID có thể không nhận ra đây là cùng một phương tiện. Vì vậy, độ ổn định của tracking vẫn ảnh hưởng trực tiếp đến độ chính xác thống kê vi phạm.

2.8 Tối ưu hệ thống trên thiết bị nhúng

2.8.1 Các ràng buộc của thiết bị nhúng

Thiết bị nhúng có giới hạn về năng lực tính toán, dung lượng RAM, băng thông bộ nhớ, công suất tiêu thụ và khả năng tản nhiệt. Trong hệ thống

xử lý video, detector phải hoạt động liên tục, trong khi các module tracking, lane detection, gán làn, phân tích vi phạm và hiển thị kết quả cũng đồng thời sử dụng tài nguyên. Vì vậy, quá trình tối ưu cần được thực hiện trên toàn bộ pipeline thay vì chỉ tập trung vào mô hình học sâu.

Các yếu tố ảnh hưởng đến tốc độ xử lý gồm kích thước ảnh đầu vào, số lượng phương tiện trong khung hình, tần suất cập nhật mô hình làn, số lượng ảnh trung gian được tạo, thao tác hiển thị, ghi video và sao chép dữ liệu giữa các module. Nhiệt độ thiết bị cũng ảnh hưởng đến hiệu năng khi hệ thống hoạt động trong thời gian dài, vì CPU có thể giảm xung khi xảy ra thermal throttling.

Quá trình benchmark cần phân biệt giữa thời gian suy luận của detector và thời gian xử lý đầu-cuối của toàn bộ hệ thống. Ngoài latency của mô hình, cần đo thời gian của từng stage, FPS toàn pipeline, mức sử dụng CPU, RAM và nhiệt độ thiết bị. Các chức năng debug, hiển thị nhiều cửa sổ và ghi video cần được tắt hoặc đo riêng để tránh làm sai lệch kết quả đánh giá.

2.8.2 OpenVINO và tối ưu suy luận

OpenVINO là bộ công cụ hỗ trợ chuyển đổi, tối ưu và triển khai mô hình học sâu trên các thiết bị được hỗ trợ. Quy trình tổng quát gồm chuyển đổi mô hình từ framework nguồn sang biểu diễn OpenVINO, lưu mô hình dưới dạng OpenVINO Intermediate Representation và biên dịch mô hình cho thiết bị thực thi bằng OpenVINO Runtime [?].

Đối với mô hình PyTorch, quá trình chuyển đổi có thể được thực hiện thông qua API `openvino.convert_model`. Mô hình sau chuyển đổi có thể được lưu dưới dạng cặp tệp XML và BIN, sau đó được đọc và biên dịch bởi

OpenVINO Runtime trên thiết bị đích.

Trong hệ thống này, backend của detector cần được tách khỏi các module phía sau. Cả backend PyTorch và backend OpenVINO phải trả về cùng định dạng kết quả gồm tọa độ bounding box, nhãn lớp và confidence score. Nhờ đó, việc thay đổi backend suy luận không làm thay đổi logic của ByteTrack, lane assignment và violation detection.

Nếu kích thước đầu vào của detector được giữ cố định trong quá trình triển khai, mô hình nên sử dụng static input shape. Cấu hình này giúp runtime có thêm thông tin để tối ưu việc cấp phát bộ nhớ và thực thi mô hình.

OpenVINO cung cấp CPU Plugin cho nền tảng Arm. Tuy nhiên, mức hỗ trợ và hiệu quả tối ưu của từng kiểu dữ liệu trên Arm có thể khác so với Intel x86-64. Vì vậy, khả năng chạy mô hình và mức tăng tốc thực tế trên Raspberry Pi 5 cần được xác nhận bằng benchmark trực tiếp trên thiết bị.

2.8.3 Lượng tử hóa INT8

Post-Training Quantization, viết tắt là PTQ, là kỹ thuật chuyển trọng số và activation của mô hình từ độ chính xác cao sang biểu diễn số nguyên có số bit thấp hơn. Với lượng tử hóa tuyến tính 8 bit, một giá trị thực r có thể được ánh xạ thành giá trị lượng tử hóa q theo:

$$q = \text{clip} \left(\text{round} \left(\frac{r}{s} \right) + z, q_{\min}, q_{\max} \right), \quad (2.8.1)$$

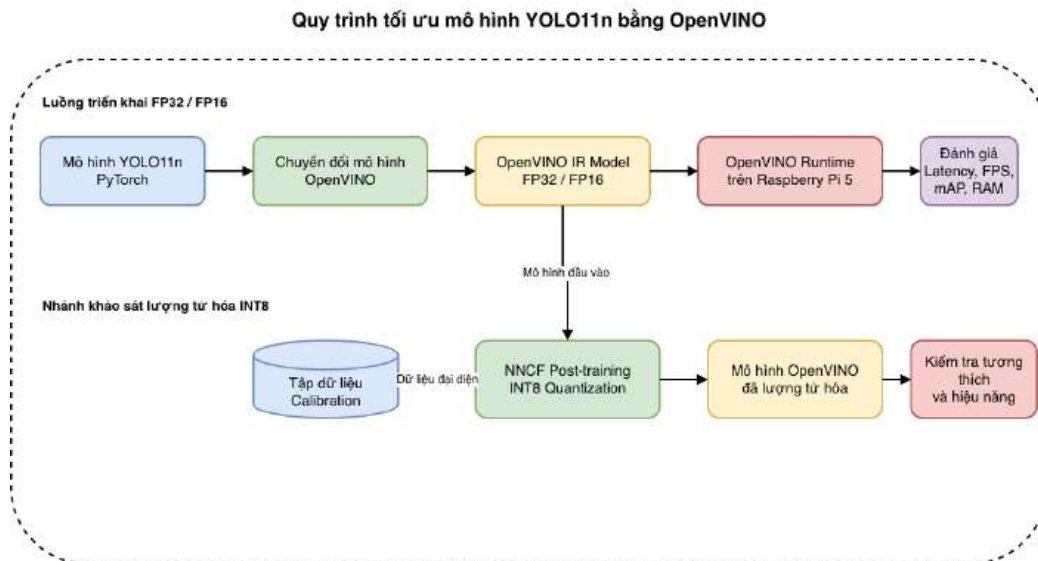
trong đó s là hệ số scale, z là zero-point, còn $q_{\min}$ và $q_{\max}$ xác định miền giá trị của kiểu dữ liệu lượng tử hóa.

NNCF hỗ trợ PTQ bằng cách sử dụng một tập calibration đại diện

để ước lượng miền phân bố của activation. Phương pháp này không yêu cầu huấn luyện lại toàn bộ mô hình, nhưng độ chính xác sau lượng tử hóa vẫn cần được đánh giá lại [?]. Nếu basic quantization gây suy giảm đáng kể, có thể sử dụng quy trình quantization có kiểm soát độ chính xác hoặc giữ một số phép toán nhạy cảm ở độ chính xác cao hơn.

Trong bài toán giao thông, suy giảm độ chính xác có thể biểu hiện qua việc bỏ sót xe máy nhỏ, giảm confidence của phương tiện bị che khuất hoặc tăng sai số bounding box. Vì vậy, mô hình sau lượng tử hóa cần được so sánh với mô hình gốc thông qua mAP, Precision, Recall, latency, FPS và mức sử dụng tài nguyên.

Đối với Raspberry Pi 5, INT8 được xem là một hướng tối ưu cần khảo sát thay vì mặc định được xem là phương án tăng tốc. Khả năng thực thi, kiểu dữ liệu thực tế của các toán tử và mức cải thiện hiệu năng phải được kiểm tra trực tiếp trên OpenVINO Runtime của thiết bị đích. Nếu INT8 không mang lại lợi ích phù hợp, mô hình FP32 hoặc FP16 có thể được sử dụng làm cấu hình triển khai.



Hình 2.8.1: Quy trình dự kiến chuyển đổi, lượng tử hóa và đánh giá mô hình YOLO11n bằng OpenVINO trên thiết bị đích.

2.8.4 Tối ưu ở cấp pipeline

Với camera cố định, cấu trúc làn đường thay đổi chậm hơn vị trí của phương tiện. Do đó, lane detection không nhất thiết phải chạy đầy đủ ở mọi khung hình. Một chiến lược phù hợp là tách quá trình xử lý thành giai đoạn calibration và giai đoạn runtime.

Trong giai đoạn calibration, hệ thống tích lũy candidate qua nhiều khung hình, đánh giá topology và khóa mô hình lane khi kết quả đạt độ ổn định cần thiết. Trong giai đoạn runtime, các boundary đã khóa được tái sử dụng và chỉ được cập nhật định kỳ hoặc khi confidence giảm dưới ngưỡng. Cách xử lý này giúp giảm chi phí tính toán nhưng vẫn cho phép hệ thống thích nghi với những thay đổi nhỏ của hình ảnh.

Ngoài ra, hệ thống cần tắt các cửa sổ debug khi triển khai, tái sử dụng ma trận Homography và mask đã cấp phát, giới hạn chiều dài lịch sử

track, giảm thao tác sao chép ảnh và chỉ ghi video hoặc ảnh vi phạm khi cần thiết. Có thể áp dụng frame skipping cho detector hoặc chạy lane detection theo chu kỳ, nhưng tần suất xử lý cần được lựa chọn sao cho không làm mất các phương tiện di chuyển nhanh.

Profiling cần được thực hiện trên từng stage gồm đọc frame, tiền xử lý, detector, tracker, lane detection, phân tích vi phạm, vẽ kết quả và ghi video. Việc đo riêng từng thành phần giúp xác định đúng bottleneck và tránh tối ưu những module chỉ chiếm tỷ lệ nhỏ trong tổng thời gian xử lý.

2.9 Tổng kết chương

Chương 2 đã hệ thống hóa các cơ sở lý thuyết phục vụ việc xây dựng hệ thống nhận biết phương tiện đi sai làn và đi ngược chiều từ video giao thông. Trước hết, chương phân tích những đặc điểm của dữ liệu video từ camera cố định và các khó khăn do phối cảnh, che khuất, thay đổi kích thước đối tượng, mật độ giao thông và điều kiện chiếu sáng gây ra.

Tiếp theo, chương trình bày nền tảng của phát hiện và theo dõi phương tiện, bao gồm mô hình YOLO11n, chỉ số IoU, confidence score, Non-Maximum Suppression, phương pháp tracking-by-detection và thuật toán ByteTrack. Các nội dung này cung cấp cơ sở để phát hiện phương tiện trong từng khung hình, duy trì Track ID và hình thành lịch sử chuyển động theo thời gian.

Đối với bài toán nhận diện làn đường, chương đã làm rõ vai trò của ROI, Homography và biểu diễn Bird's-Eye View; đồng thời trình bày các kỹ thuật tiền xử lý, trích xuất candidate, Histogram Projection theo chiều sâu, liên kết peak, Polynomial Fitting, đánh giá topology, Temporal Smoothing

và Persistence Map. Từ tập đường biên được phát hiện và ổn định, hệ thống có thể xây dựng cấu trúc làn, xử lý biên ngoài và xác định vùng lưu thông giữa các boundary liền kề.

Trên cơ sở kết quả detection, tracking và mô hình làn, chương tiếp tục trình bày nguyên lý biến đổi điểm đại diện của phương tiện sang không gian BEV, gán phương tiện vào làn và kiểm tra các quy tắc lưu thông. Việc phát hiện sai làn được thực hiện bằng cách so sánh lớp phương tiện với tập phương tiện được phép của từng làn, trong khi hành vi đi ngược chiều được xác định từ lịch sử chuyển động và hướng hợp lệ của làn. Các quyết định vi phạm được xác nhận qua nhiều khung hình bằng cơ chế voting và violation streak nhằm hạn chế cảnh báo sai và tránh đếm trùng cùng một phương tiện.

Cuối cùng, chương phân tích các ràng buộc khi triển khai trên thiết bị nhúng và trình bày những hướng tối ưu gồm giảm kích thước đầu vào, điều chỉnh tần suất thực thi giữa các module, tái sử dụng mô hình làn, tắt chức năng debug, profiling toàn pipeline và khảo sát khả năng sử dụng OpenVINO hoặc lượng tử hóa INT8. Hiệu quả của các phương án này cần được xác nhận thông qua đánh giá trực tiếp về độ chính xác, latency, FPS và mức sử dụng tài nguyên trên thiết bị đích.

Những cơ sở lý thuyết được trình bày trong chương này là nền tảng để Chương 3 xây dựng phương pháp và kiến trúc tổng thể của hệ thống. Chương tiếp theo tập trung mô tả phương án được lựa chọn, luồng xử lý dữ liệu, cách tổ chức và liên kết các module, các tham số quan trọng cũng như quy trình thực nghiệm, thay vì lặp lại nguyên lý chung của từng thuật toán.

CHƯƠNG 3

THIẾT KẾ HỆ THỐNG

3.1 Yêu cầu của hệ thống

Hệ thống được xây dựng nhằm tự động phân tích video giao thông từ camera cố định để nhận biết phương tiện đi sai làn và hỗ trợ phát hiện phương tiện đi ngược chiều. Hệ thống cần kết hợp phát hiện đối tượng, theo dõi đa đối tượng, mô hình hóa làn đường và phân tích hành vi theo thời gian trong một pipeline thống nhất.

Do đầu ra của mỗi module được sử dụng làm đầu vào cho các bước tiếp theo, hệ thống không chỉ cần bảo đảm chất lượng của từng thành phần riêng lẻ mà còn phải duy trì tính nhất quán giữa các module. Khối phát hiện phải cung cấp bounding box, lớp phương tiện và confidence đủ để tracker thực hiện association. Khối lane detection phải tạo ra tập boundary có thứ tự trái-phải rõ ràng và topology hợp lệ. Khối phân tích vi phạm chỉ được đưa ra quyết định khi Track ID, Lane ID, hướng chuyển động và mô hình làn đã đạt các điều kiện ổn định cần thiết.

3.1.1 Yêu cầu chức năng

Hệ thống phải có khả năng tiếp nhận video từ tệp hoặc nguồn camera, đọc dữ liệu theo từng khung hình và xử lý trong vùng quan tâm ROI. Khối phát hiện phương tiện phải nhận diện bốn lớp gồm *motorbike*, *car*, *truck* và *bus*. Sau bước phát hiện, hệ thống phải duy trì Track ID và lịch sử vị trí cho từng phương tiện qua nhiều khung hình.

Đối với phân tích làn đường, hệ thống phải hỗ trợ xác định ROI, biến đổi vùng mặt đường sang Bird's-Eye View, trích xuất lane candidate, mô hình hóa các đường biên và xây dựng cấu trúc làn từ các boundary liền kề. Lane ID phải được đánh số theo một quy ước thống nhất và được sử dụng để gán phương tiện vào làn tương ứng.

Hệ thống phải kiểm tra hai hành vi chính gồm đi sai làn và đi ngược chiều. Sai làn được xác định bằng cách so sánh lớp phương tiện với tập lớp được phép của từng làn. Ngược chiều được xác định từ lịch sử chuyển động của Track ID và hướng lưu thông hợp lệ của làn. Các quy tắc phương tiện được phép, hướng lưu thông và thứ tự Lane ID cần có khả năng cấu hình theo từng vị trí camera.

Kết quả đầu ra phải có khả năng hiển thị bounding box, lớp phương tiện, confidence, Track ID, Lane ID, đường biên làn và trạng thái vi phạm. Hệ thống cũng cần hỗ trợ thống kê số phương tiện và số trường hợp vi phạm, lưu video kết quả, ghi dữ liệu đánh giá, xuất ảnh debug theo từng stage và chạy ở chế độ không hiển thị khi benchmark hiệu năng.

3.1.2 Yêu cầu về độ chính xác và độ ổn định

Chất lượng của khối phát hiện phương tiện được đánh giá thông qua Precision, Recall và mAP trên tập dữ liệu kiểm định. Mô hình cần duy trì khả năng phát hiện các phương tiện có kích thước khác nhau, đặc biệt là xe máy ở xa camera, đồng thời hạn chế nhầm lẫn giữa các lớp phương tiện.

Tracker cần duy trì Track ID ổn định khi phương tiện bị che khuất trong thời gian ngắn hoặc khi confidence của detector suy giảm tạm thời. Chất lượng tracking được xem xét thông qua số trường hợp ID switch, track

bị mất và tỷ lệ phương tiện được duy trì ID liên tục trong vùng quan sát.

Lane detector phải xác định được số lượng boundary phù hợp với cấu trúc mặt đường, duy trì thứ tự trái-phải và hạn chế các boundary giao nhau. Khoảng cách giữa các đường biên liên kề phải dương và không thay đổi bất thường theo chiều sâu. Hệ thống cần hạn chế tạo lane ảo từ mép ROI, vỉa hè, bóng đổ, cạnh phương tiện hoặc các cấu trúc không thuộc vạch làn.

Các boundary phải được ổn định theo thời gian nhằm giảm hiện tượng lane flicker. Mô hình làn chỉ được sử dụng cho quá trình gán lane và phân tích vi phạm khi đạt ngưỡng confidence, model score hoặc điều kiện khóa được xác định trước. Khi topology chưa ổn định hoặc thiếu boundary cần thiết, hệ thống phải duy trì trạng thái chưa xác định thay vì suy luận vi phạm từ dữ liệu không đầy đủ.

Lane ID của mỗi Track ID phải được tổng hợp qua nhiều khung hình để hạn chế dao động khi phương tiện nằm gần đường biên. Hệ thống chỉ xác nhận sai làn hoặc ngược chiều sau khi điều kiện tương ứng tồn tại qua số khung hình tối thiểu. Các track quá mới, có quỹ đạo quá ngắn, có hướng chưa xác định hoặc có Lane ID không ổn định không được sử dụng để kết luận vi phạm.

Sau khi một Track ID được xác nhận vi phạm, trạng thái này phải được lưu để tránh đếm lặp cùng một phương tiện trong các khung hình tiếp theo. Trường hợp ID switch hoặc track bị mất và khởi tạo lại cần được ghi nhận như một nguồn sai số của quá trình thống kê.

3.1.3 Yêu cầu về hiệu năng và khả năng triển khai

Thiết bị triển khai mục tiêu của hệ thống là Raspberry Pi 5. Vì vậy, pipeline cần hạn chế các phép tính lặp lại, thao tác sao chép ảnh không cần thiết và việc tạo nhiều dữ liệu trung gian trong chế độ triển khai.

Do camera được cố định và cấu trúc làn thay đổi chậm hơn phương tiện, hệ thống phải cho phép tách quá trình xử lý làn thành giai đoạn calibration và giai đoạn runtime. Trong giai đoạn calibration, lane detector tích lũy bằng chứng qua nhiều khung hình để lựa chọn và khóa mô hình làn. Sau khi mô hình đã ổn định, các boundary được tái sử dụng; quá trình cập nhật đầy đủ chỉ được thực hiện theo chu kỳ hoặc khi độ tin cậy giảm dưới ngưỡng.

Tần suất chạy detector, lane detector và các bước kiểm tra định kỳ cần có khả năng điều chỉnh thông qua tham số cấu hình. Các chức năng tạo ảnh debug, vẽ overlay, hiển thị cửa sổ và ghi video phải có thể bật hoặc tắt độc lập. Điều này cho phép phân biệt chi phí của thuật toán với chi phí hiển thị và nhập xuất dữ liệu.

Kiến trúc phần mềm cần tách giao diện của detector khỏi backend suy luận. Các backend khác nhau phải trả về cùng một định dạng gồm bounding box, lớp phương tiện và confidence, nhờ đó có thể thay đổi giữa PyTorch và các backend tối ưu khác mà không sửa đổi logic tracking, lane detection, lane assignment và violation analysis.

Hệ thống cần hỗ trợ profiling theo từng stage gồm đọc khung hình, phát hiện phương tiện, tracking, lane detection, phân tích vi phạm, vẽ kết quả và ghi video. Các chỉ số đánh giá hiệu năng gồm latency của từng module, thời gian xử lý toàn pipeline, FPS, mức sử dụng CPU và RAM. Nhiệt độ thiết bị cũng cần được theo dõi trong các thử nghiệm kéo dài để phát hiện

ảnh hưởng của thermal throttling.

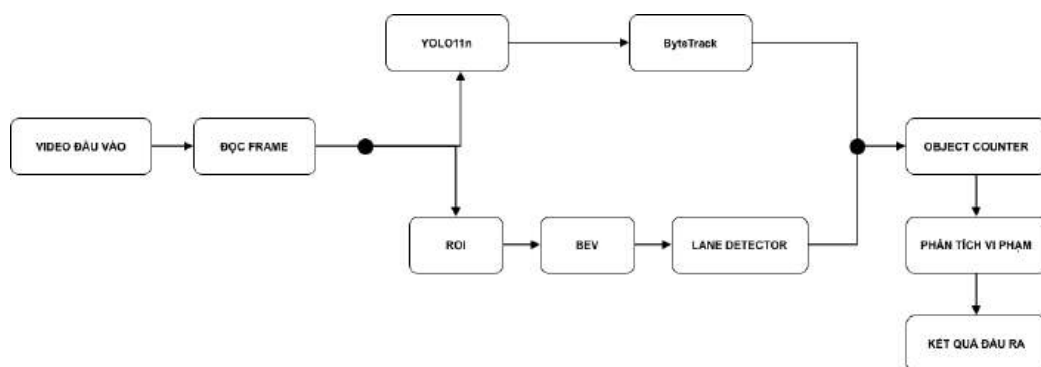
Việc tối ưu chỉ được đánh giá là có hiệu quả khi giảm thời gian xử lý hoặc tài nguyên sử dụng mà không làm suy giảm đáng kể độ chính xác phát hiện, độ ổn định của tracking, cấu trúc làn và kết quả nhận biết vi phạm.

3.2 Tổng quan kiến trúc hệ thống

Hệ thống được tổ chức theo kiến trúc pipeline, trong đó mỗi khung hình được xử lý qua hai nhánh chính gồm phân tích phương tiện và phân tích làn đường. Ở nhánh thứ nhất, YOLO11n phát hiện phương tiện và ByteTrack duy trì Track ID qua các khung hình. Ở nhánh thứ hai, vùng mặt đường trong ROI được biến đổi sang không gian Bird's-Eye View, sau đó lane detector trích xuất, mô hình hóa, ổn định và quản lý trạng thái khóa của các đường biên làn.

Kết quả từ hai nhánh được đưa vào khối phân tích phương tiện–làn. Khối này sử dụng bounding box, lớp phương tiện, Track ID và mô hình lane boundary để xác định Lane ID, cập nhật lịch sử vị trí, ước lượng hướng chuyển động và kiểm tra điều kiện vi phạm. Kết quả cuối cùng được sử dụng để vẽ thông tin lên khung hình, ghi thống kê hoặc lưu video đầu ra.

Về mặt thuật toán, YOLO11n và ByteTrack được xem là hai thành phần liên tiếp. Trong phần triển khai bằng thư viện Ultralytics, hai bước này được gọi thông qua giao diện `model.track` với tùy chọn duy trì trạng thái track qua nhiều khung hình.



Hình 3.2.1: Kiến trúc tổng thể của hệ thống, gồm nhánh phát hiện–theo dõi phương tiện, nhánh mô hình hóa làn và khối phân tích vi phạm.

3.2.1 Các khối chức năng

Khối thu nhận dữ liệu thực hiện mở video hoặc nguồn camera, đọc các thông tin như độ phân giải, tốc độ khung hình và lần lượt lấy từng frame. Khối phát hiện và tracking tiếp nhận frame, thực hiện suy luận YOLO11n và trả về danh sách phương tiện gồm Track ID, bounding box, lớp đối tượng và confidence.

Khối phân tích làn tiếp nhận ROI của mặt đường, thực hiện phép biến đổi Homography sang không gian BEV, tạo các mask bằng chứng, trích xuất candidate boundary, lựa chọn topology và cập nhật mô hình làn theo thời gian. Module này đồng thời quản lý các trạng thái calibration, ready và lock nhằm quyết định khi nào mô hình lane có thể được sử dụng cho các bước phía sau.

Khối lane semantics lưu trữ hoặc xây dựng cấu hình về thứ tự Lane ID, tập lớp phương tiện được phép và hướng lưu thông hợp lệ của từng làn. Các quy tắc này phụ thuộc vào số lượng làn, quy ước đánh số và tổ chức giao thông tại vị trí camera, thay vì chỉ được suy luận từ hình ảnh.

Khởi phân tích phương tiện–làn sử dụng điểm đại diện của bounding box và các lane boundary để xác định Lane ID cho từng Track ID. Module này quản lý lịch sử lane, lịch sử vị trí, hướng chuyển động, wrong-lane streak, wrong-way streak và trạng thái đã ghi nhận vi phạm.

Khi mô hình làn chưa đạt trạng thái sẵn sàng, hệ thống vẫn có thể thực hiện phát hiện và tracking nhưng không đưa ra kết luận vi phạm dựa trên lane. Trong trường hợp này, Lane ID hoặc trạng thái vi phạm được giữ ở giá trị *unknown*.

Khởi hiển thị sử dụng kết quả từ các module để vẽ bounding box, Track ID, lớp phương tiện, lane boundary, vùng làn và trạng thái vi phạm lên khung hình. Khởi profiling đo thời gian xử lý của các stage quan trọng và ghi kết quả ra terminal hoặc tệp benchmark.

3.2.2 Tổ chức module phần mềm

File `main.py` là điểm khởi chạy của chương trình. Module này đọc tham số dòng lệnh, kiểm tra đường dẫn, khởi tạo cấu hình và tạo đối tượng điều phối xử lý video. File `arguments.py` định nghĩa các tham số liên quan đến đường dẫn video, mô hình phát hiện, kích thước ảnh đầu vào, nguồn ROI, nguồn lane, chế độ ánh sáng, debug, hiển thị và profiling.

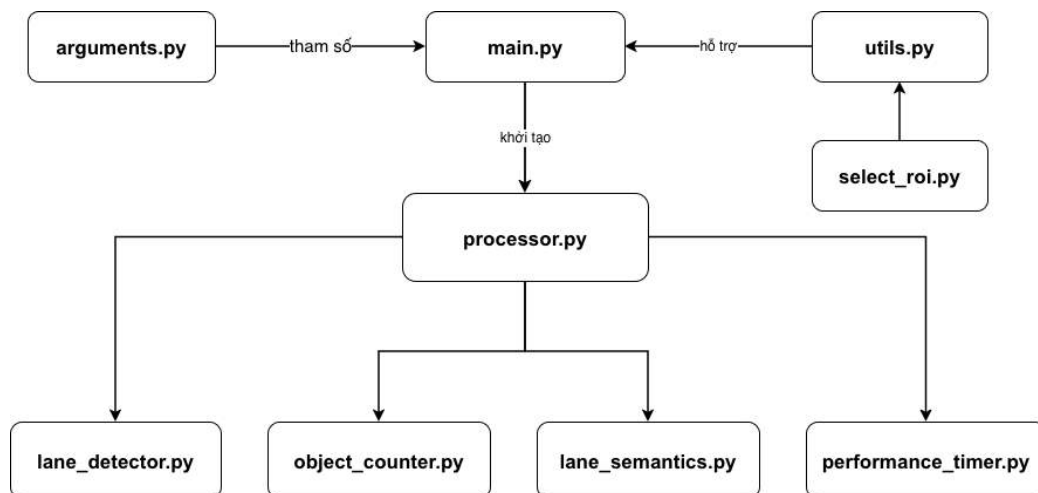
File `processor.py` chịu trách nhiệm điều phối vòng lặp chính. Module này đọc từng frame, gọi detector–tracker, cập nhật lane detector theo lịch xử lý, truyền kết quả cho object counter, thực hiện vẽ overlay và quản lý đầu ra.

File `lane_detector.py` thực hiện xử lý ROI, Homography, tạo ảnh BEV, tiền xử lý lane mask, trích xuất candidate, Polynomial Fitting, đánh giá topology, Temporal Smoothing và quản lý trạng thái khóa mô hình làn.

File `object_counter.py` tiếp nhận các track và mô hình làn để quản lý lịch sử vị trí, Lane ID, hướng chuyển động, bộ đếm phương tiện và trạng thái vi phạm. Mặc dù có tên là object counter, module này còn thực hiện các chức năng phân tích phương tiện–làn và duy trì trạng thái theo Track ID.

File `lane_semantics.py` mô tả quy tắc của từng làn, gồm tập lớp phương tiện được phép, hướng lưu thông và quy ước đánh số Lane ID. Kết quả từ module này được truyền cho `object_counter.py` sau khi cấu trúc làn đã đạt độ ổn định cần thiết.

File `select_roi.py` hỗ trợ lựa chọn các điểm ROI trên frame tham chiếu. File `utils.py` cung cấp các hàm dùng chung liên quan đến đường dẫn, đọc cấu hình, xử lý video và chuyển đổi dữ liệu. File `performance_timer.py` đo thời gian xử lý của từng thành phần trong pipeline.



Hình 3.2.2: Quan hệ gọi và trao đổi dữ liệu giữa các module phần mềm trong hệ thống.

3.2.3 Luồng xử lý một khung hình

Mỗi vòng lặp bắt đầu bằng việc đọc một khung hình từ nguồn video. Khung hình này được truyền vào hàm `model.track` với tùy chọn `persist=True` để thực hiện phát hiện phương tiện và duy trì Track ID qua các khung hình liên tiếp. Kết quả trả về sau đó được chuẩn hóa thành danh sách track gồm ID, bounding box, lớp phương tiện và confidence trước khi chuyển sang các bước xử lý tiếp theo.

Sau khi có danh sách track, processor kiểm tra trạng thái hiện tại của mô hình làn. Nếu lane detector đã khóa, hệ thống tái sử dụng trực tiếp mô hình làn đã ổn định mà không cần chạy lại toàn bộ pipeline nhận diện làn ở khung hình hiện tại. Nếu mô hình làn chưa khóa, processor tiếp tục kiểm tra xem khung hình hiện tại có đến chu kỳ cập nhật lane hay không.

Khi thỏa điều kiện cập nhật, pipeline lane detection được thực hiện. Quá trình này bao gồm trích xuất ROI, biến đổi sang không gian Bird's-Eye View, tạo lane mask, phát hiện candidate boundary, fit đường cong, lựa chọn topology và cập nhật lịch sử lane. Sau đó, lane detector kiểm tra các điều kiện sẵn sàng hoặc điều kiện khóa để quyết định mô hình làn hiện tại có thể được sử dụng cho các bước phía sau hay chưa. Nếu chưa đến chu kỳ cập nhật, hệ thống bỏ qua bước cập nhật lane và giữ nguyên trạng thái làn hiện tại.

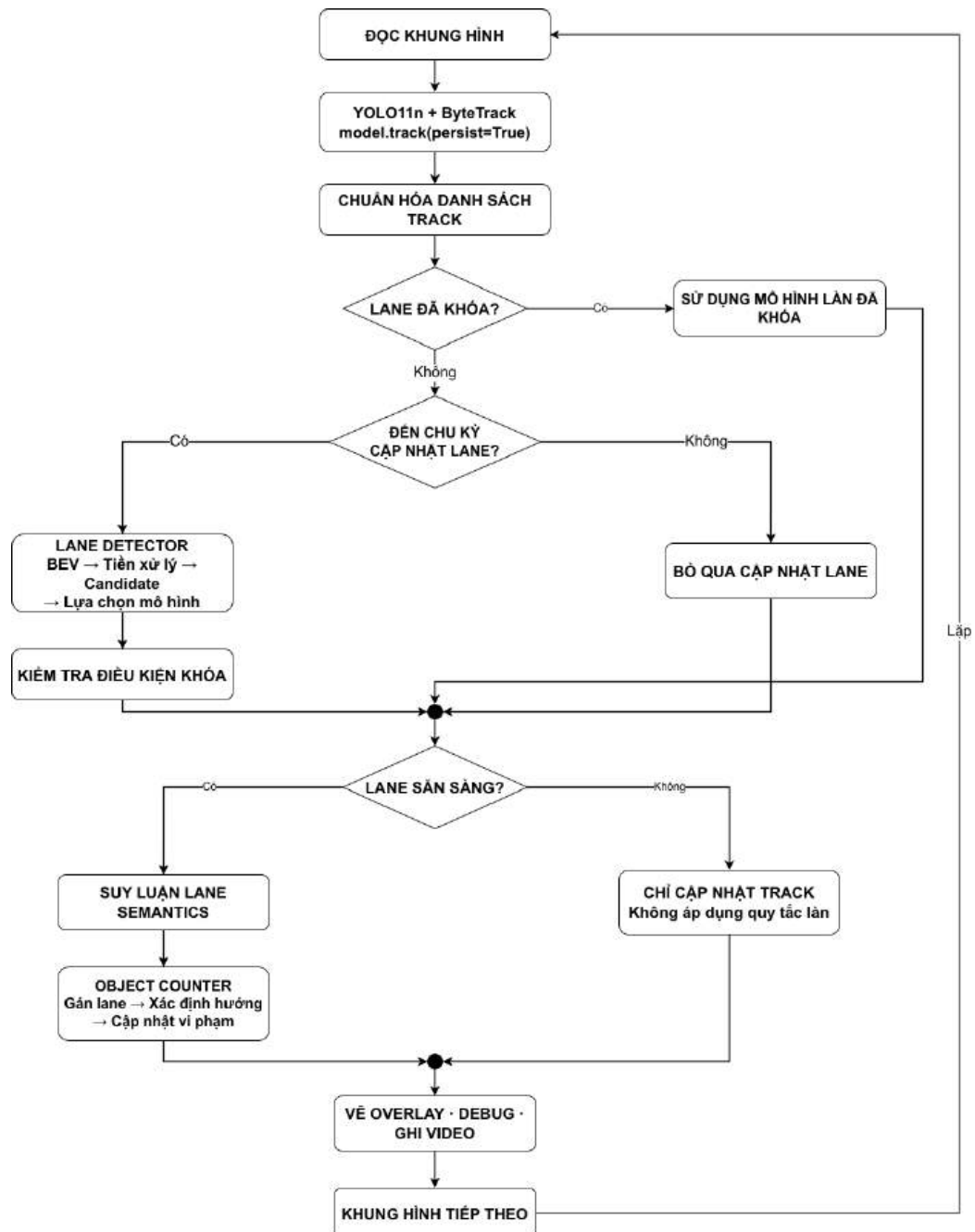
Khi mô hình làn đạt trạng thái sẵn sàng, hệ thống nạp hoặc suy luận lane semantics tương ứng với số lượng làn, thứ tự Lane ID và hướng lưu thông tại vị trí camera. Các boundary cùng với cấu hình semantics được truyền cho object counter để thực hiện gán lane và phân tích vi phạm.

Với mỗi Track ID, object counter cập nhật lịch sử vị trí, lịch sử lane

và hướng chuyển động. Nếu lớp phương tiện không thuộc tập lớp được phép của làn, bộ đếm wrong-lane streak được tăng. Nếu hướng chuyển động ổn định khác với hướng hợp lệ của làn, bộ đếm wrong-way streak được tăng. Vi phạm chỉ được xác nhận khi streak tương ứng đạt số khung hình tối thiểu đã đặt.

Nếu mô hình làn chưa sẵn sàng, hoặc Lane ID và hướng chuyển động chưa đủ tin cậy, hệ thống chỉ cập nhật lịch sử track mà chưa áp dụng quy tắc làn. Trong trường hợp này, trạng thái lane hoặc vi phạm được giữ ở mức *unknown* và bộ đếm vi phạm không được tăng.

Sau khi quá trình phân tích hoàn tất, hệ thống vẽ overlay, cập nhật thống kê, hiển thị các cửa sổ debug nếu được bật và ghi video nếu người dùng yêu cầu. Vòng lặp sau đó chuyển sang khung hình tiếp theo và lặp lại cho đến khi video kết thúc.



Hình 3.2.3: Luồng xử lý một khung hình trong hệ thống, từ phát hiện và tracking đến cập nhật mô hình làn, gán Lane ID và xác nhận vị phạm.

3.3 Phương pháp đề xuất

Phương pháp đề xuất kết hợp YOLO11n, ByteTrack, xử lý hình học trong không gian Bird's-Eye View và các cơ chế ổn định theo thời gian. Trong đó, YOLO11n và ByteTrack cung cấp thông tin về lớp, bounding box, Track ID và lịch sử chuyển động của phương tiện. Phần xử lý lần thực hiện trích xuất candidate boundary, mô hình hóa đường cong, lựa chọn topology và tích lũy bằng chứng qua nhiều khung hình. Kết quả từ hai nhánh được kết hợp để gán phương tiện vào làn, ước lượng hướng và xác nhận vi phạm.

Đóng góp chính của phương pháp tập trung vào pipeline xây dựng mô hình làn cho camera cố định, gồm phân tích candidate theo nhiều dải chiều sâu, lựa chọn đồng thời tập boundary theo ràng buộc topology, xử lý boundary ngoài, ổn định theo thời gian và chỉ kích hoạt logic vi phạm khi mô hình làn đạt trạng thái tin cậy.

3.3.1 Phát hiện, theo dõi và xác định điểm đại diện

Mỗi khung hình được đưa vào YOLO11n với kích thước suy luận được cấu hình từ tham số dòng lệnh. Trong phần triển khai, Ultralytics thực hiện phát hiện và tracking thông qua hàm `model.track` với tùy chọn `persist=True`. Đầu ra được chuẩn hóa thành danh sách track gồm bounding box, lớp phương tiện, confidence và Track ID. Các lớp ngoài phạm vi *motorbike*, *car*, *truck* và *bus* được loại khỏi quá trình phân tích vi phạm.

Với bounding box $B = (x_1, y_1, x_2, y_2)$, điểm đại diện của phương tiện được chọn là trung điểm cạnh đáy:

$$P_v = \left(\frac{x_1 + x_2}{2}, y_2 \right). \quad (3.3.1)$$

Điểm bottom-center gần vị trí phương tiện tiếp xúc với mặt đường hơn tâm hình học của bounding box, do đó phù hợp hơn cho bài toán gán làn. Điểm này được biến đổi sang không gian BEV trước khi so sánh với các lane boundary.

Khi mô hình làn đã sẵn sàng, hàm `get_vehicle_lane` tính vị trí các boundary tại cùng tọa độ y với điểm phương tiện và xác định khoảng giữa hai boundary chứa điểm đó. Lane index thô được lưu vào lịch sử của từng Track ID. Majority voting và lane-change streak được sử dụng để giảm hiện tượng Lane ID dao động khi bounding box hoặc boundary thay đổi nhẹ. Stable Lane ID chỉ được cập nhật khi lane mới xuất hiện ổn định qua số khung hình tối thiểu.

3.3.2 Xác định ROI và xây dựng Bird's-Eye View

ROI của mỗi video có thể được tải từ file cấu hình, truyền trực tiếp qua tham số dòng lệnh hoặc lựa chọn bằng công cụ tương tác. ROI đa giác được sử dụng để tạo road mask và giới hạn vùng mặt đường cần xử lý. Bên cạnh đó, bốn điểm nguồn đại diện cho mặt phẳng đường được sử dụng để tính ma trận Homography và ánh xạ sang một hình chữ nhật trong không gian BEV.

Ma trận Homography $\mathbf{H}$ và ma trận nghịch đảo $\mathbf{H}^{-1}$ được tính một lần khi khởi tạo lane detector. Ảnh mặt đường được biến đổi sang BEV có kích thước cố định. Road mask sau biến đổi xác định toàn bộ vùng hợp lệ, trong khi một inner mask được co nhẹ được sử dụng cho quá trình tìm lane

candidate nhằm giảm nhiễu sát mép.

Mép ROI không được xem trực tiếp là lane boundary. ROI chỉ giới hạn vùng xử lý và có thể chứa lề đường, vỉa hè hoặc vùng không thuộc phần đường chính. Khi cần suy luận boundary ngoài, hệ thống sử dụng thêm khoảng cách làn, bằng chứng mép đường và các ràng buộc topology thay vì mặc định lấy mép ROI.

3.3.3 Trích xuất bằng chứng và candidate vạch làn

Ảnh BEV được biến đổi sang các kênh màu và độ sáng thích hợp. CLAHE tăng tương phản cục bộ; white mask và yellow mask giữ lại những vùng có màu sắc tương ứng với lane marking; gradient mask làm nổi bật các cạnh gần phương dọc; local contrast hỗ trợ phát hiện những vùng vạch có độ tương phản thấp.

Các nguồn bằng chứng được kết hợp có điều kiện trong road inner mask để tạo Lane Search Binary. Việc kết hợp này nhằm giữ các pixel có khả năng thuộc vạch sơn nhưng hạn chế cạnh phương tiện, texture mặt đường, bóng đổ và vùng phản xạ.

Sau morphology và lọc connected component, ảnh BEV được chia thành nhiều dải theo chiều sâu. Histogram của từng dải được tính để phát hiện các peak cục bộ. Các peak ở những dải liên tiếp được liên kết dựa trên khoảng cách ngang, mức thay đổi vị trí và số dải được phép bỏ qua, từ đó hình thành candidate boundary. Ngoài liên kết theo dải, histogram tổng hợp được sử dụng để khởi tạo một nhánh tìm kiếm bổ sung, giúp phát hiện thêm những candidate mà quá trình band linking có thể bỏ sót.

3.3.4 Mô hình hóa và lựa chọn cấu trúc làn

Candidate đạt yêu cầu về số điểm và độ phủ được mô hình hóa bằng đa thức bậc hai:

$$x(y) = ay^2 + by + c. \quad (3.3.2)$$

Sau lần fit đầu tiên, residual giữa điểm quan sát và đường cong được tính để loại bỏ outlier. Một hành lang tìm kiếm hẹp quanh candidate sau đó được sử dụng để thu thập lại lane evidence và tinh chỉnh đường cong về gần tâm vạch sơn.

Mỗi candidate được đánh giá dựa trên lượng pixel hỗ trợ, độ phủ theo chiều sâu, tính liên tục, fitting residual, hướng, độ cong, khoảng cách tới mép vùng đường và mức chồng lấn với vùng phương tiện. Các candidate nằm quá gần nhau được kiểm tra và hợp nhất nhằm tránh xem hai cạnh của cùng một vạch sơn là hai boundary độc lập.

Model selector đánh giá các tổ hợp candidate thay vì lựa chọn độc lập những candidate có điểm cao nhất. Mỗi tổ hợp được sắp xếp từ trái sang phải, kiểm tra crossing tại nhiều giá trị y , đánh giá chiều rộng làn và mức biến thiên hình học. Điểm tổng quát của một mô hình được biểu diễn bởi:

$$\text{Score}(M) = \sum_j w_j S_j(M) - P(M), \quad (3.3.3)$$

trong đó $S_j(M)$ là các thành phần đánh giá mức hỗ trợ, tính liên tục, chất lượng hình học và topology; w_j là trọng số tương ứng; còn $P(M)$ là tổng các thành phần phạt. Penalty được áp dụng cho crossing, khoảng cách không hợp lý, boundary ảo và trường hợp bỏ qua candidate có bằng chứng

mạnh.

Boundary ngoài được ưu tiên lấy từ bằng chứng quan sát trực tiếp. Khi không có candidate phù hợp, hệ thống có thể ngoại suy boundary từ chiều rộng làn trung vị và giới hạn kết quả trong road mask. Boundary ngoại suy được đánh dấu là virtual boundary và có độ tin cậy thấp hơn boundary được phát hiện trực tiếp.

Nếu mô hình có n boundary hợp lệ và bao phủ đầy đủ vùng đường cần xét, số vùng làn được xác định bởi:

$$N_{\text{lane}} = n - 1. \quad (3.3.4)$$

3.3.5 Ổn định và khóa mô hình làn

Lane detector không xác nhận mô hình chỉ từ một lần quan sát. Các mô hình được tạo qua nhiều lần cập nhật được lưu và nhóm theo số lượng boundary cùng cấu trúc topology. Với các mô hình tương thích, vị trí boundary tại nhiều mức y được tổng hợp để xây dựng consensus model.

Boundary drift giữa mô hình hiện tại và consensus model được tính theo:

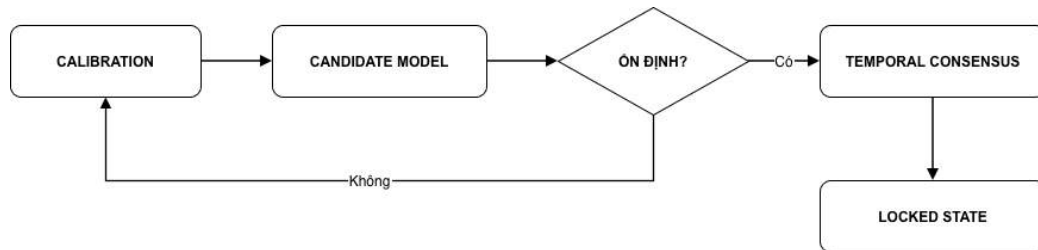
$$D = \frac{1}{nK} \sum_{i=1}^n \sum_{k=1}^K |x_{i,\text{current}}(y_k) - x_{i,\text{consensus}}(y_k)|. \quad (3.3.5)$$

Lock hit chỉ được tăng khi số lượng boundary ổn định, topology hợp lệ, confidence đạt ngưỡng và boundary drift đủ nhỏ. Khi mô hình không thỏa điều kiện, bộ đếm được giữ nguyên hoặc giảm theo chính sách cài đặt.

Khi số lần xác nhận đạt ngưỡng lock history, consensus model được sử dụng làm mô hình tham chiếu trong giai đoạn runtime. Các boundary

này được tái sử dụng cho quá trình gán làn và không phải trải qua toàn bộ pipeline model selection ở mọi khung hình. Tuy nhiên, hệ thống vẫn có thể kiểm tra hoặc cập nhật định kỳ để phát hiện trường hợp confidence suy giảm hoặc mô hình không còn phù hợp.

Processor chỉ truyền lane semantics và kích hoạt logic vi phạm khi mô hình làn đã đạt trạng thái khóa. Cơ chế này hạn chế việc gán làn và kết luận vi phạm dựa trên một topology chưa ổn định.



Hình 3.3.1: Quá trình tích lũy candidate model, xây dựng temporal consensus và chuyển từ giai đoạn calibration sang trạng thái khóa.

3.3.6 Gán làn và phát hiện vi phạm

Khi mô hình làn đã khóa, object counter sử dụng điểm bottom-center của từng phương tiện và hàm `get_vehicle_lane` để xác định Lane ID. Lane history và lane-change streak được dùng để ổn định kết quả; Lane ID chỉ thay đổi khi giá trị mới tồn tại đủ số khung hình liên tiếp.

Lane semantics được lựa chọn từ cấu hình tương ứng với vị trí camera, số lượng làn, thứ tự Lane ID và hướng lưu thông. Mỗi làn được gán tập `allowed_classes` và hướng hợp lệ. Nếu Lane ID nằm ngoài phạm vi, cấu hình bị thiếu hoặc mô hình làn chưa sẵn sàng, trạng thái được giữ là *unknown* và không xác nhận vi phạm.

Điều kiện sai làn của track i được xác định bởi:

$$c_i \notin \text{AllowedClasses}(l_i), \quad (3.3.6)$$

trong đó c_i là lớp phương tiện và l_i là Stable Lane ID. Wrong-lane streak chỉ tăng khi điều kiện này được duy trì liên tục.

Hướng chuyển động được xác định từ lịch sử tọa độ của Track ID và được làm ổn định bằng cơ chế voting. Với hướng hợp lệ của làn l_i là $D_{\text{expected}}(l_i)$, điều kiện ngược chiều được xác định khi:

$$D_{\text{track},i} \neq D_{\text{expected}}(l_i). \quad (3.3.7)$$

Các track có quỹ đạo quá ngắn, độ dịch chuyển nhỏ, hướng chưa xác định hoặc Lane ID không ổn định không được dùng để kết luận ngược chiều. Vi phạm chỉ được xác nhận khi streak tương ứng đạt số khung hình tối thiểu. Những Track ID đã được ghi nhận được lưu riêng theo loại vi phạm để tránh đếm trùng.

3.3.7 Tối ưu lịch xử lý và profiling

Processor điều chỉnh tần suất thực thi của các module nhằm giảm chi phí tính toán. Trong giai đoạn calibration, lane detection được chạy theo *lane detect interval* thay vì ở mọi khung hình. Sau khi mô hình làn được khóa, các boundary được tái sử dụng và quá trình tiền xử lý, candidate extraction và model selection đầy đủ chỉ được thực hiện khi đến chu kỳ kiểm tra hoặc khi mô hình cần cập nhật.

Các chức năng tạo debug view, gọi imshow, vẽ overlay và ghi video

có thể chiếm một phần đáng kể thời gian xử lý. Vì vậy, các chức năng này có thể được bật hoặc tắt độc lập. Khi benchmark pipeline cốt lõi, chế độ hiển thị và ghi video được tắt để tránh làm sai lệch kết quả.

Performance Timer ghi nhận thời gian gần nhất, thời gian trung bình, các phân vị P50 và P95 cùng số lần gọi của những stage được instrument trong code, chẳng hạn đọc frame, YOLO tracking, warp perspective, preprocessing, candidate extraction, model selection, lane assignment, object counting, vẽ overlay, display và video writing.

Kiến trúc detector được tách khỏi các module phía sau thông qua một định dạng track thống nhất gồm Track ID, bounding box, lớp phương tiện và confidence. Nhờ đó, backend PyTorch có thể được thay thế bằng OpenVINO hoặc backend khác mà không làm thay đổi logic lane detection và violation analysis.

3.4 Cài đặt thực nghiệm

3.4.1 Nguồn dữ liệu và bộ dữ liệu sử dụng

Dữ liệu của đề tài gồm tập ảnh được sử dụng để huấn luyện và đánh giá mô hình phát hiện phương tiện, cùng với tập video dùng để kiểm thử toàn bộ pipeline. Bên cạnh dữ liệu có sẵn, tác giả trích xuất, chọn lọc và gán nhãn bổ sung các khung hình từ video giao thông nhằm phản ánh điều kiện giao thông hỗn hợp tại Việt Nam.

Bộ dữ liệu phát hiện gồm bốn lớp phương tiện là *motorbike*, *car*, *truck* và *bus*. Các khung hình bao phủ nhiều mức mật độ giao thông, góc quan sát, kích thước phương tiện và điều kiện chiếu sáng. Xe máy chiếm phần lớn số đối tượng, trong khi xe tải và xe buýt xuất hiện ít hơn, tạo ra sự

mất cân bằng đáng kể giữa các lớp.

Các video kiểm thử toàn pipeline được ghi nhận bằng camera cố định. Mỗi video có cấu hình ROI riêng và bao gồm các trường hợp vạch liền, vạch đứt, vạch mờ, che khuất, đường thẳng hoặc đường cong nhẹ. Những điều kiện này được sử dụng để đánh giá khả năng phát hiện phương tiện, duy trì Track ID, xây dựng mô hình làn và xác nhận vi phạm.



Hình 3.4.1: Một số điều kiện xuất hiện trong dữ liệu kiểm thử, gồm mật độ giao thông cao, vạch làn rõ, góc quan sát khác nhau và phương tiện có kích thước lớn.

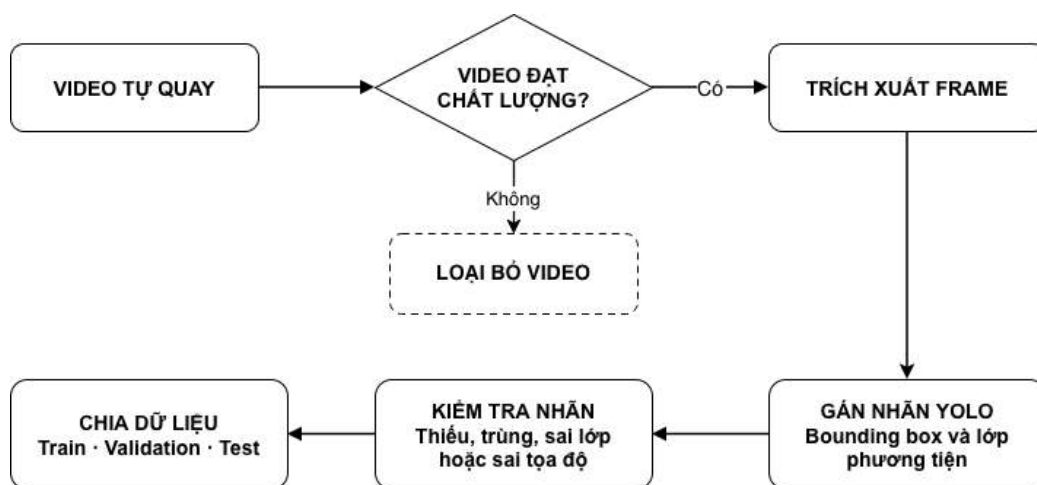
3.4.2 Quy trình xây dựng và gán nhãn dữ liệu

Video được kiểm tra chất lượng trước khi trích xuất khung hình. Các đoạn video bị rung mạnh, mất nét, có độ phân giải không phù hợp hoặc không quan sát rõ mặt đường được loại bỏ. Khung hình được trích xuất theo

khoảng thời gian xác định nhằm giảm số lượng ảnh gần trùng lặp.

Mỗi phương tiện được gán một bounding box và nhãn lớp tương ứng theo định dạng YOLO. Bounding box bao phủ phần thân phương tiện có thể quan sát được và hạn chế chứa quá nhiều nền. Với phương tiện bị che khuất, nhãn được tạo dựa trên phần nhìn thấy nếu vẫn có thể xác định lớp một cách đáng tin cậy. Các đối tượng quá nhỏ, mờ hoặc không xác định được lớp được loại khỏi tập dữ liệu.

Sau khi gán nhãn, dữ liệu được kiểm tra các lỗi gồm thiếu file nhãn, bounding box trùng, sai chỉ số lớp, tọa độ nằm ngoài miền chuẩn hóa và bounding box có kích thước bất thường. Việc phân chia tập train, validation và test được thực hiện theo video hoặc theo các đoạn video tách biệt nhằm hạn chế rò rỉ các khung hình gần giống nhau giữa các tập.



Hình 3.4.2: Quy trình kiểm tra video, trích xuất khung hình, gán nhãn, kiểm tra nhãn và phân chia dữ liệu.

Phân bố trên cho thấy số lượng mẫu giữa các lớp không cân bằng. Xe máy chiếm tỷ lệ lớn, trong khi xe tải và xe buýt có số lượng hạn chế. Đặc điểm này được xem xét khi phân tích Precision, Recall và mAP theo từng

Bảng 3.4.1: Phân chia bộ dữ liệu phát hiện phương tiện.

Thành phần	Số lượng ảnh	Tỷ lệ
Train	949	76,78%
Validation	167	13,51%
Test	120	9,71%
Tổng cộng	1236	100%

Bảng 3.4.2: Phân bố số đối tượng theo lớp và tập dữ liệu.

Lớp	Train	Validation	Test	Tổng
Motorbike	25428	4372	3121	32921
Car	8336	1455	1185	10976
Truck	240	46	32	318
Bus	270	60	41	371

lớp. Tuy nhiên, sự khác biệt về hiệu năng không chỉ phụ thuộc vào số lượng mẫu mà còn chịu ảnh hưởng của kích thước đối tượng, mức che khuất và độ đa dạng của dữ liệu.

3.4.3 Cấu hình huấn luyện mô hình phát hiện

YOLO11n được huấn luyện để phát hiện bốn lớp phương tiện. Các thông số trong Bảng 3.4.3 được lấy từ cấu hình và log của lần huấn luyện tạo ra checkpoint best.pt được sử dụng trong hệ thống.

Các phép augmentation được sử dụng theo cấu hình huấn luyện thực tế và có thể gồm thay đổi độ sáng, saturation, scale, translation và Mosaic. Các tham số augmentation cần được lấy từ log hoặc file cấu hình của run tương ứng, thay vì suy đoán từ cấu hình mặc định.

Bảng 3.4.3: Cấu hình huấn luyện mô hình YOLO11n.

Thông số	Giá trị
Mô hình	YOLO11n
Số lớp	4
Tên lớp	Motorbike, car, truck và bus
Kích thước ảnh huấn luyện	640×640 pixel
Kích thước ảnh suy luận	224×224 pixel
runtime	
Số epoch cấu hình	40
Batch size	16
Optimizer	AdamW
Learning rate ban đầu (lr0)	0,001
Tỷ lệ learning rate cuối (lrf)	0,01
Lịch điều chỉnh learning rate	Cosine
Số epoch warm-up	2
Thiết bị huấn luyện	NVIDIA Tesla T4, CUDA device 0
Số worker	4
Chế độ cache	Disk
Seed	42

3.4.4 Cấu hình hệ thống và môi trường triển khai

Hệ thống được phát triển bằng Python với các thư viện chính gồm OpenCV, NumPy, Ultralytics, PyTorch và psutil. Máy tính phát triển được sử dụng để huấn luyện mô hình YOLO11n, kiểm thử pipeline, điều chỉnh tham số và phân tích các kết quả trung gian. Raspberry Pi 5 chạy hệ điều hành Raspberry Pi OS 64-bit được lựa chọn làm thiết bị triển khai mục tiêu.

Trong quá trình phát triển và kiểm thử pipeline trên máy tính, PyTorch và Ultralytics được sử dụng làm backend suy luận chính cho mô hình phát hiện phương tiện. Để khảo sát khả năng triển khai trên Raspberry Pi 5, mô hình được chuyển đổi sang OpenVINO và áp dụng lượng tử hóa INT8. Các backend được chuẩn hóa để trả về cùng một định dạng đầu ra gồm bounding box, lớp phương tiện và confidence, nhờ đó việc thay đổi backend không làm thay đổi logic tracking, lane detection, lane assignment và phân tích vi phạm.

PyTorch được xem là cấu hình phát triển, trong khi OpenVINO INT8 được sử dụng như cấu hình thực nghiệm phục vụ khảo sát hiệu năng module phát hiện trên Raspberry Pi 5. Kết quả so sánh giữa các cấu hình được trình bày trong Chương 4.

Các tham số chính của lane detector được trình bày trong Bảng 3.4.4. Các giá trị này được lấy từ phiên bản mã nguồn và cấu hình sử dụng trong quá trình thực nghiệm. Một số tham số được thay đổi theo chế độ ánh sáng nhằm tăng khả năng duy trì lane boundary trong điều kiện quan sát ban đêm.

Giá trị *maximum missed frames* ở chế độ Night được đặt lớn hơn so với chế độ Day/Auto vì vạch làn trong điều kiện ban đêm có thể bị mất tạm

Bảng 3.4.4: Các tham số chính của lane detector.

Thông số	Giá trị	Ý nghĩa
Kích thước BEV	640×360 pixel	Kích thước ảnh mặt đường sau phép biến đổi phối cảnh.
Bậc đa thức	2	Sử dụng đa thức bậc hai để mô hình hóa lane boundary.
Chế độ ánh sáng	Day, Night hoặc Auto	Lựa chọn nhóm tham số tiền xử lý phù hợp với điều kiện chiếu sáng.
Số depth band	14 với Day/Auto; 18 với Night	Số dải chiều sâu dùng để tính histogram cục bộ và phát hiện peak.
Calibration frames	40 khung hình	Số khung hình được sử dụng trong giai đoạn hiệu chỉnh mô hình làn.
Calibration stride	2 khung hình	Khoảng cách lấy mẫu giữa các khung hình trong quá trình hiệu chỉnh.
Lock history	12 lần cập nhật	Số lần xác nhận mô hình hợp lệ cần thiết trước khi khóa mô hình làn.
Maximum missed frames	10 với Day/Auto; 45 với Night	Số lần cập nhật tối đa mà boundary có thể không được quan sát trước khi bị giảm độ tin cậy hoặc loại bỏ.
Persistence alpha	0,78	Trọng số của thông tin lịch sử khi tích lũy và làm ổn định bằng chứng làn theo thời gian.
Lane detect interval	3 khung hình	Lane detector được kích hoạt sau mỗi ba khung hình video thay vì chạy ở mọi khung hình.
Minimum boundary distance	43 px với chiều rộng BEV là 640 px	Khoảng cách ngang tối thiểu giữa hai boundary, tương đương khoảng 0.067 lần chiều

thời do độ tương phản thấp, ánh sáng đèn xe hoặc phản xạ mặt đường. Việc duy trì boundary trong thời gian dài hơn giúp hạn chế mô hình làn bị xóa hoặc thay đổi chỉ vì một số lần quan sát không ổn định.

Tham số *lane detect interval* điều khiển tần suất processor gọi toàn bộ lane detector, trong khi *calibration stride* điều khiển khoảng lấy mẫu được sử dụng trong quá trình hiệu chỉnh. Việc phân biệt hai tham số này giúp giảm chi phí xử lý nhưng vẫn duy trì đủ số quan sát để xây dựng mô hình làn ổn định.

3.4.5 Giao thức đánh giá

Mô hình phát hiện phương tiện được đánh giá bằng Precision, Recall, F1-score, mAP@0.5 và mAP@0.5:0.95 theo từng lớp. Tập dữ liệu và kích thước ảnh sử dụng trong quá trình đánh giá phải được ghi rõ cùng với kết quả.

Tracking được đánh giá thông qua số phương tiện duy trì Track ID liên tục, số trường hợp ID switch, track bị mất và track trùng. Do chưa xây dựng ground truth theo định dạng MOT đầy đủ, các chỉ số này được thống kê thủ công trên các đoạn video được lựa chọn và không được xem là các metric MOT chuẩn như MOTA, IDF1 hoặc HOTA.

Lane detection được đánh giá thông qua số boundary tham chiếu và số boundary được phát hiện, số vùng làn, model score, marking support, geometry quality, boundary drift, thời gian khóa và boundary error. Ground truth boundary được đánh dấu thủ công trong không gian BEV. Boundary error được tính theo khoảng cách ngang giữa boundary dự đoán và boundary tham chiếu tại nhiều mức y , sau đó lấy trung bình trên các boundary và

khung hình được đánh giá.

Khi có ground truth đầy đủ ở cấp phương tiện hoặc cấp Track ID, kết quả phát hiện vi phạm có thể được đánh giá thông qua True Positive, False Positive, False Negative, Precision, Recall và F1-score. Mỗi phương tiện chỉ được thống kê một lần cho mỗi loại vi phạm.

Trong phạm vi thực nghiệm hiện tại, bộ dữ liệu chưa có ground truth đầy đủ cho hành vi sai làn và đi ngược chiều ở cấp Track ID. Vì vậy, chức năng phát hiện vi phạm được kiểm tra định tính thông qua các trường hợp đại diện trên video, tập trung vào khả năng gán Lane ID, duy trì trạng thái qua nhiều frame và tạo cảnh báo đúng theo luật làn.

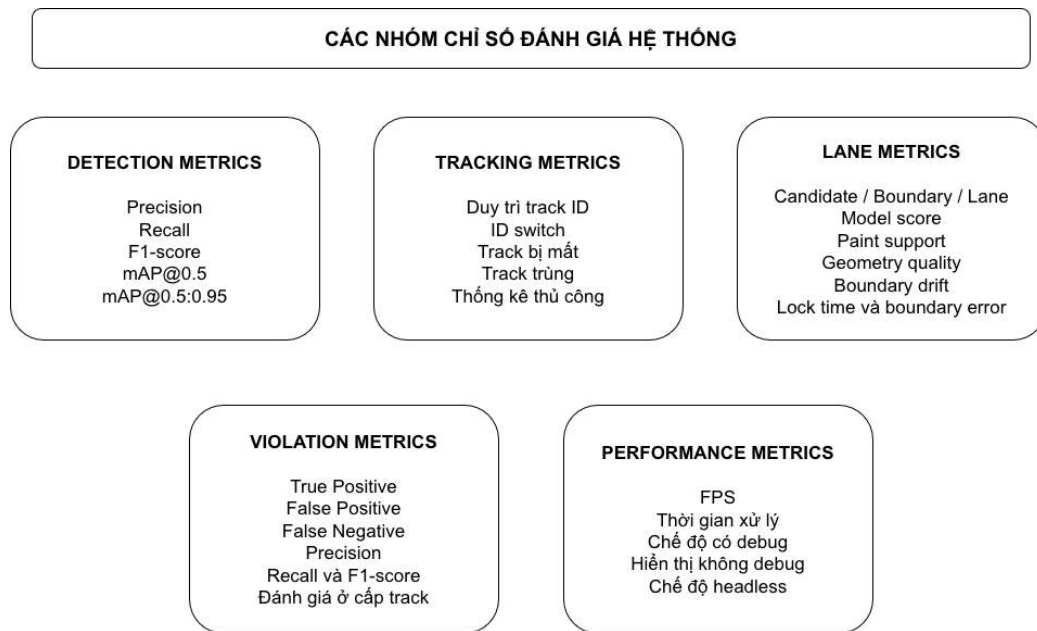
Hiệu năng hệ thống được đo theo các chế độ:

- **Debug:** tạo và hiển thị các ảnh trung gian;
- **Display:** chỉ hiển thị kết quả chính;
- **Headless:** không hiển thị cửa sổ và không tạo debug view;
- **Headless không ghi video:** chỉ thực hiện pipeline cốt lõi để đo hiệu năng thuật toán.

Các chỉ số được thiết kế để đánh giá hiệu năng gồm thời gian xử lý từng stage, thời gian toàn pipeline, FPS, P50, P95, mức sử dụng CPU, RAM và nhiệt độ thiết bị. Tuy nhiên, trong phạm vi thực nghiệm hiện tại, việc khảo sát trên Raspberry Pi 5 tập trung vào FPS và thời gian suy luận của riêng module phát hiện phương tiện ở các kích thước đầu vào và định dạng mô hình khác nhau.

Các giá trị thời gian toàn frame trong phép khảo sát module phát hiện được suy ra từ FPS. Benchmark đầy đủ cho toàn bộ pipeline, bao gồm

ByteTrack, lane detector, lane assignment, phân tích vi phạm, CPU, RAM, nhiệt độ và độ trễ P50/P95, được xác định là nội dung cần tiếp tục thực hiện trong giai đoạn phát triển tiếp theo.



Hình 3.4.3: Các nhóm chỉ số dùng để đánh giá detection, tracking, lane detection, nhận biết vi phạm và hiệu năng hệ thống.

3.5 Tổng kết chương

Chương 3 đã trình bày yêu cầu, kiến trúc tổng thể và phương pháp đề xuất của hệ thống nhận biết phương tiện đi sai làn và đi ngược chiều từ video giao thông. Hệ thống được tổ chức theo một pipeline thống nhất gồm thu nhận khung hình, phát hiện và theo dõi phương tiện, nhận diện làn đường, gán phương tiện vào làn, ước lượng hướng chuyển động và xác nhận vi phạm theo thời gian. Các module phần mềm được tách theo chức năng và trao đổi dữ liệu thông qua những cấu trúc đầu vào, đầu ra thống nhất, qua

đó hỗ trợ kiểm thử độc lập và thay đổi backend suy luận mà không làm thay đổi toàn bộ hệ thống.

Trọng tâm của phương pháp là quá trình xây dựng mô hình làn trong không gian Bird's-Eye View. Hệ thống kết hợp bằng chứng về màu sắc, độ sáng, tương phản và gradient để tạo ảnh nhị phân phục vụ tìm kiếm vạch làn. Các peak được phát hiện theo nhiều dải chiều sâu, liên kết thành candidate boundary và tinh chỉnh dựa trên pixel vạch sơn. Thay vì lựa chọn độc lập từng candidate, hệ thống đánh giá đồng thời các tổ hợp boundary dựa trên chất lượng hình học, thứ tự trái-phải, chiều rộng làn, crossing và các thành phần penalty. Các kết quả qua nhiều lần cập nhật tiếp tục được tổng hợp bằng temporal consensus để lựa chọn và khóa mô hình làn ổn định.

Thông tin làn chỉ được sử dụng cho quá trình phân tích vi phạm khi mô hình đạt trạng thái tin cậy cần thiết. Với mỗi Track ID, hệ thống duy trì lịch sử vị trí, lịch sử Lane ID và hướng chuyển động. Lane history và lane-change streak giúp hạn chế Lane ID dao động khi phương tiện nằm gần boundary, trong khi wrong-lane streak và wrong-way streak yêu cầu điều kiện vi phạm tồn tại qua nhiều khung hình trước khi xác nhận. Hướng dòng xe được tổng hợp từ nhiều track ổn định để hỗ trợ xác định quy ước chuyển động và thứ tự làn theo góc quan sát, còn luật về lớp phương tiện được phép và hướng hợp lệ của từng làn được cấu hình theo tổ chức giao thông tại vị trí camera.

Chương này cũng đã mô tả nguồn dữ liệu, quy trình trích xuất và gán nhãn, cấu hình huấn luyện mô hình phát hiện, các tham số chính của lane detector, môi trường triển khai và giao thức đánh giá. Các nhóm chỉ số được xác định cho từng thành phần gồm detection, tracking, lane detection, nhận

biết vi phạm và hiệu năng toàn pipeline.

Những nội dung trên là cơ sở để chương tiếp theo trình bày kết quả thực nghiệm và phân tích hiệu quả của hệ thống. Các kết quả được đánh giá không chỉ ở độ chính xác của từng module mà còn ở mức độ ổn định của Track ID, cấu trúc làn, khả năng xác nhận vi phạm và chi phí xử lý của toàn bộ pipeline.

CHƯƠNG 4

KẾT QUẢ VÀ THẢO LUẬN

Chương này trình bày kết quả thực nghiệm của hệ thống nhận biết phương tiện đi sai làn đường từ video giao thông của camera giám sát cố định. Việc đánh giá được thực hiện theo từng thành phần của pipeline, bao gồm phát hiện phương tiện, theo dõi đa đối tượng, nhận diện cấu trúc làn đường, gán phương tiện vào làn, phân tích trạng thái vi phạm và khảo sát hiệu năng triển khai.

Mô hình YOLO11n được đánh giá trên tập validation với bốn lớp phương tiện gồm xe máy, ô tô, xe tải và xe buýt. ByteTrack và lane detector được kiểm tra trên ba video giao thông đại diện cho các điều kiện quan sát khác nhau. Kết quả nhận biết vi phạm được phân tích thông qua các trường hợp quan sát trực tiếp trên video do dữ liệu hiện tại chưa có ground truth đầy đủ cho từng Track ID.

4.1 KẾT QUẢ HOẠT ĐỘNG CỦA HỆ THỐNG ĐỀ XUẤT

4.1.1 Kết quả huấn luyện mô hình YOLO11n

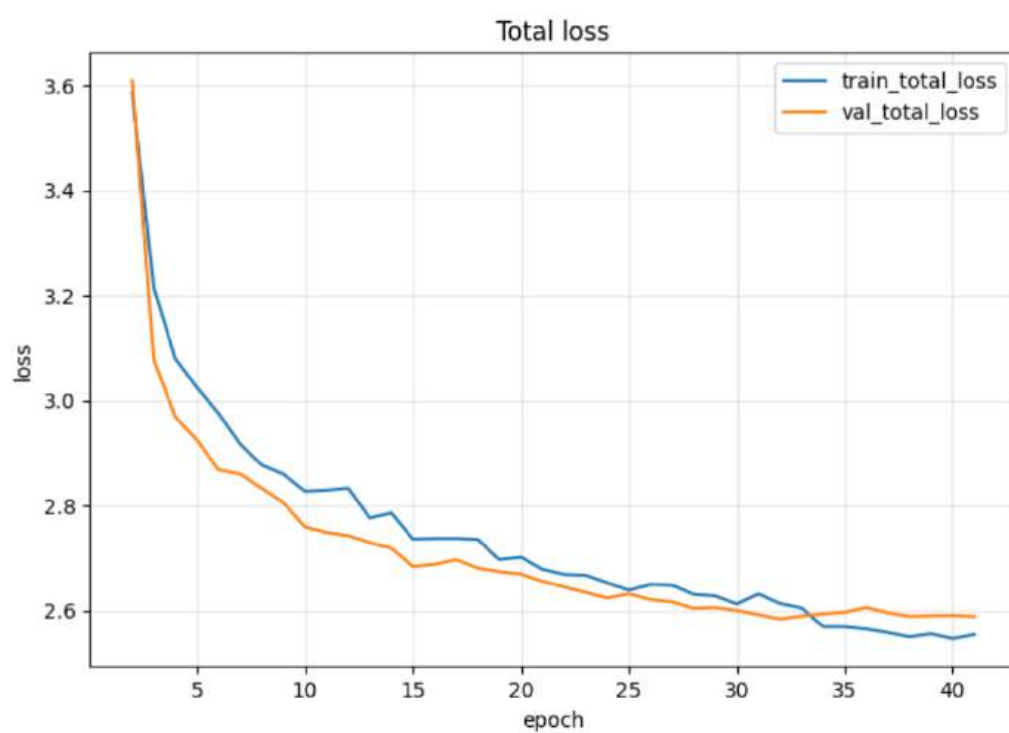
Mô hình YOLO11n được fine-tune trong 40 epoch với kích thước ảnh huấn luyện 640×640 pixel, batch size bằng 16 và bộ tối ưu AdamW. Quá trình huấn luyện được thực hiện trên GPU NVIDIA Tesla T4. Checkpoint `best.pt`, tương ứng với kết quả validation tốt nhất trong quá trình huấn luyện, được sử dụng cho các thí nghiệm tiếp theo.

Sau quá trình huấn luyện, các lớp convolution và batch normalization

được fusion nhằm giảm chi phí trong quá trình suy luận. Mô hình sau fusion gồm 101 lớp, có khoảng 2,58 triệu tham số và khối lượng tính toán được thống kê khoảng 6,3 GFLOPs tại kích thước đầu vào 640×640 pixel. Quy mô này cho thấy YOLO11n là một mô hình tương đối nhỏ gọn, phù hợp với định hướng triển khai trên thiết bị có tài nguyên giới hạn.

Bảng 4.1.1: Thông tin cấu trúc và cấu hình huấn luyện YOLO11n.

Thông số	Giá trị
Số lớp sau fusion	101
Số tham số	2.582.932
Khối lượng tính toán tại 640×640	6,3 GFLOPs
Kích thước ảnh huấn luyện	640×640 pixel
Số lớp phương tiện	4
Số epoch thực hiện	40
Batch size	16
Bộ tối ưu	AdamW
Phần cứng huấn luyện	NVIDIA Tesla T4



Hình 4.1.1: Sự thay đổi của các thành phần loss trong quá trình huấn luyện YOLO11n.

Quan sát Hình 4.1.1 cho thấy các thành phần loss trên tập huấn luyện và tập validation đều giảm nhanh trong những epoch đầu, sau đó giảm chậm dần và tiến tới trạng thái ổn định ở giai đoạn cuối. Tại epoch cuối, tổng train loss và validation loss lần lượt xấp xỉ 2,55 và 2,59. Chênh lệch giữa hai giá trị là nhỏ, cho thấy mô hình chưa xuất hiện dấu hiệu overfitting rõ rệt trong phạm vi 40 epoch.

4.1.2 Kết quả phát hiện phương tiện

Checkpoint best.pt được đánh giá trên tập validation gồm 167 ảnh với tổng cộng 5.933 đối tượng thuộc bốn lớp motorbike, car, truck và bus. Việc đánh giá được thực hiện với kích thước ảnh 640×640 pixel, tương ứng với kích thước được sử dụng trong quá trình huấn luyện.

F1-score được tính từ Precision và Recall theo công thức:

$$F1 = \frac{2PR}{P + R}, \quad (4.1.1)$$

trong đó P là Precision và R là Recall. Precision phản ánh tỷ lệ dự đoán đúng trên tổng số detection do mô hình tạo ra, trong khi Recall phản ánh tỷ lệ đối tượng thực tế được mô hình phát hiện.

Chỉ số mAP@0.5 biểu diễn mean Average Precision tại ngưỡng IoU bằng 0,5. Chỉ số mAP@0.5:0.95 được tính bằng cách lấy trung bình AP tại các ngưỡng IoU từ 0,5 đến 0,95 với bước tăng 0,05. Vì sử dụng nhiều ngưỡng IoU nghiêm ngặt hơn, mAP@0.5:0.95 phản ánh đồng thời khả năng phát hiện và độ chính xác định vị bounding box.

Kết quả trong Bảng 4.1.2 cho thấy mô hình đạt Precision trung bình bằng 0,780, Recall bằng 0,634, F1-score bằng 0,700, mAP@0.5 bằng 0,705

Bảng 4.1.2: Kết quả phát hiện phương tiện của YOLO11n trên tập validation.

Lớp	Số đối tượng	Precision	Recall	F1-score	mAP@0.5	mAP@0.5:0.95
Motorbike	4.372	0,821	0,741	0,779	0,820	0,507
Car	1.455	0,835	0,764	0,798	0,840	0,615
Truck	46	0,806	0,587	0,679	0,633	0,450
Bus	60	0,656	0,446	0,531	0,528	0,404
Trung bình bốn lớp	5.933	0,780	0,634	0,700	0,705	0,494

và mAP@0.5:0.95 bằng 0,494. Precision cao hơn Recall cho thấy mô hình tương đối hạn chế các dự đoán sai nhưng vẫn bỏ sót một bộ phận phương tiện trong tập validation.

Lớp car đạt kết quả cao nhất với Precision bằng 0,835, Recall bằng 0,764, F1-score bằng 0,798, mAP@0.5 bằng 0,840 và mAP@0.5:0.95 bằng 0,615. Mặc dù số lượng đối tượng car thấp hơn motorbike, ô tô thường chiếm diện tích lớn hơn trong ảnh và có hình dạng tương đối ổn định, giúp mô hình trích xuất đặc trưng và định vị bounding box thuận lợi hơn.

Lớp motorbike đạt Precision bằng 0,821, Recall bằng 0,741 và mAP@0.5 bằng 0,820. Đây là kết quả tương đối tốt trong bối cảnh xe máy thường có kích thước nhỏ, xuất hiện với mật độ cao và dễ bị che khuất một phần.

Tuy nhiên, mAP của motorbike giảm từ 0,820 tại ngưỡng IoU bằng 0,5 xuống 0,507 khi đánh giá trên dải IoU từ 0,5 đến 0,95. Chênh lệch này cho thấy mô hình có thể phát hiện được nhiều xe máy ở mức chồng lấp vừa phải nhưng độ chính xác định vị bounding box giảm tại các ngưỡng IoU nghiêm ngặt.

Hai lớp truck và bus có kết quả thấp hơn. Lớp truck đạt Recall bằng 0,587 và mAP@0.5 bằng 0,633, trong khi lớp bus chỉ đạt Recall bằng 0,446 và mAP@0.5 bằng 0,528. Một nguyên nhân quan trọng là số lượng mẫu

validation của hai lớp còn thấp, lần lượt chỉ có 46 đối tượng truck và 60 đối tượng bus.

Do kích thước tập mẫu nhỏ, các chỉ số của truck và bus có thể biến động mạnh và chưa phản ánh đầy đủ khả năng tổng quát hóa của mô hình. Ngoài ra, hai lớp này có thể xuất hiện với kích thước nhỏ khi ở xa camera, bị che khuất hoặc có đặc điểm hình dạng tương tự ở một số góc quan sát.

Kết quả trên cho thấy cần bổ sung thêm dữ liệu truck và bus với nhiều khoảng cách, góc nhìn và điều kiện chiếu sáng khác nhau. Bên cạnh đó, việc đánh giá cuối cùng nên được thực hiện trên tập test độc lập để hạn chế ảnh hưởng của việc lựa chọn checkpoint dựa trên validation.

4.1.3 Kết quả theo dõi đa đối tượng

ByteTrack được sử dụng để liên kết các detection qua nhiều frame và duy trì một Track ID cho mỗi phương tiện. Do tập dữ liệu hiện tại chưa có ground truth theo định dạng Multi-Object Tracking, việc đánh giá tracking được thực hiện thủ công trên ba video thử nghiệm.

Mỗi phương tiện được quan sát từ thời điểm đi vào vùng ROI đến khi rời khỏi vùng này. Số phương tiện thực tế được xác định bằng cách đếm thủ công các phương tiện duy nhất xuất hiện trong ROI. Số Track ID tạo ra là tổng số ID khác nhau được ByteTrack sinh ra trong toàn bộ video.

Một ID switch được ghi nhận khi cùng một phương tiện bị chuyển từ Track ID đang sử dụng sang một Track ID khác trong khi vẫn còn xuất hiện trong vùng quan sát. Track bị mất là trường hợp hệ thống không duy trì được phương tiện cho đến khi phương tiện rời ROI.

Trong phép đánh giá này, mỗi phương tiện chỉ được xếp vào một

nhóm lỗi chính nhằm tránh trường hợp cùng một phương tiện bị trừ đồng thời ở cả nhóm ID switch và nhóm track bị mất.

Tỷ lệ duy trì ID đúng được xác định theo công thức:

$$R_{ID} = \frac{N_{\text{vehicle}} - N_{\text{IDSwitch}} - N_{\text{lost}}}{N_{\text{vehicle}}} \times 100\%, \quad (4.1.2)$$

trong đó N_{vehicle} là số phương tiện thực tế, N_{IDSwitch} là số phương tiện xảy ra ID switch và N_{lost} là số phương tiện bị mất track.

Đây là chỉ số đánh giá thủ công được sử dụng trong phạm vi đề tài và không thay thế các metric MOT chuẩn như IDF1, MOTA hoặc HOTA.

Bảng 4.1.3: Kết quả đánh giá thủ công khả năng duy trì Track ID.

Video	Số xe thực tế	Số Track ID	ID switch	Track bị mất	Duy trì ID đúng (%)
V1	100	103	3	2	95,0
V2	148	156	7	5	91,9
V3	82	86	4	3	91,5
Tổng hợp	330	345	14	10	92,7

Kết quả trong Bảng 4.1.3 cho thấy hệ thống duy trì đúng Track ID cho khoảng 92,7% số phương tiện được quan sát. V1 đạt tỷ lệ cao nhất, bằng 95,0%, do điều kiện ban ngày, đường thẳng và mức độ che khuất thấp hơn.

Hai video V2 và V3 đạt tỷ lệ thấp hơn, lần lượt là 91,9% và 91,5%. V2 có mật độ phương tiện cao, trong khi V3 chịu ảnh hưởng của ánh sáng yếu và phản xạ mặt đường. Các điều kiện này làm giảm độ ổn định của detection và tăng khả năng mất Track ID.

Số Track ID được tạo ra lớn hơn số phương tiện thực tế trên cả ba

video. Chênh lệch này cho thấy một số quỹ đạo bị phân mảnh, nghĩa là cùng một phương tiện có thể được khởi tạo lại bằng Track ID mới sau khi detector hoặc tracker mất đối tượng trong một khoảng thời gian.

Các trường hợp ID switch chủ yếu xuất hiện khi nhiều xe máy di chuyển sát nhau, phương tiện bị che khuất hoàn toàn hoặc detector bỏ sót đối tượng trong nhiều frame liên tiếp. Khi Track ID thay đổi, lịch sử vị trí, lịch sử Lane ID, hướng chuyển động và bộ đếm vi phạm có thể bị khởi tạo lại.

4.1.4 Kết quả nhận diện làn đường

Lane detector được đánh giá theo các giai đoạn gồm biến đổi phối cảnh, tiền xử lý ảnh, phát hiện peak, liên kết candidate, hiệu chỉnh đa thức, lựa chọn mô hình và ổn định boundary theo thời gian.

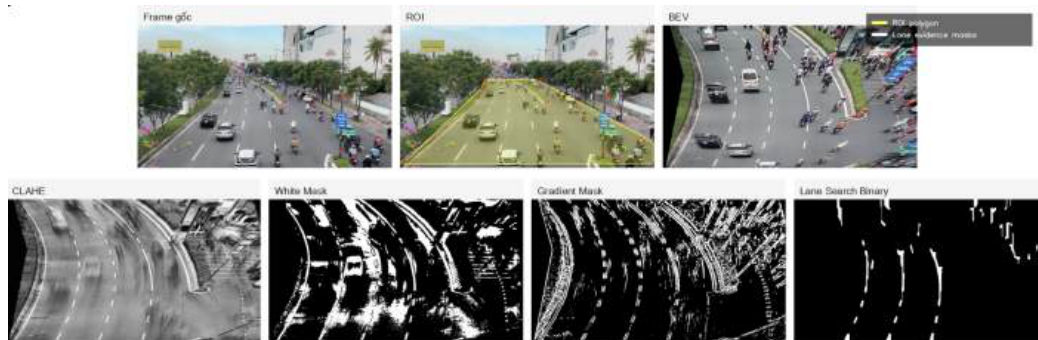
Việc quan sát các kết quả trung gian giúp xác định chính xác giai đoạn gây lỗi khi mô hình cuối cùng bị thiếu làn, xuất hiện lane ảo hoặc boundary bị lệch.

4.1.4.1 Kết quả tiền xử lý

Ảnh trong ROI được biến đổi sang Bird's-Eye View nhằm giảm ảnh hưởng của phối cảnh và làm cho khoảng cách giữa các boundary ổn định hơn theo chiều sâu.

Trên ảnh BEV, CLAHE được sử dụng để tăng tương phản cục bộ. White Mask làm nổi bật các pixel có đặc trưng màu sắc và độ sáng gần với vạch sơn, trong khi Gradient Mask phát hiện các vùng có biến thiên cường độ phù hợp với cấu trúc biên của vạch làn.

Các nguồn đặc trưng được kết hợp và tiếp tục xử lý bằng morphology cùng connected-component filtering để tạo Lane Search Binary. Những vùng pixel trắng quá nhỏ hoặc có hình dạng không phù hợp được loại bỏ nhằm giảm nhiễu từ mặt đường, bóng đổ và phương tiện.



Hình 4.1.2: Các bước tiền xử lý ảnh phục vụ nhận diện làn đường.

Kết quả cho thấy CLAHE giúp làm rõ các vạch làn có độ tương phản thấp. Tuy nhiên, kỹ thuật này cũng có thể làm nổi bật texture mặt đường, bóng đổ và các vết nứt. Vì vậy, ảnh sau CLAHE không được sử dụng độc lập mà được kết hợp với các điều kiện màu sắc, gradient và hình thái học.

Trong video ban đêm, ánh sáng đèn xe và phản xạ mặt đường có thể tạo ra các vùng sáng mạnh hơn vạch sơn. Các vùng này làm xuất hiện peak giả trong histogram và có nguy cơ được liên kết thành candidate không hợp lệ.

4.1.4.2 Kết quả trích xuất candidate và hiệu chỉnh đa thức

Hệ thống chia ảnh BEV thành nhiều dải theo chiều sâu và tính histogram độc lập tại từng dải. Cấu hình hiện tại sử dụng 14 dải trong chế độ Day hoặc Auto và 18 dải trong chế độ Night.

Tại mỗi dải, những vị trí có mật độ pixel vạch cao được xác định dưới

dạng peak. Các peak có tính liên tục về vị trí được liên kết từ dưới lên trên để hình thành candidate boundary. Phương pháp này giúp duy trì candidate khi một số đoạn vạch bị phương tiện che khuất, bị đứt nét hoặc có độ tương phản thấp.

Tập điểm của mỗi candidate được mô hình hóa bằng đa thức bậc hai:

$$x(y) = ay^2 + by + c, \quad (4.1.3)$$

trong đó y là tọa độ chiều sâu trong ảnh BEV, $x(y)$ là vị trí ngang của boundary, còn a , b và c là các hệ số của đa thức.

Candidate refinement thiết lập một hành lang tìm kiếm quanh đường cong ban đầu và thu thập lại các pixel có khả năng thuộc vạch sơn. Candidate sau đó được fit lại bằng tập điểm đã hiệu chỉnh.

Sai số boundary được tính bằng khoảng cách ngang trung bình giữa boundary dự đoán và boundary tham chiếu trong ảnh BEV. Sai số chỉ được tính trên các boundary dự đoán có thể đối sánh với boundary tham chiếu. Boundary bị bỏ sót được phản ánh riêng thông qua số lượng boundary phát hiện được và không được đưa vào giá trị sai số trung bình.

Mức cải thiện sau refinement được tính theo công thức:

$$I = \frac{E_{\text{before}} - E_{\text{after}}}{E_{\text{before}}} \times 100\%, \quad (4.1.4)$$

trong đó E_{before} và E_{after} lần lượt là sai số boundary trước và sau refinement.

Kết quả cho thấy refinement làm giảm sai số boundary trên cả ba video. V1 giảm từ 14,93 pixel xuống 4,28 pixel, tương ứng mức cải thiện

Bảng 4.1.4: Ảnh hưởng của candidate refinement đến sai số boundary.

Video	Trước refinement (px)	Sau refinement (px)	Cải thiện (%)
V1	14,93	4,28	71,4
V2	8,90	2,95	66,8
V3	20,84	4,22	79,8

71,4%. V2 giảm từ 8,90 pixel xuống 2,95 pixel, tương ứng mức cải thiện 66,8%. V3 giảm từ 20,84 pixel xuống 4,22 pixel, tương ứng mức cải thiện 79,8%.

Mặc dù V3 có mức cải thiện tương đối lớn, kết quả này chỉ phản ánh các boundary được phát hiện và đối sánh thành công. Một boundary ngoài của V3 vẫn bị bỏ sót, vì vậy kết quả cấu trúc làn tổng thể của video này chưa đạt yêu cầu.

4.1.4.3 Kết quả lựa chọn và khóa mô hình làn

Model selector đánh giá các tổ hợp candidate dựa trên mức hỗ trợ của pixel vạch, chất lượng hình học, chiều rộng làn và các thành phần phạt. Tập boundary có tổng điểm cao nhất và thỏa các điều kiện topology được lựa chọn làm mô hình làn của lần cập nhật hiện tại.

Đối với V1 và V2, hệ thống phát hiện trực tiếp bốn boundary nội bộ và suy luận thêm hai boundary ngoài. Tổng cộng sáu boundary cuối cùng được sử dụng để tạo ra năm vùng làn, phù hợp với cấu trúc tham chiếu. Cơ chế suy luận boundary ngoài giúp hệ thống vẫn tạo được đầy đủ vùng làn trong trường hợp mép đường không có vạch sơn rõ ràng hoặc nằm sát giới hạn ROI. Các boundary suy luận không được xem là detection trực tiếp từ ảnh mà được xác định dựa trên khoảng cách làn trung bình và hình học của

Bảng 4.1.5: Kết quả xác định cấu trúc làn đường.

Video	Boundary tham chiếu	Phát hiện trực tiếp	Boundary suy luận	Boundary cuối cùng	Số làn cuối cùng	Nhận xét
V1	6	4	2	6	5	Đúng cấu trúc làn
V2	6	4	2	6	5	Đúng cấu trúc làn
V3	6	3	2	5	4	Thiếu một boundary nội bộ

Boundary tham chiếu được xác định thủ công. Hai boundary ngoài được hệ thống suy luận dựa trên vị trí boundary nội bộ, chiều rộng làn ước lượng và giới hạn vùng mặt đường.

các boundary đã phát hiện. Đối với V3, hệ thống chỉ phát hiện trực tiếp ba trong bốn boundary nội bộ. Sau khi bổ sung hai boundary ngoài, mô hình cuối cùng có năm boundary và chỉ tạo được bốn vùng làn. Boundary nội bộ bị thiếu có mức hỗ trợ pixel thấp do ánh sáng yếu, phản xạ mặt đường và vùng sáng từ đèn phương tiện trong điều kiện ban đêm. Kết quả cho thấy phương pháp có khả năng khôi phục tương đối tốt cấu trúc làn khi các boundary nội bộ chính được phát hiện ổn định. Tuy nhiên, cơ chế suy luận boundary ngoài không thể thay thế một boundary nội bộ bị mất, vì boundary nội bộ quyết định trực tiếp việc phân chia hai vùng làn liền kề.

Trong giai đoạn calibration, các mô hình ứng viên được tích lũy theo thời gian. Khi một topology xuất hiện đủ thường xuyên, boundary drift nhỏ và model score đạt ngưỡng yêu cầu, hệ thống tạo consensus model và chuyển sang trạng thái khóa.

Bảng 4.1.6: Kết quả ổn định và khóa mô hình làn.

Video	Frame khóa	Drift	Score	Kết quả
V1	153	0,88	0,98	Khóa đúng
V2	120	0,00	0,81	Khóa đúng
V3	Không khóa	–	0,61	Không đạt ngưỡng

Drift là mức thay đổi vị trí boundary giữa các lần cập nhật; Score là điểm tin cậy nội bộ của mô hình làn.

V1 và V2 đạt được mô hình consensus ổn định và chuyển sang trạng thái khóa. Giá trị drift bằng 0,00 của V2 cho thấy tại mức làm tròn được sử dụng, các boundary của consensus model gần như không thay đổi giữa các lần cập nhật được đánh giá.

V3 không đạt trạng thái khóa do mô hình cuối cùng chỉ có năm trong

sáu boundary cần thiết. Model score đạt 0,61, thấp hơn mức tin cậy của hai video ban ngày. Kết quả này cho thấy cơ chế temporal consensus không khóa một cấu trúc làn chưa đầy đủ, qua đó hạn chế việc sử dụng một mô hình sai cho quá trình gán làn và phân tích vi phạm.

Temporal consensus giúp hạn chế số lượng và vị trí boundary thay đổi liên tục giữa các frame. Sau khi khóa, hệ thống tái sử dụng mô hình làn đã ổn định và chỉ cập nhật định kỳ hoặc khi độ tin cậy suy giảm.

4.1.5 Kết quả gán phương tiện vào làn

Khi mô hình làn đã sẵn sàng, điểm bottom-center của bounding box được biến đổi sang không gian BEV và so sánh với các boundary tại cùng tọa độ chiều sâu. Lane history và lane-change streak được sử dụng để hạn chế Lane ID dao động khi phương tiện nằm gần đường biên.

Kết quả gán làn được đánh giá trên các mẫu quan sát được lựa chọn thủ công. Mỗi mẫu biểu diễn một trường hợp Lane ID đã đạt trạng thái ổn định trong một đoạn quan sát. Vì vậy, số mẫu đánh giá trong bảng không được xem là số Track ID duy nhất của toàn bộ video.

Bảng 4.1.7: Kết quả gán phương tiện vào làn.

Video	Số mẫu đánh giá	Gán đúng	Gán sai	Không xác định	Độ chính xác
V1	150	137	13	0	91,33%
V2	100	98	2	0	98,00%
V3	91	–	–	91	Không đánh giá

V1 đạt độ chính xác gán làn bằng 91,33%, trong khi V2 đạt 98,00%. Kết quả cho thấy khi lane model xác định đúng cấu trúc và đạt trạng thái khóa, điểm bottom-center kết hợp với lane history có thể gán làn tương đối

ổn định.

V3 không được đánh giá độ chính xác vì lane detector không tạo được đầy đủ cấu trúc năm làn. Trong trường hợp này, Lane ID dự đoán không tương ứng hoàn toàn với Lane ID tham chiếu, do đó các mẫu được ghi nhận là không xác định.

Các trường hợp gán sai chủ yếu xuất hiện khi boundary bị lệch, bounding box không bao phủ chính xác vùng tiếp xúc với mặt đường hoặc phương tiện đang thực hiện chuyển làn. Lane history giúp giảm dao động tức thời nhưng không thể khắc phục hoàn toàn lỗi hình học của mô hình làn.

4.1.6 Kết quả phát hiện vi phạm

Vi phạm sai làn được xác định bằng cách so sánh lớp phương tiện với tập lớp được phép lưu thông trong làn hiện tại. Vi phạm đi ngược chiều được xác định dựa trên lịch sử quỹ đạo và hướng lưu thông hợp lệ của làn.

Một vi phạm chỉ được xác nhận khi trạng thái tương ứng được duy trì đủ số frame yêu cầu. Cơ chế xác nhận qua nhiều frame giúp hạn chế cảnh báo sai do Lane ID dao động hoặc detector nhận nhầm lớp trong một frame đơn lẻ.

False Positive có thể xuất hiện khi phương tiện đang chuyển làn, boundary bị lệch hoặc nhận lớp thay đổi tạm thời. False Negative có thể xảy ra khi track bị gián đoạn, phương tiện chỉ xuất hiện trong vùng vi phạm trong thời gian ngắn hoặc detection bị bỏ sót liên tục.

Trong phạm vi dữ liệu hiện tại, kết quả phát hiện vi phạm được đánh giá định tính thông qua các trường hợp quan sát trực tiếp trên video. Hình 4.1.3 minh họa một trường hợp hệ thống gán Lane ID và cảnh báo phương

tiện đi sai làn.



Hình 4.1.3: Ví dụ hệ thống phát hiện và cảnh báo phương tiện đi sai làn dựa trên lớp phương tiện và Lane ID.

Kết quả minh họa cho thấy hệ thống đã triển khai được luồng xử lý từ detection, tracking, gán làn đến tạo cảnh báo vi phạm. Tuy nhiên, do chưa có ground truth vi phạm ở cấp độ track, chương này chưa thể kết luận định lượng về Precision, Recall hoặc F1-score của kết quả phát hiện vi phạm cuối cùng.

4.2 ĐÁNH GIÁ HIỆU NĂNG VÀ KHẢ NĂNG TRIỂN KHAI

4.2.1 Hiệu năng của mô hình phát hiện trên GPU

Kết quả validation trên GPU NVIDIA Tesla T4 cho thấy thời gian tiền xử lý trung bình khoảng 1,0 ms, thời gian suy luận khoảng 4,6 ms và thời gian hậu xử lý khoảng 16,7 ms trên mỗi ảnh.

Các giá trị này chỉ phản ánh module phát hiện phương tiện và không đại diện cho thời gian xử lý của toàn bộ pipeline.

Bảng 4.2.1: Thời gian xử lý của YOLO11n trên GPU NVIDIA Tesla T4.

Giai đoạn	Thời gian trung bình
Tiền xử lý	1,0 ms/ảnh
Suy luận	4,6 ms/ảnh
Hậu xử lý	16,7 ms/ảnh

Mô hình PyTorch có kích thước khoảng 5,2 MB. Phiên bản ONNX và OpenVINO sau khi export có kích thước lần lượt khoảng 10,1 MB và 10,2 MB. Sự khác biệt về dung lượng không phản ánh trực tiếp tốc độ suy luận vì mỗi backend sử dụng cách biểu diễn graph và trọng số khác nhau.

Bảng 4.2.2: Kích thước các định dạng mô hình sau khi export.

Định dạng	Kích thước
PyTorch best.pt	5,2 MB
ONNX best.onnx	10,1 MB
OpenVINO IR	10,2 MB

4.2.2 So sánh hiệu năng trên máy tính phát triển và Raspberry Pi 5

Hiệu năng của module phát hiện phương tiện được khảo sát trên hai môi trường gồm máy tính phát triển sử dụng GPU NVIDIA Tesla T4 và thiết bị nhúng Raspberry Pi 5. Việc so sánh nhằm đánh giá mức độ thay đổi về tốc độ xử lý khi chuyển mô hình từ môi trường có tài nguyên tính toán cao sang nền tảng nhúng có tài nguyên hạn chế.

Trên môi trường GPU NVIDIA Tesla T4, thời gian tiền xử lý, suy luận và hậu xử lý trung bình của YOLO11n lần lượt là 1,0 ms, 4,6 ms và 16,7 ms trên mỗi ảnh. Tổng thời gian xử lý của riêng module phát hiện xấp xỉ:

$$T_{\text{detector}} = T_{\text{preprocess}} + T_{\text{inference}} + T_{\text{postprocess}} = 22,3 \text{ ms/ảnh}. \quad (4.2.1)$$

Từ tổng thời gian trên, thông lượng lý thuyết của module phát hiện được xác định theo công thức:

$$FPS_{\text{detector}} = \frac{1000}{T_{\text{detector}}}, \quad (4.2.2)$$

tương ứng khoảng 44,8 FPS trên GPU NVIDIA Tesla T4.

Tài nguyên của môi trường máy tính phát triển cho phép chạy đồng thời các thành phần gồm YOLO11n, ByteTrack, lane detector, lane assignment và phân tích vi phạm. Trong các video ban ngày, hệ thống duy trì được luồng xử lý ổn định, Track ID được cập nhật liên tục và mô hình làn có thể đạt trạng thái khóa.

Khi chuyển sang Raspberry Pi 5, module phát hiện vẫn có khả năng thực thi nhưng tốc độ thấp hơn do giới hạn về năng lực xử lý, băng thông bộ nhớ và tài nguyên phần cứng. Để khảo sát ảnh hưởng của kích thước đầu vào và lượng tử hóa, YOLO11n được kiểm tra tại hai kích thước 224×224 và 640×640 pixel, với phiên bản thông thường và phiên bản OpenVINO INT8.

Bảng 4.2.3: So sánh hiệu năng module phát hiện trên máy tính phát triển và Raspberry Pi 5.

Nền tảng	Định dạng	Kích thước	FPS	Suy luận (ms/ảnh)	Tổng thời gian (ms/ảnh)
NVIDIA Tesla T4	PyTorch	640×640	44,8*	4,60	22,30
Raspberry Pi 5	YOLO11n	224×224	14,85	60,69	67,34
Raspberry Pi 5	OpenVINO INT8	224×224	30,17	23,60	33,15
Raspberry Pi 5	YOLO11n	640×640	3,79	250,42	263,85
Raspberry Pi 5	OpenVINO INT8	640×640	8,68	98,57	115,21

*Giá trị FPS trên NVIDIA Tesla T4 được suy ra từ tổng thời gian tiền xử lý, suy luận và hậu xử lý trung bình của module phát hiện.

Đối với Raspberry Pi 5, thời gian trung bình của toàn bộ một lần xử lý detector được suy ra từ FPS theo công thức:

$$T_{\text{frame}} = \frac{1000}{\text{FPS}}, \quad (4.2.3)$$

trong đó T_{frame} có đơn vị là mili giây trên mỗi frame.

Ở kích thước 224×224 pixel, phiên bản YOLO11n thông thường đạt 14,85 FPS với thời gian suy luận 60,69 ms/ảnh. Khi sử dụng OpenVINO INT8, tốc độ tăng lên 30,17 FPS và thời gian suy luận giảm xuống 23,60 ms/ảnh. Hệ số tăng tốc được tính như sau:

$$S_{224} = \frac{30,17}{14,85} \approx 2,03. \quad (4.2.4)$$

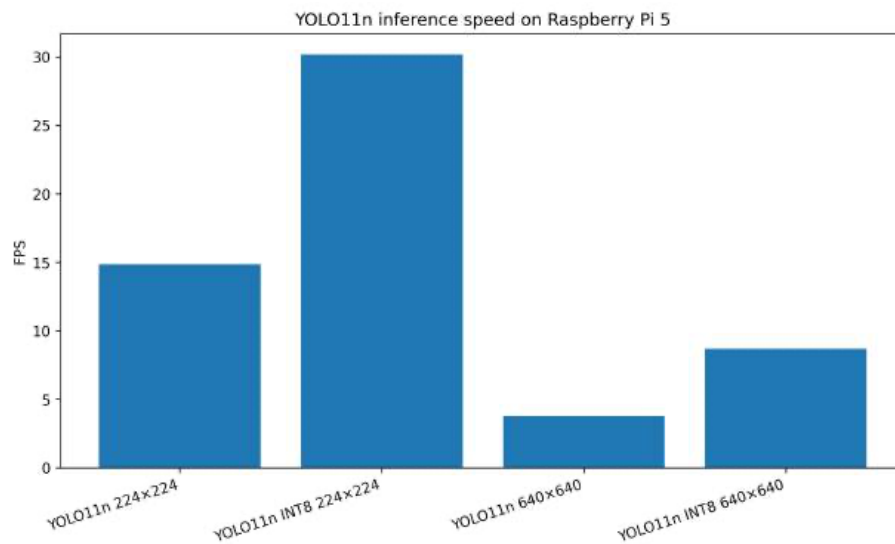
Như vậy, tại kích thước 224×224 , lượng tử hóa INT8 giúp FPS tăng khoảng 103,2%, thời gian suy luận giảm khoảng 61,1% và thời gian toàn frame giảm từ 67,34 ms xuống 33,15 ms, tương ứng mức giảm khoảng 50,8%.

Ở kích thước 640×640 pixel, phiên bản thông thường đạt 3,79 FPS với thời gian suy luận 250,42 ms/ảnh. Phiên bản OpenVINO INT8 đạt 8,68

FPS với thời gian suy luận 98,57 ms/ảnh. Hệ số tăng tốc được xác định theo công thức:

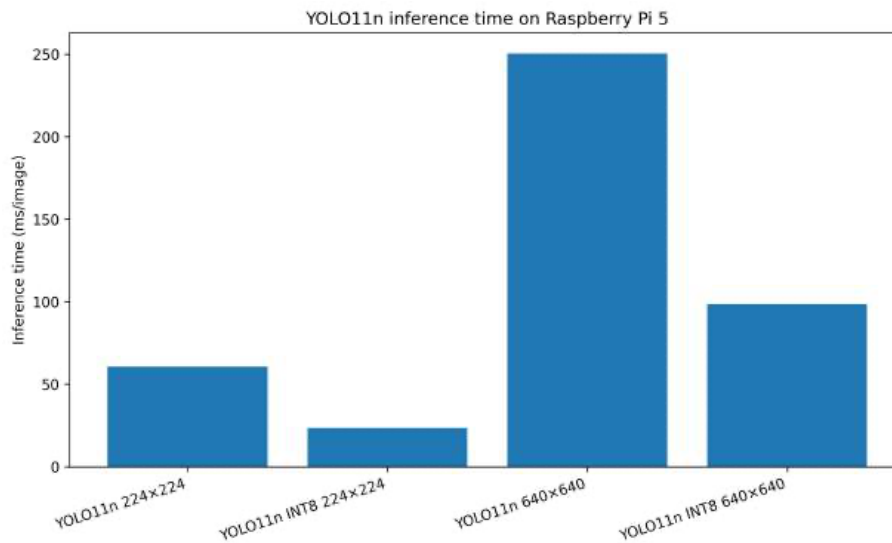
$$S_{640} = \frac{8,68}{3,79} \approx 2,29. \quad (4.2.5)$$

Tại kích thước này, FPS tăng khoảng 129,0%, thời gian suy luận giảm khoảng 60,6% và thời gian toàn frame giảm từ 263,85 ms xuống 115,21 ms, tương ứng mức giảm khoảng 56,3%.



Hình 4.2.1: So sánh tốc độ xử lý của các cấu hình YOLO11n trên Raspberry Pi 5.

Hình 4.2.1 cho thấy phiên bản OpenVINO INT8 đạt tốc độ cao hơn rõ rệt so với phiên bản thông thường ở cả hai kích thước đầu vào. Cấu hình OpenVINO INT8 224×224 đạt tốc độ cao nhất, bằng 30,17 FPS. Cấu hình này phù hợp khi hệ thống ưu tiên khả năng xử lý liên tục và chi phí tính toán thấp trên thiết bị nhúng.



Hình 4.2.2: So sánh thời gian suy luận của các cấu hình YOLO11n trên Raspberry Pi 5.

Hình 4.2.2 thể hiện xu hướng ngược với biểu đồ FPS. Cấu hình có FPS cao hơn có thời gian suy luận thấp hơn. Phiên bản OpenVINO INT8 224×224 có thời gian suy luận thấp nhất, bằng 23,60 ms/ảnh, trong khi phiên bản YOLO11n thông thường 640×640 có thời gian suy luận cao nhất, bằng 250,42 ms/ảnh.

Tại cùng kích thước đầu vào 640×640 pixel, module phát hiện trên GPU NVIDIA Tesla T4 có thông lượng lý thuyết khoảng 44,8 FPS. Phiên bản YOLO11n thông thường trên Raspberry Pi 5 đạt 3,79 FPS, trong khi phiên bản OpenVINO INT8 đạt 8,68 FPS. Như vậy, GPU Tesla T4 có tốc độ cao hơn khoảng 11,8 lần so với phiên bản thông thường trên Raspberry Pi 5 và cao hơn khoảng 5,2 lần so với phiên bản OpenVINO INT8 trên Raspberry Pi 5.

Sự chênh lệch trên phản ánh khác biệt lớn về năng lực xử lý giữa GPU

máy chủ và nền tảng nhúng. Tuy nhiên, kết quả cũng cho thấy Raspberry Pi 5 vẫn có khả năng thực thi mô hình. Ở kích thước 640×640 , phiên bản OpenVINO INT8 duy trì tốc độ gần 9 FPS. Khi giảm kích thước xuống 224×224 , tốc độ tăng lên 30,17 FPS.

Cấu hình OpenVINO INT8 224×224 phù hợp với trường hợp ưu tiên tốc độ xử lý. Ngược lại, cấu hình OpenVINO INT8 640×640 giữ lại nhiều thông tin ảnh hơn và phù hợp hơn với các phương tiện nhỏ hoặc nằm xa camera, nhưng có chi phí tính toán cao hơn.

Kết quả cho thấy hệ thống có hai mức vận hành. Trên môi trường máy tính phát triển, tài nguyên phần cứng cho phép chạy đồng thời các thành phần của pipeline với độ ổn định cao. Trên Raspberry Pi 5, module phát hiện vẫn có khả năng triển khai, nhưng cần lựa chọn kích thước đầu vào và định dạng mô hình phù hợp để cân bằng giữa tốc độ xử lý và lượng thông tin được giữ lại.

Các giá trị FPS trong phần này chỉ phản ánh module phát hiện phương tiện. Khi tích hợp thêm ByteTrack, lane detector, lane assignment, thao tác hiển thị và logic xác nhận vi phạm, tốc độ xử lý của toàn bộ pipeline trên Raspberry Pi 5 sẽ thấp hơn kết quả của module YOLO11n riêng biệt.

4.3 THẢO LUẬN

Kết quả thực nghiệm cho thấy hệ thống đã xây dựng và tích hợp được một pipeline xử lý thống nhất, bắt đầu từ phát hiện phương tiện, duy trì Track ID, nhận diện cấu trúc làn đường, gán phương tiện vào làn, xác định hướng chuyển động đến tạo cảnh báo vi phạm. Việc đánh giá theo từng module cho phép xác định rõ khả năng hoạt động, mức độ ổn định và những hạn

chế của hệ thống trong từng điều kiện thử nghiệm.

Đối với module phát hiện phương tiện, YOLO11n đạt Precision trung bình bằng 0,780, Recall bằng 0,634, F1-score bằng 0,700, mAP@0.5 bằng 0,705 và mAP@0.5:0.95 bằng 0,494 trên tập validation. Kết quả này cho thấy mô hình đạt chất lượng phát hiện ở mức khá và có thể cung cấp đầu vào tương đối ổn định cho các module theo dõi và phân tích giao thông phía sau.

Precision cao hơn Recall cho thấy mô hình kiểm soát tương đối tốt số detection sai, nhưng vẫn còn bỏ sót một số phương tiện. Các trường hợp bỏ sót thường liên quan đến phương tiện có kích thước nhỏ, nằm xa camera, bị che khuất hoặc xuất hiện trong khu vực có mật độ giao thông cao. Do detection là đầu vào của toàn bộ pipeline, các trường hợp bỏ sót này có thể làm track bị gián đoạn hoặc khiến phương tiện không được đưa vào quá trình phân tích làn và vi phạm.

Hai lớp car và motorbike đạt kết quả tốt hơn truck và bus. Lớp car đạt mAP@0.5 bằng 0,840, trong khi lớp motorbike đạt 0,820. Đây cũng là hai lớp xuất hiện phổ biến trong dữ liệu giao thông được sử dụng. Ngược lại, số lượng mẫu truck và bus còn thấp nên các chỉ số của hai lớp này chưa ổn định và có thể thay đổi đáng kể khi dữ liệu đánh giá được mở rộng.

Các kết quả detection hiện được báo cáo trên tập validation, đồng thời đây cũng là tập được sử dụng để lựa chọn checkpoint best.pt. Vì vậy, các chỉ số trên phản ánh chất lượng của mô hình trong phạm vi dữ liệu hiện có nhưng chưa thay thế cho kết quả đánh giá trên một tập test độc lập. Việc bổ sung tập test tách biệt theo nguồn video sẽ giúp đánh giá khách quan hơn khả năng tổng quát hóa của mô hình.

Đối với module theo dõi, ByteTrack duy trì đúng Track ID cho khoảng 92,7% số phương tiện được quan sát thủ công. Kết quả này cho thấy tracker có thể duy trì phần lớn quỹ đạo đủ lâu để hệ thống tích lũy lịch sử vị trí, Lane ID, hướng chuyển động và số frame vi phạm.

Các lỗi tracking chủ yếu xuất hiện khi nhiều phương tiện di chuyển sát nhau, phương tiện bị che khuất hoàn toàn hoặc detector không tạo detection trong nhiều frame liên tiếp. Khi đó, quỹ đạo có thể bị phân mảnh và cùng một phương tiện được khởi tạo lại bằng Track ID mới. Điều này ảnh hưởng trực tiếp đến quá trình bỏ phiếu Lane ID, xác định hướng chuyển động và xác nhận vi phạm qua nhiều frame.

Đối với lane detector, candidate refinement làm giảm sai số boundary từ 66,8% đến 79,8% trên ba video thử nghiệm. Kết quả này cho thấy việc thu thập lại pixel trong hành lang tìm kiếm quanh candidate ban đầu giúp đường cong sau refinement bám sát vạch làn tốt hơn.

Trên hai video ban ngày V1 và V2, hệ thống phát hiện trực tiếp bốn boundary nội bộ và suy luận thêm hai boundary ngoài. Sáu boundary cuối cùng tạo thành năm vùng làn, phù hợp với cấu trúc tham chiếu. Mô hình làn đạt trạng thái khóa với score lần lượt là 0,98 và 0,81, cho thấy cấu trúc boundary được duy trì tương đối ổn định theo thời gian.

Cơ chế suy luận hai boundary ngoài là cần thiết do mép ngoài của mặt đường không phải lúc nào cũng có vạch sơn rõ ràng hoặc có thể nằm sát biên ROI. Các boundary này được suy ra dựa trên vị trí của các boundary nội bộ, chiều rộng làn ước lượng và giới hạn vùng mặt đường. Nhờ đó, hệ thống vẫn có thể hoàn thiện cấu trúc làn khi các vạch phân chia bên trong được phát hiện đầy đủ.

Đối với V3, hệ thống chỉ phát hiện trực tiếp ba trong bốn boundary nội bộ. Mặc dù hai boundary ngoài vẫn được suy luận, mô hình cuối cùng chỉ có năm boundary và tạo được bốn trong năm vùng làn tham chiếu. Video này không đạt trạng thái khóa do cấu trúc làn chưa đầy đủ và model score chỉ đạt 0,61.

Kết quả trên cho thấy cơ chế suy luận boundary ngoài có thể bổ sung mép đường bị thiếu nhưng không thể thay thế một boundary nội bộ, vì boundary nội bộ quyết định trực tiếp việc phân chia hai làn liền kề. Đây là nguyên nhân chính làm kết quả trên video ban đêm thấp hơn hai video ban ngày.

Trong điều kiện ánh sáng yếu, phản xạ mặt đường và ánh sáng đèn phương tiện tạo ra nhiều vùng sáng không thuộc vạch làn. Các vùng này có thể hình thành peak giả trong histogram hoặc làm giảm mức hỗ trợ của candidate thật. Vì vậy, khả năng xử lý ban đêm vẫn là hạn chế chính của lane detector hiện tại.

Khi lane model đầy đủ và đạt trạng thái khóa, độ chính xác gán phương tiện vào làn đạt 91,33% trên V1 và 98,00% trên V2. Kết quả này cho thấy việc kết hợp điểm bottom-center, boundary trong không gian BEV, lane history và lane-change streak giúp duy trì Lane ID tương đối ổn định.

Các trường hợp gán sai chủ yếu xuất hiện khi phương tiện nằm gần boundary, đang thực hiện chuyển làn hoặc bounding box không bao phủ chính xác vùng tiếp xúc với mặt đường. Lane history giúp hạn chế việc Lane ID thay đổi do dao động trong một vài frame, nhưng không thể khắc phục hoàn toàn lỗi hình học khi mô hình làn bị lệch hoặc thiếu boundary.

Hệ thống đã triển khai được logic phát hiện phương tiện đi sai làn và

hỗ trợ xác định phương tiện đi ngược chiều. Vi phạm không được xác nhận từ một frame đơn lẻ mà phải duy trì qua số frame yêu cầu. Cách xử lý này giúp giảm cảnh báo sai do nhãn lớp, Lane ID hoặc hướng chuyển động dao động tạm thời.

Tuy nhiên, kết quả phát hiện vi phạm hiện mới được đánh giá định tính do chưa có ground truth đầy đủ ở cấp độ Track ID. Vì vậy, kết quả trong chương này chứng minh hệ thống đã thực hiện được đầy đủ luồng xử lý và tạo cảnh báo, nhưng chưa đủ cơ sở để kết luận định lượng về Precision, Recall hoặc F1-score của kết quả nhận biết vi phạm end-to-end.

Về hiệu năng, trên môi trường phát triển sử dụng GPU NVIDIA Tesla T4, module phát hiện có tổng thời gian tiền xử lý, suy luận và hậu xử lý xấp xỉ 22,3 ms/ảnh, tương ứng thông lượng lý thuyết khoảng 44,8 FPS. Tài nguyên của môi trường này cho phép chạy đồng thời YOLO11n, ByteTrack, lane detector, lane assignment và logic phân tích vi phạm với độ ổn định cao hơn.

Khi chuyển sang Raspberry Pi 5, hiệu năng giảm rõ rệt do giới hạn về khả năng tính toán, băng thông bộ nhớ và tài nguyên hệ thống. Ở kích thước 640×640 pixel, phiên bản YOLO11n thông thường chỉ đạt 3,79 FPS, trong khi phiên bản OpenVINO INT8 đạt 8,68 FPS. Mặc dù thấp hơn môi trường GPU, kết quả này cho thấy Raspberry Pi 5 vẫn có khả năng thực thi mô hình ở kích thước đầu vào đầy đủ.

Lượng tử hóa OpenVINO INT8 đem lại mức cải thiện rõ rệt trên Raspberry Pi 5. Tại kích thước 224×224 pixel, tốc độ tăng từ 14,85 FPS lên 30,17 FPS, tương ứng hệ số tăng tốc khoảng 2,03 lần. Tại kích thước 640×640 pixel, tốc độ tăng từ 3,79 FPS lên 8,68 FPS, tương ứng khoảng

2,29 lần.

Cấu hình OpenVINO INT8 224×224 đạt tốc độ cao nhất và phù hợp với trường hợp ưu tiên khả năng xử lý liên tục trên thiết bị nhúng. Tuy nhiên, việc giảm kích thước ảnh có thể làm mất thông tin của các phương tiện nhỏ hoặc nằm xa camera. Ngược lại, cấu hình 640×640 giữ lại nhiều thông tin ảnh hơn nhưng có chi phí xử lý cao và tốc độ thấp hơn.

Như vậy, hệ thống thể hiện hai mức vận hành rõ ràng. Trên máy tính phát triển, pipeline đầy đủ có thể vận hành ổn định và phù hợp với quá trình kiểm thử, phân tích và xử lý video. Trên Raspberry Pi 5, module phát hiện vẫn có khả năng triển khai, nhưng cần lựa chọn kích thước đầu vào và định dạng mô hình phù hợp để cân bằng giữa tốc độ xử lý và chất lượng thông tin ảnh.

Các giá trị FPS trên Raspberry Pi 5 hiện phản ánh riêng module phát hiện phương tiện. Khi tích hợp thêm ByteTrack, lane detector, lane assignment, thao tác hiển thị và logic xác nhận vi phạm, tốc độ của toàn bộ pipeline sẽ thấp hơn. Do đó, kết quả hiện tại chứng minh tính khả thi của hướng triển khai nhúng, đồng thời cho thấy cần tiếp tục tối ưu lịch xử lý, giảm tần suất cập nhật lane, hạn chế thao tác vẽ và lựa chọn backend suy luận phù hợp.

Nhìn chung, hệ thống hoạt động tốt nhất trong điều kiện camera cố định, ban ngày, vạch làn còn quan sát được và hai lớp phương tiện phổ biến là motorbike và car. Các kết quả thực nghiệm cho thấy pipeline đề xuất có khả năng vận hành từ đầu vào video đến tạo cảnh báo vi phạm, đồng thời có thể chuyển sang Raspberry Pi 5 với cấu hình tối ưu. Những hạn chế còn lại tập trung vào dữ liệu ban đêm, các lớp truck và bus, khả năng khôi phục

boundary nội bộ và hiệu năng của toàn pipeline trên thiết bị nhúng.

4.4 TỔNG KẾT CHƯƠNG

Chương này đã trình bày kết quả đánh giá các thành phần chính của hệ thống nhận biết phương tiện đi sai làn và đi ngược chiều từ video giao thông của camera giám sát cố định. Các kết quả được phân tích theo từng module gồm phát hiện phương tiện, theo dõi đa đối tượng, nhận diện cấu trúc làn, gán phương tiện vào làn, xác định vi phạm và khảo sát hiệu năng triển khai.

Mô hình YOLO11n đạt Precision trung bình bằng 0,780, Recall bằng 0,634, F1-score bằng 0,700, mAP@0.5 bằng 0,705 và mAP@0.5:0.95 bằng 0,494 trên tập validation gồm 167 ảnh với tổng cộng 5.933 đối tượng. Các đường cong loss và chỉ số đánh giá cho thấy mô hình hội tụ tương đối ổn định trong phạm vi 40 epoch và chưa xuất hiện dấu hiệu overfitting nghiêm trọng.

Trong bốn lớp phương tiện, lớp car đạt kết quả tốt nhất với mAP@0.5 bằng 0,840, tiếp theo là motorbike với 0,820. Hai lớp truck và bus có kết quả thấp hơn, chủ yếu do số lượng mẫu đánh giá còn hạn chế và các đối tượng có thể xuất hiện với kích thước nhỏ hoặc bị che khuất trong khung hình.

ByteTrack duy trì đúng Track ID cho khoảng 92,7% số phương tiện được quan sát thủ công. Kết quả này cho phép hệ thống lưu lịch sử vị trí, Lane ID, hướng chuyển động và trạng thái vi phạm của phần lớn phương tiện trong vùng ROI. Các lỗi tracking chủ yếu xuất hiện khi phương tiện bị che khuất, di chuyển sát nhau hoặc detector bỏ sót đối tượng trong nhiều

frame liên tiếp.

Đối với lane detector, candidate refinement làm giảm sai số boundary từ 66,8% đến 79,8% trên ba video thử nghiệm. Trên V1 và V2, hệ thống phát hiện trực tiếp bốn boundary nội bộ và suy luận thêm hai boundary ngoài, từ đó tạo đúng sáu boundary cuối cùng và năm vùng làn. Mô hình làn đạt trạng thái khóa với score lần lượt là 0,98 và 0,81, cho thấy cấu trúc làn được duy trì ổn định trong điều kiện ban ngày.

Đối với V3, một boundary nội bộ không được phát hiện ổn định trong điều kiện ban đêm. Mặc dù hai boundary ngoài vẫn được suy luận, mô hình cuối cùng chỉ có năm boundary và tạo được bốn trong năm vùng làn tham chiếu. Do cấu trúc làn chưa đầy đủ và model score chỉ đạt 0,61, hệ thống không chuyển sang trạng thái khóa. Kết quả này cho thấy khả năng xử lý ban đêm và khôi phục boundary nội bộ vẫn là hạn chế chính của phương pháp hiện tại.

Khi mô hình làn đầy đủ và đạt trạng thái khóa, độ chính xác gán phương tiện vào làn đạt 91,33% trên V1 và 98,00% trên V2. Việc kết hợp điểm bottom-center, boundary trong không gian Bird's-Eye View, lane history và lane-change streak giúp Lane ID của phương tiện được duy trì tương đối ổn định qua nhiều frame.

Hệ thống đã tích hợp được logic phát hiện phương tiện đi sai làn và hỗ trợ xác định phương tiện đi ngược chiều. Cơ chế xác nhận vi phạm qua nhiều frame giúp giảm cảnh báo sai do nhãn lớp, Lane ID hoặc hướng chuyển động dao động tạm thời. Tuy nhiên, kết quả vi phạm hiện mới được đánh giá định tính do chưa có ground truth đầy đủ ở cấp độ Track ID, vì vậy chưa thể kết luận định lượng về độ chính xác end-to-end của hệ thống.

Trên môi trường phát triển sử dụng GPU NVIDIA Tesla T4, module phát hiện có tổng thời gian tiền xử lý, suy luận và hậu xử lý xấp xỉ 22,3 ms/ảnh, tương ứng thông lượng lý thuyết khoảng 44,8 FPS. Tài nguyên của môi trường này cho phép chạy đồng thời các module của pipeline với độ ổn định cao hơn, phù hợp cho quá trình kiểm thử, phân tích và xử lý video.

Khi chuyển sang Raspberry Pi 5, hiệu năng thấp hơn môi trường máy tính phát triển nhưng module phát hiện vẫn có khả năng thực thi. Phiên bản OpenVINO INT8 tại kích thước 224×224 đạt 30,17 FPS, trong khi phiên bản INT8 tại kích thước 640×640 đạt 8,68 FPS. So với phiên bản chưa lượng tử hóa, OpenVINO INT8 giúp tăng tốc khoảng 2,03 lần ở kích thước 224×224 và 2,29 lần ở kích thước 640×640 .

Các kết quả trên cho thấy việc giảm kích thước đầu vào và sử dụng lượng tử hóa INT8 là hai giải pháp có hiệu quả rõ rệt khi triển khai mô hình trên thiết bị nhúng. Cấu hình INT8 224×224 phù hợp khi ưu tiên tốc độ xử lý, trong khi cấu hình INT8 640×640 giữ lại nhiều thông tin ảnh hơn nhưng có chi phí tính toán lớn hơn. Các giá trị FPS này chỉ phản ánh riêng module phát hiện và không đại diện cho tốc độ của toàn bộ pipeline.

Nhìn chung, hệ thống đã hoàn thành việc xây dựng và tích hợp các thành phần chính từ phát hiện phương tiện đến tạo cảnh báo vi phạm. Hệ thống hoạt động ổn định hơn trong điều kiện ban ngày, camera cố định và vạch làn còn quan sát được; đồng thời thể hiện tính khả thi khi chuyển sang Raspberry Pi 5 với cấu hình mô hình phù hợp. Các hướng cải thiện tiếp theo tập trung vào mở rộng dữ liệu truck và bus, tăng khả năng xử lý ban đêm, khôi phục boundary nội bộ, xây dựng ground truth vi phạm và tối ưu hiệu năng của toàn bộ pipeline trên thiết bị nhúng.

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 KẾT LUẬN

Trong bối cảnh số lượng phương tiện giao thông ngày càng gia tăng, việc tự động phân tích video từ camera giám sát có ý nghĩa quan trọng đối với công tác quản lý, điều tiết và bảo đảm an toàn giao thông. Phương pháp giám sát thủ công phụ thuộc nhiều vào nhân lực, khó xử lý liên tục khối lượng video lớn và có thể bỏ sót các hành vi vi phạm. Bài toán này càng trở nên phức tạp trong điều kiện giao thông hỗn hợp tại Việt Nam, nơi xe máy chiếm tỷ lệ cao, các phương tiện thường xuyên che khuất nhau và việc tổ chức làn đường không đồng nhất giữa các vị trí quan sát.

Trên cơ sở đó, luận văn đã nghiên cứu và xây dựng hệ thống nhận biết phương tiện đi sai làn và đi ngược chiều từ video giao thông của camera giám sát cố định. Hệ thống được tổ chức thành một pipeline thống nhất, gồm thu nhận khung hình, phát hiện phương tiện, theo dõi đa đối tượng, nhận diện cấu trúc làn đường, gán phương tiện vào làn, xác định hướng chuyển động và xác nhận trạng thái vi phạm qua nhiều khung hình. Bên cạnh khả năng hoạt động trên máy tính phát triển, hệ thống còn được khảo sát theo hướng triển khai trên Raspberry Pi 5.

Đối với nhiệm vụ phát hiện phương tiện, mô hình YOLO11n được huấn luyện với bốn lớp gồm motorbike, car, truck và bus. Mô hình có khoảng 2,58 triệu tham số và khối lượng tính toán khoảng 6,3 GFLOPs tại kích thước đầu vào 640×640 pixel. Cấu trúc tương đối nhỏ gọn của

YOLO11n phù hợp với yêu cầu cân bằng giữa chất lượng phát hiện và chi phí tính toán của hệ thống.

Quá trình huấn luyện được thực hiện trong 40 epoch. Các thành phần loss trên tập huấn luyện và tập validation đều giảm nhanh trong giai đoạn đầu, sau đó giảm chậm dần và tiến tới trạng thái tương đối ổn định. Tổng train loss và validation loss ở giai đoạn cuối lần lượt xấp xỉ 2,55 và 2,59. Khoảng cách giữa hai giá trị không lớn và không xuất hiện xu hướng phân kỳ rõ rệt, cho thấy mô hình chưa có dấu hiệu overfitting nghiêm trọng trong phạm vi cấu hình thực nghiệm.

Trên tập validation gồm 167 ảnh với tổng cộng 5.933 đối tượng, mô hình đạt Precision trung bình bằng 0,780, Recall bằng 0,634, F1-score bằng 0,700, mAP@0.5 bằng 0,705 và mAP@0.5:0.95 bằng 0,494. Kết quả này cho thấy YOLO11n đạt chất lượng phát hiện ở mức khá và có thể cung cấp dữ liệu đầu vào cho các module tracking và phân tích giao thông phía sau.

Trong bốn lớp phương tiện, lớp car đạt kết quả tốt nhất với mAP@0.5 bằng 0,840 và mAP@0.5:0.95 bằng 0,615. Lớp motorbike đạt mAP@0.5 bằng 0,820 và mAP@0.5:0.95 bằng 0,507. Đây là kết quả tương đối tốt trong bối cảnh xe máy thường có kích thước nhỏ, xuất hiện với mật độ cao và dễ bị che khuất.

Hai lớp truck và bus có kết quả thấp hơn, với mAP@0.5 lần lượt bằng 0,633 và 0,528. Nguyên nhân chính là số lượng mẫu của hai lớp trong tập validation còn hạn chế, chỉ gồm 46 đối tượng truck và 60 đối tượng bus. Ngoài ra, các phương tiện này có thể xuất hiện ở khoảng cách xa, bị che khuất hoặc có hình dạng quan sát tương tự nhau trong một số góc camera.

Đối với nhiệm vụ theo dõi đa đối tượng, ByteTrack được sử dụng để

liên kết các detection qua nhiều frame và duy trì Track ID cho từng phương tiện. Kết quả đánh giá thủ công trên ba video cho thấy tracker duy trì đúng Track ID cho khoảng 92,7% số phương tiện được quan sát. Kết quả này cho phép hệ thống lưu lịch sử vị trí, Lane ID, hướng chuyển động và trạng thái vi phạm của phần lớn phương tiện trong vùng ROI.

Các trường hợp mất track hoặc thay đổi Track ID chủ yếu xuất hiện khi nhiều phương tiện di chuyển sát nhau, phương tiện bị che khuất hoàn toàn hoặc detector bỏ sót trong nhiều frame liên tiếp. Khi Track ID bị thay đổi, lịch sử của phương tiện có thể bị khởi tạo lại, từ đó ảnh hưởng đến việc bỏ phiếu Lane ID, xác định hướng và xác nhận vi phạm theo thời gian.

Một nội dung trọng tâm của luận văn là xây dựng thuật toán nhận diện cấu trúc làn đường phù hợp với video camera cố định. Vùng mặt đường được chuyển sang không gian Bird's-Eye View nhằm giảm ảnh hưởng của phối cảnh và làm cho khoảng cách giữa các boundary ổn định hơn theo chiều sâu. Các nguồn bằng chứng gồm độ sáng, màu sắc, tương phản và gradient được kết hợp để tạo ảnh nhị phân phục vụ quá trình tìm kiếm vạch làn.

Thay vì chỉ tính histogram tại phần dưới của ảnh BEV, hệ thống chia ảnh thành nhiều dải chiều sâu và tìm peak độc lập tại từng dải. Các peak có tính liên tục về vị trí được liên kết thành candidate boundary. Cách tiếp cận này giúp giảm phụ thuộc vào vùng đáy ảnh, nơi thường có nhiều phương tiện che khuất hoặc chứa các cạnh nhiễu không thuộc vạch làn.

Tập điểm của mỗi candidate được mô hình hóa bằng đa thức bậc hai. Sau lần khớp ban đầu, hệ thống tạo một hành lang tìm kiếm quanh đường cong, thu thập lại các pixel có khả năng thuộc vạch sơn và thực hiện

candidate refinement. Kết quả thực nghiệm cho thấy refinement làm giảm sai số boundary từ 66,8% đến 79,8% trên ba video thử nghiệm. Điều này chứng minh bước tinh chỉnh giúp đường cong cuối cùng bám sát vạch làn tốt hơn so với việc chỉ sử dụng các peak ban đầu.

Bộ lựa chọn mô hình không đánh giá độc lập từng candidate mà xem xét đồng thời các tổ hợp boundary. Các tiêu chí bao gồm mức hỗ trợ của pixel vạch, độ bao phủ theo chiều sâu, sai số khớp, chất lượng hình học, thứ tự trái-phải, chiều rộng làn và khả năng giao nhau giữa các boundary. Các thành phần penalty được áp dụng đối với candidate sát mép, boundary có chất lượng thấp hoặc tổ hợp không tạo thành cấu trúc làn hợp lý.

Đối với hai video ban ngày V1 và V2, hệ thống phát hiện trực tiếp bốn boundary nội bộ và suy luận thêm hai boundary ngoài. Sáu boundary cuối cùng tạo thành năm vùng làn, phù hợp với cấu trúc tham chiếu. Hai boundary ngoài được suy luận từ vị trí của các boundary nội bộ, chiều rộng làn ước lượng và giới hạn vùng mặt đường. Cơ chế này cần thiết do mép đường không phải lúc nào cũng có vạch sơn rõ ràng hoặc có thể nằm sát biên ROI.

Trên V1 và V2, mô hình làn đạt trạng thái khóa với score lần lượt bằng 0,98 và 0,81. Cơ chế temporal consensus giúp tổng hợp kết quả qua nhiều lần cập nhật, hạn chế số lượng và vị trí boundary thay đổi liên tục giữa các frame. Sau khi khóa, mô hình làn ổn định được tái sử dụng và chỉ cập nhật định kỳ, phù hợp với đặc điểm của camera giám sát cố định.

Đối với video ban đêm V3, hệ thống chỉ phát hiện trực tiếp ba trong bốn boundary nội bộ. Mặc dù hai boundary ngoài vẫn được suy luận, mô hình cuối cùng chỉ có năm boundary và tạo được bốn trong năm vùng làn

tham chiếu. Model score chỉ đạt 0,61 nên hệ thống không chuyển sang trạng thái khóa.

Kết quả trên cho thấy cơ chế suy luận boundary ngoài có thể bổ sung mép đường không quan sát rõ nhưng không thể thay thế một boundary nội bộ bị mất. Boundary nội bộ quyết định trực tiếp việc phân chia hai làn liền kề. Vì vậy, khi một boundary nội bộ không được phát hiện, cấu trúc làn cuối cùng sẽ không đầy đủ.

Nguyên nhân chính làm giảm chất lượng nhận diện làn trong video ban đêm là độ tương phản thấp, phản xạ mặt đường và ánh sáng đèn phương tiện. Các yếu tố này có thể tạo peak giả hoặc làm giảm mức hỗ trợ của candidate thật. Do đó, hệ thống hiện hoạt động ổn định hơn trong điều kiện ban ngày, khi vạch làn còn quan sát được và cấu trúc đường thẳng hoặc cong nhẹ.

Quá trình gán phương tiện vào làn được thực hiện bằng cách biến đổi điểm bottom-center của bounding box sang không gian BEV và so sánh vị trí này với các boundary tại cùng tọa độ chiều sâu. Lane history và lane-change streak được sử dụng để hạn chế Lane ID dao động khi phương tiện nằm gần đường biên hoặc đang thực hiện chuyển làn.

Khi mô hình làn đầy đủ và đạt trạng thái khóa, độ chính xác gán làn đạt 91,33% trên V1 và 98,00% trên V2. Kết quả này cho thấy sự kết hợp giữa bottom-center, mô hình boundary trong không gian BEV và lịch sử Lane ID có khả năng duy trì kết quả gán làn tương đối ổn định qua nhiều frame.

Vi phạm sai làn được xác định bằng cách so sánh lớp phương tiện với tập lớp được phép lưu thông trong làn hiện tại. Vi phạm đi ngược chiều

được phân tích dựa trên lịch sử quỹ đạo và hướng lưu thông hợp lệ của làn. Hệ thống không xác nhận vi phạm từ một frame đơn lẻ mà yêu cầu trạng thái vi phạm tồn tại đủ số frame quy định. Cơ chế này giúp hạn chế cảnh báo sai do nhãn lớp, Lane ID hoặc hướng chuyển động dao động tạm thời.

Hệ thống đã triển khai được đầy đủ luồng xử lý từ phát hiện, tracking, gán làn đến tạo cảnh báo vi phạm. Tuy nhiên, kết quả vi phạm hiện mới được đánh giá định tính do chưa có ground truth đầy đủ ở cấp độ Track ID. Vì vậy, kết quả thực nghiệm chứng minh khả năng hoạt động của pipeline nhưng chưa đủ cơ sở để kết luận định lượng về Precision, Recall hoặc F1-score của kết quả phát hiện vi phạm end-to-end.

Về hiệu năng, trên môi trường sử dụng GPU NVIDIA Tesla T4, module phát hiện có thời gian tiền xử lý, suy luận và hậu xử lý trung bình lần lượt bằng 1,0 ms, 4,6 ms và 16,7 ms trên mỗi ảnh. Tổng thời gian của riêng module phát hiện xấp xỉ 22,3 ms/ảnh, tương ứng thông lượng lý thuyết khoảng 44,8 FPS. Tài nguyên của môi trường máy tính phát triển cho phép chạy đồng thời các thành phần của pipeline với độ ổn định cao hơn.

Khi chuyển sang Raspberry Pi 5, hiệu năng thấp hơn do giới hạn về năng lực xử lý, băng thông bộ nhớ và tài nguyên phần cứng. Ở kích thước 224×224 pixel, phiên bản YOLO11n thông thường đạt 14,85 FPS, trong khi phiên bản OpenVINO INT8 đạt 30,17 FPS. Lượng tử hóa giúp tăng tốc khoảng 2,03 lần và giảm thời gian suy luận từ 60,69 ms xuống 23,60 ms trên mỗi ảnh.

Ở kích thước 640×640 pixel, phiên bản thông thường đạt 3,79 FPS, trong khi phiên bản OpenVINO INT8 đạt 8,68 FPS. Phiên bản INT8 tăng tốc khoảng 2,29 lần và giảm thời gian suy luận từ 250,42 ms xuống 98,57

ms trên mỗi ảnh.

Kết quả cho thấy cấu hình OpenVINO INT8 224×224 phù hợp khi ưu tiên tốc độ xử lý trên thiết bị nhúng. Cấu hình OpenVINO INT8 640×640 có tốc độ thấp hơn nhưng giữ lại nhiều thông tin ảnh hơn, đặc biệt đối với các phương tiện nhỏ hoặc nằm xa camera. Các giá trị FPS này chỉ phản ánh module phát hiện phương tiện và không đại diện cho tốc độ của toàn bộ pipeline trên Raspberry Pi 5.

Nhìn chung, luận văn đã hoàn thành mục tiêu xây dựng và tích hợp các thành phần chính của hệ thống nhận biết phương tiện đi sai làn và đi ngược chiều. Hệ thống hoạt động tốt trên máy tính phát triển và đạt kết quả ổn định hơn trong điều kiện ban ngày, camera cố định và vạch làn còn quan sát rõ. Kết quả trên Raspberry Pi 5 cho thấy mô hình vẫn có khả năng triển khai trên thiết bị nhúng khi lựa chọn cấu hình đầu vào và định dạng suy luận phù hợp.

Những đóng góp chính của luận văn gồm xây dựng bộ dữ liệu phương tiện giao thông bốn lớp, huấn luyện và tích hợp YOLO11n, sử dụng ByteTrack để duy trì lịch sử phương tiện, phát triển phương pháp phát hiện peak theo nhiều dải chiều sâu, liên kết candidate, refinement đường cong, lựa chọn mô hình dựa trên topology, suy luận boundary ngoài và ổn định mô hình làn bằng temporal consensus. Trên cơ sở đó, hệ thống thực hiện gán phương tiện vào làn và xác nhận hành vi vi phạm ở cấp độ track thay vì đưa ra kết luận từ từng frame độc lập.

5.2 HƯỚNG PHÁT TRIỂN

Trên cơ sở các kết quả đã đạt được và những hạn chế còn tồn tại, hệ thống cần tiếp tục được phát triển theo các hướng liên quan đến dữ liệu, thuật toán nhận diện làn, tracking, đánh giá vi phạm và tối ưu triển khai trên thiết bị nhúng.

Trước hết, bộ dữ liệu phát hiện phương tiện cần được mở rộng và cân bằng hơn giữa các lớp. Số lượng mẫu truck và bus hiện thấp hơn đáng kể so với motorbike và car, làm cho chất lượng phát hiện của hai lớp này chưa ổn định. Dữ liệu mới cần bao phủ nhiều góc nhìn, khoảng cách, điều kiện chiếu sáng, mật độ giao thông và mức độ che khuất khác nhau. Các trường hợp phương tiện nhỏ ở xa camera và phương tiện bị che khuất một phần cần được ưu tiên bổ sung.

Bên cạnh việc thu thập thêm dữ liệu, có thể áp dụng các kỹ thuật tăng cường dữ liệu có xét đến phân bố lớp, oversampling đối với lớp ít mẫu hoặc điều chỉnh trọng số hàm mất mát. Tuy nhiên, cần bảo đảm các ảnh tăng cường vẫn phản ánh đúng đặc điểm của video giao thông thực tế và không tạo ra các mẫu không hợp lý về hình học hoặc ánh sáng.

Khả năng nhận diện làn trong điều kiện ban đêm cũng cần được cải thiện. Các kỹ thuật tăng tương phản như CLAHE có thể làm rõ vạch làn nhưng đồng thời khuếch đại texture mặt đường, vùng phản xạ và ánh sáng đèn xe. Một hướng phát triển là xây dựng cơ chế tiền xử lý thích nghi theo điều kiện chiếu sáng, trong đó các ngưỡng màu sắc, gradient và morphology được điều chỉnh riêng cho chế độ ngày và đêm.

Ngoài phương pháp xử lý ảnh truyền thống, có thể tích hợp mô hình lane segmentation như một nguồn bằng chứng bổ sung. Kết quả segmentation

không nhất thiết thay thế hoàn toàn pipeline hiện tại mà có thể được kết hợp với histogram, candidate linking và topology-aware model selection. Cách tiếp cận lại có thể giúp duy trì khả năng giải thích của thuật toán hiện tại đồng thời cải thiện độ ổn định trong điều kiện vạch mờ hoặc ánh sáng phức tạp.

Mô hình đa thức bậc hai hiện phù hợp với đoạn đường thẳng và đường cong nhẹ, nhưng còn hạn chế ở vùng nhập làn, tách làn, giao lộ hoặc đường cong mạnh. Trong tương lai, có thể sử dụng spline, B-spline, đường cong Bezier hoặc mô hình piecewise polynomial để biểu diễn boundary theo từng đoạn. Mô hình cấu trúc làn cũng có thể được mở rộng sang dạng đồ thị nhằm mô tả rõ quan hệ nhập, tách và kết nối giữa các làn.

Cơ chế suy luận boundary ngoài hiện dựa trên chiều rộng làn và hình học của các boundary nội bộ. Phương pháp này hoạt động tốt khi các boundary nội bộ được phát hiện đầy đủ nhưng không thể khôi phục một boundary phân chia làn bị mất. Vì vậy, cần xây dựng cơ chế đánh giá và phục hồi boundary nội bộ dựa trên lịch sử nhiều frame, khoảng cách làn dự kiến, hướng quỹ đạo phương tiện và các candidate có độ tin cậy thấp.

Cơ chế temporal consensus và lane locking cần được bổ sung khả năng tự động tái hiệu chỉnh. Sau khi khóa, hệ thống nên tiếp tục theo dõi các chỉ số như paint support, boundary drift, lane width và mức phù hợp giữa Lane ID với quỹ đạo phương tiện. Khi chất lượng mô hình giảm dưới ngưỡng hoặc camera bị rung, hệ thống có thể tự động chuyển trở lại giai đoạn calibration thay vì tiếp tục sử dụng mô hình không còn phù hợp.

Đối với tracking, cần cải thiện khả năng duy trì Track ID trong cảnh giao thông đông và che khuất mạnh. Có thể khảo sát các tracker như BoT-

SORT hoặc DeepSORT, trong đó đặc trưng ngoại hình được sử dụng để hỗ trợ liên kết khi thông tin chuyển động và IoU không còn đủ tin cậy. Tuy nhiên, chi phí tính toán của đặc trưng Re-Identification cần được đánh giá kỹ trước khi triển khai trên Raspberry Pi 5.

Phương pháp xác định hướng chuyển động cũng cần được mở rộng cho đoạn đường cong. Thay vì chỉ sử dụng thay đổi tọa độ theo trục dọc của ảnh, vector chuyển động của phương tiện có thể được chiếu lên tiếp tuyến của lane centerline tại vị trí hiện tại. Việc so sánh vector vận tốc với hướng hình học của làn sẽ giúp phát hiện đi ngược chiều chính xác hơn trong các đoạn đường có độ cong lớn.

Một hướng phát triển quan trọng khác là xây dựng bộ ground truth đầy đủ cho toàn bộ pipeline. Dữ liệu cần được gán nhãn Track ID theo định dạng MOT, vị trí boundary tại nhiều mức chiều sâu, Lane ID của từng phương tiện và trạng thái vi phạm theo track. Bộ dữ liệu này sẽ cho phép đánh giá tracking bằng MOTA, IDF1 hoặc HOTA; đánh giá lane detection bằng sai số boundary; đồng thời tính Precision, Recall và F1-score cho kết quả nhận biết vi phạm end-to-end.

Đối với Raspberry Pi 5, bước phát triển tiếp theo không chỉ tập trung vào module YOLO11n mà cần benchmark toàn bộ pipeline. Quá trình đánh giá nên đo riêng thời gian của detection, tracking, lane detection, lane assignment, phân tích vi phạm và thao tác hiển thị. Các thông số CPU, RAM, nhiệt độ, FPS trung bình, độ trễ trung vị và độ trễ P95 cũng cần được ghi nhận để đánh giá tính ổn định trong thời gian chạy dài.

Việc sử dụng OpenVINO INT8 đã cho thấy hiệu quả tăng tốc rõ rệt. Trong giai đoạn tiếp theo, có thể tiếp tục tối ưu bằng cách giảm tần suất cập

nhập lane detector sau khi mô hình làn đã khóa, xử lý detection cách frame, tái sử dụng bộ nhớ, hạn chế sao chép ảnh, tắt các cửa sổ debug và chỉ ghi lại những frame có sự kiện. Cần khảo sát mức ảnh hưởng của các giải pháp này đến độ chính xác và độ trễ của toàn bộ hệ thống.

Hệ thống cũng cần được mở rộng từ xử lý video lưu sẵn sang luồng camera trực tiếp. Khi triển khai thực tế, chương trình phải xử lý thêm các vấn đề như mất kết nối, thay đổi tốc độ khung hình, độ trễ mạng, frame bị lỗi và đồng bộ thời gian. Một giao diện giám sát có thể được xây dựng để hiển thị video, Track ID, Lane ID, loại vi phạm, thời gian xảy ra và ảnh bằng chứng.

Cuối cùng, hệ thống có thể được mở rộng để nhận biết thêm các hành vi như chuyển làn không đúng quy định, dừng đỗ sai vị trí, đi vào làn khẩn cấp, vượt đèn đỏ hoặc không tuân thủ biển báo. Việc kết hợp thông tin từ đèn tín hiệu, biển báo, bản đồ làn và nhiều camera sẽ tạo tiền đề xây dựng một hệ thống giám sát giao thông thông minh có khả năng phân tích toàn diện hơn.

Nhìn chung, kết quả của luận văn đã tạo được nền tảng cho một hệ thống giám sát vi phạm giao thông tự động dựa trên thị giác máy tính. Việc tiếp tục hoàn thiện dữ liệu, tăng khả năng xử lý ban đêm, cải thiện tracking và lane topology, xây dựng ground truth end-to-end và tối ưu toàn bộ pipeline trên Raspberry Pi 5 sẽ giúp hệ thống tiến gần hơn đến khả năng triển khai trong môi trường giao thông thực tế.

TÀI LIỆU THAM KHẢO

- [1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*. Pearson, 4th ed., 2018.
- [2] Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, “Object detection in 20 years: A survey,” *Proceedings of the IEEE*, vol. 111, no. 3, pp. 257–276, 2023.
- [3] R. Girshick, J. Donahue, T. Darrell, and J. Malik, “Rich feature hierarchies for accurate object detection and semantic segmentation,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 580–587, 2014.
- [4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788, 2016.
- [5] Ultralytics Team, “Ultralytics YOLO11 model architecture and usage guide,” 2024.
- [6] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [7] H. W. Kuhn, “The hungarian method for the assignment problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.

- [8] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, “ByteTrack: Multi-object tracking by associating every detection box,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 1–21, 2022.
- [9] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2nd ed., 2004.
- [10] J. F. Canny, “A computational approach to edge detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 8, no. 6, pp. 679–698, 1986.
- [11] G. A. F. Seber and A. J. Lee, *Linear Regression Analysis*. John Wiley & Sons, 2003.